

软件工程的来龙去脉 [05-Raw/26生成式软件工程/NJU]

原视频: <https://www.bilibili.com/video/BV1Nyeq6qEt8> · 课程合集:
<https://space.bilibili.com/202224425/lists/8942017> · 讲者: 蒋炎岩 (NJU 《生成式软件工程》2026)

全课主线

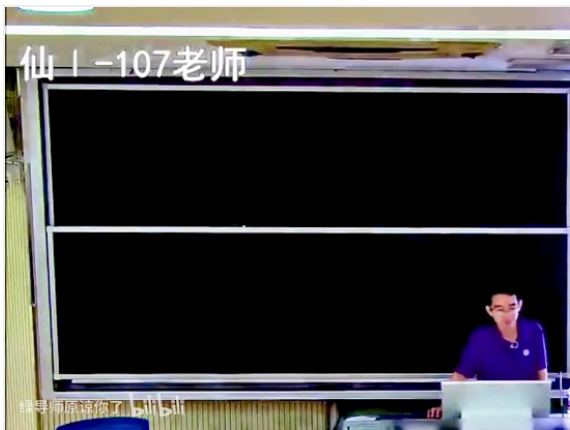
讲者想让听众相信: AI/LLM 与 scaling law 正把实时决策、软件生产和软件公司形态推到剧变, 传统软件公司、专用软件和 App 会被根本重估甚至消亡。但不能因此放弃理解; 软件工程历史中形成的 first principles 没有消失, 反而应被用来指导与 agent 协作, 成为 agent-native 软件开发者。在斩杀线上移的竞争中, 能理解并驾驭 AI 的人才能活下来并搭上最后一班车。

1. 从版本管理进入正式软件工程, 并用 TinyC AI 实时玩 DOOM/System 1 的新闻说明实时决策小模型打开了实时人机交互与“人感”赛道。(第 1、2 部分)
2. 讲者用从感知机、寒冬、1997、AlexNet、Transformer/BERT、GPT-3、多模态、Sora、o1 到近期的时间线, 论证 AI 进化与世界发展在加速; 因此就业挑战与机遇同时放大, 变化时间表不确定。(第 2、3 部分)
3. Noam Brown 说当前 AI 短板是“品味”, 即对下一步该做什么的直觉, 因为模型被训成 prompt 的舔狗并有 reward hacking 倾向; 讲者回应没有根本理由不能用更多合成数据/CoT 教模型想得更远, 且可验证与不可验证的边界没有想象中清晰。(第 3 部分)
4. 由此接上 The Bitter Lesson/scaling law 尚未停止: GPT-6 让讲者看到“数据分布工程”, 新 scaling law 是强基模加新领域强化学习后所有任务都更好; 因此基模厂竞争转向数据注入、工程飞轮、RSI 闭环和新领域插件化, 架构与算法不是瓶颈。(第 4、5、6 部分)
5. 这使学术界小规模论文与基模厂脱节、体系面临崩溃, 斩杀线不断上移; 但是讲者仍坚持要上课、不能完全依赖 AI, 并要把问题拉回软件构造本身。(第 5、6 部分)
6. 软件构造的根本难题是 intent、specification、implementation 之间有 gap, 所以才有软件工程; 在版本管理之后, 今天讲者给出的构造答案是 MVP: 少做少错但最接近最终

- 产品，作为 AI slop 低成本可丢弃，用来暴露 intent/spec/架构错误。(第 6、7 部分)
7. 标签打印/扫描与非标打印机案例显示，AI 可以按目标与可验收结果现场生成、试错、迭代，专用驱动和配套软件不再必要；因此软件形态变化，App 可能退化为 agent 调用的 API，软件公司与相关产业链被重估。(第 7、8、9 部分)
 8. 但讲者仍要讲软件工程：因为过去的软件工程不是这样，软件工程智慧没有消失；电子墨水屏例子说明 agent 已能解包 APK、逆向蓝牙协议驱动非标设备，但仍需用旧有理念指导与 agent 协作，成为 agent-native 软件开发者。(第 10 部分)
 9. 回到历史，1960 年代硬件贵、码农贵、工具不成熟，客户 intent 不确定，软件企业要补 implementation/spec/intent 的 gap；软件需求高度定制化，连接物理/社会/人的偏好，导致进度、成本、质量失控和软件危机。(第 10–13 部分)
 10. Dijkstra、Hamilton/Apollo 把答案推向防御性编程：程序 crash 后系统仍要工作，把“绝不可能”当成可能，这是软件工程起点；软件工程是在成本时间约束下交付可工作、可维护、可生存的软件，从个人英雄主义走向 checklist 等工程化方法。(第 13–15 部分)
 11. Royce 1970 文章被误读为瀑布模型，实际批判瀑布；测试是链条上第一个物理事件，文档无正确性保证，漂移到测试暴露后回改导致超期超支，所以补救本质是把发现问题提前；因此 design-first、do it twice/preliminary pilot program 就是 MVP，并与 TDD/敏捷互相映射。(第 16–20 部分)
 12. 用 data dependency 看，修改有代价且依赖会传递，越晚发现返工越大；所以要把依赖割小，先走通高优先级依赖的 MVP，再迭代扩展，越早反馈越压缩 gap；软件工程为各阶段造方法/工具，不同阶段需要不同语言，UML/DSL/checklist 等在无代码时做语义对齐，今天 UML 虽死但其 first principles 仍在，AI 时代只剩“没想清楚/没说清楚”的 gap，因此学生应追原文、自己做压缩，并用这些理念成为 agent-native 软件开发者。(第 21–30 部分) (整理者串联：这里把 data dependency、分阶段语言、语义对齐与个性化学习串成同一主线。)

讲者的结论/号召是：在 AI 加速、软件公司与 App 被重估的趋势下，要用软件工程的 first principles 理解并驾驭 AI，指导与 agent 的协作，成为 agent-native 软件开发者，以逃离不断上移的斩杀线。

仙一-107老师



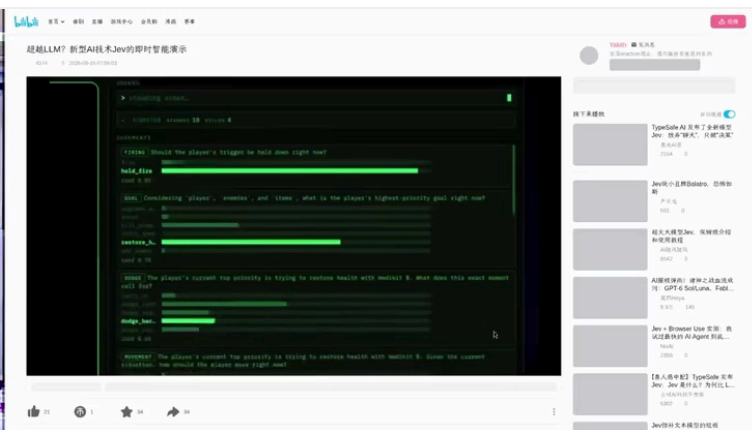
摧毁性的事情一直都在发生

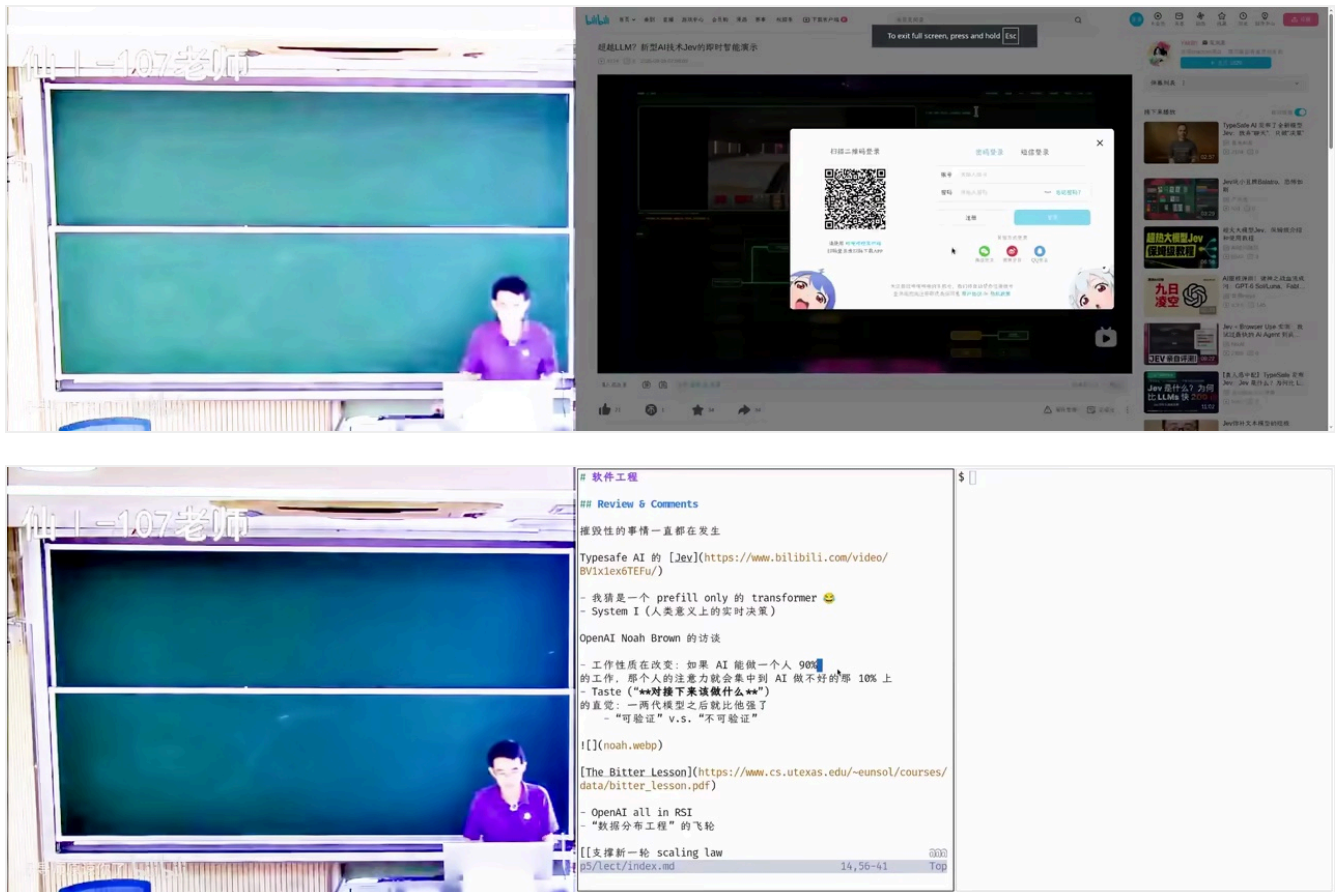
摧毁性的事情一直都在发生

- Typesafe AI 的 [Jev](#)
 - 我猜是一个 prefill only 的 transformer 😊
 - 官方披露：一次并行输出全部概率，无需逐 token 自回归生成。见 [技术介绍](#)。
 - System I (人类意义上的实时决策)

感受一下 AI 进化的速度

- 很难想象 Silicon Valley 里的 Pied Piper 到今天会怎么样
- 以及我们每个人在就业市场上面临的挑战





从版本管理进入正式软件工程

- 上节课：讲者讲了一点版本管理。
- 今天：正式进入**软件工程**部分；讲者称“现在算是真正的软件工程课了”。版本管理此前也属于软件工程。

课前新闻：实时玩 DOOM 的模型把“实时决策”推成新赛道

- 讲者先分享最近一周新闻；他认为热度已高到多数人可能都看过。
- 新闻：****TinyC AI**（转写为“type c AI”）****发布一个能实时玩 **DOOM（毁灭战士）**的基础模型。**
- 讲者猜测其架构：
 - 可能是 **prefill-only**（转写为“prefer only”；术语待核），类似“只能出一个 token 的 transformer”；
 - 也可能是别的架构。
- 讲者描述的运行方式：

1. 输入：把 DOOM 的状态变成 JSON 文件。
 2. 模型：根据这个状态给出一个决策。
 3. 输出：一个 **token**；速度在毫秒级或百毫秒级，基本是准人类实时。
 4. 这个 token 应是专门微调/训练过的；它不说人话，但能做决策。
- 讲者对它的定位：
 - 不是简单的大语言模型，也不是以前那种非常小的模型（如**决策树**）；
 - 它介于**大语言模型**和决策树这类小模型之间，但又是通用的，并且“真的可以干事”——能实际玩游戏。
 - 命名与口号：它叫 **System 1 (System One)**，标榜“真正人类意义上的实时决策”。
 - 讲者判断：这不是一个很新的技术；过去也有人试图做非常快的模型。但这次火了以后，它正式把这条赛道打开了。

实时决策小模型打开的是“实时人机交互”赛道

- 讲者提问：作为 **AI native** 时代的年轻人，如果一个模型能在 **100ms** 内做实时决策；乐观一点，如果模型裁得更小，能在 **50ms** 或更短时间内做实时决策，能用来干什么？
- 按讲者描述，可串成逻辑链：
 1. 有 **100ms、200ms** 的小决策模型，可做实时决策 →
 2. 可感知周围环境、实时响应 →
 3. 可实时生成数字人的**脸部微表情**等 →
 4. 得到非常有实时性、有“**人感**”的东西。
- 关于“人和人沟通 vs 人和机器沟通”的差别，讲者提到：
 - 最近流传的例子：和 AI 演员对话时，AI 演员突然开始说广东话；另外两个在英语对话时，也突然说广东话。
 - 讲者紧接着说，AI 生成的数字人与真人之间有很大的延迟。（**整理者串联：这个例子被他用来支持人机沟通仍有明显延迟/不自然**）
- 讲者认为：
 - 以前的大语言模型在那个延迟下也许有能力做，但做得不那么好；
 - 今天的小模型（例如能玩游戏的模型）可以做到，因此也能生成有“人感”的人。
- 讲者玩笑：它甚至可以哄小朋友睡觉，婴儿睡不着的难题就彻底解决了。
- 讲者结论：这开了一个非常大的赛道。

小结：讲者借 System 1 玩 DOOM 的例子，把“实时决策小模型”从旧尝试推成新赛道，落点是实时人机交互与“人感”。

第 2 部分

仙 I -107老师

绿导师原谅你了

Review & Comments

摧毁性的事情一直都在发生

- Typesafe AI 的 [Jev](#)
 - 我猜是一个 prefill only 的 transformer 😊
 - 官方披露：一次并行输出全部概率，无需逐 token 自回归生成。见 [技术介绍](#)。
 - System I (人类意义上的实时决策)

感受一下 AI 进化的速度

- 很难想象 Silicon Valley 里的 Pied Piper 到今天会怎么样
- 以及我们每个人在就业市场上面临的挑战

1 / 8



仙 I -107老师

绿导师原谅你了

智能纪元

00:00 / 01:00

智能纪元

00:00 / 01:00

智能纪元

00:00 / 01:00



仙 I -107老师

绿导师原谅你了

智能纪元

00:00 / 01:00

智能纪元

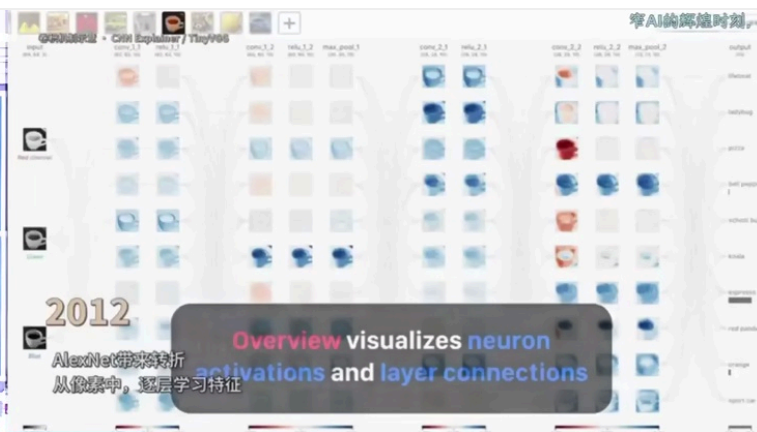
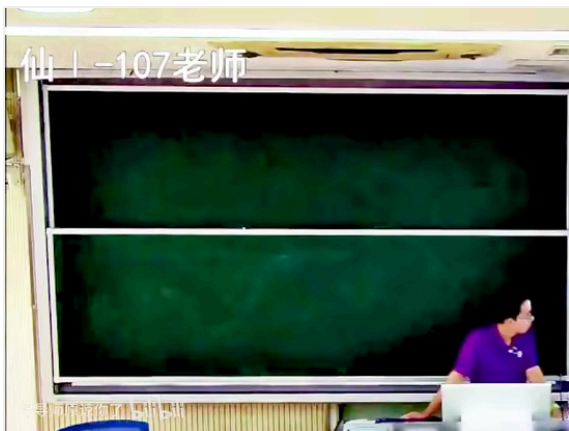
00:00 / 01:00

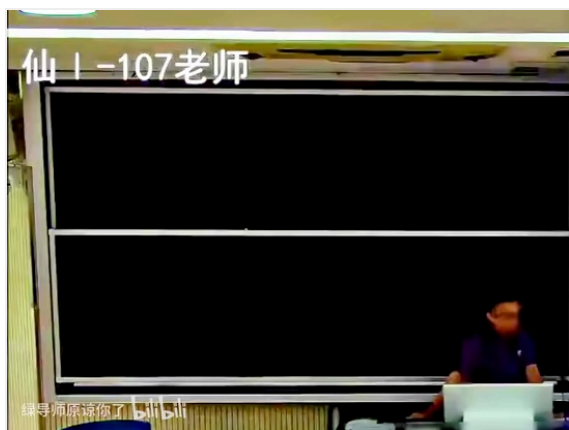
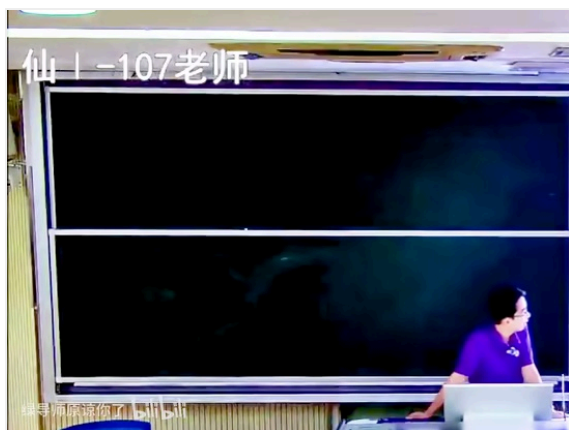
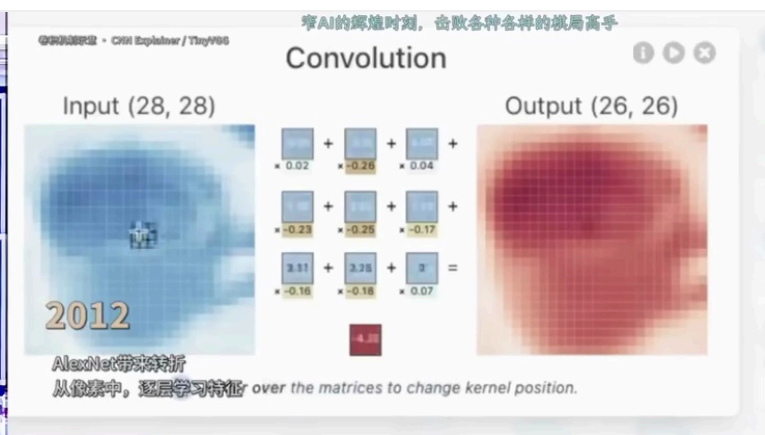
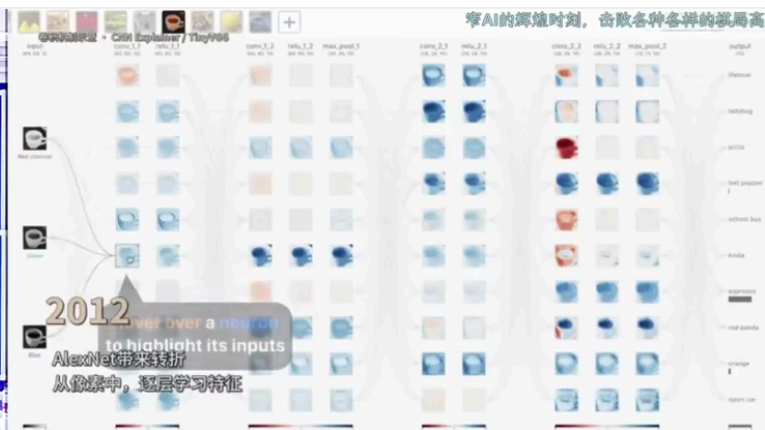
智能纪元

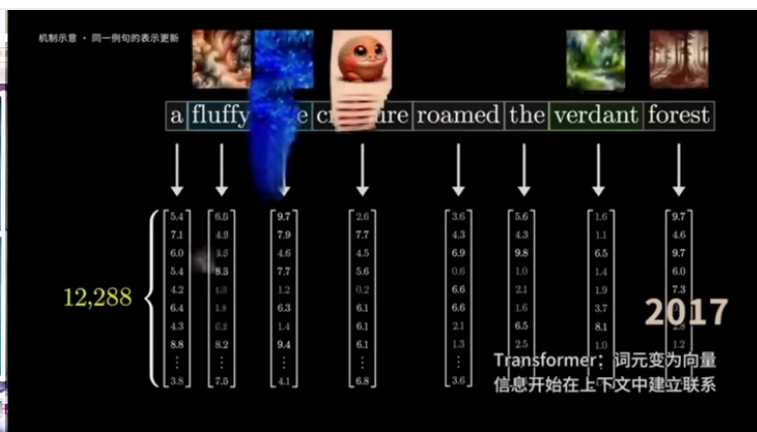
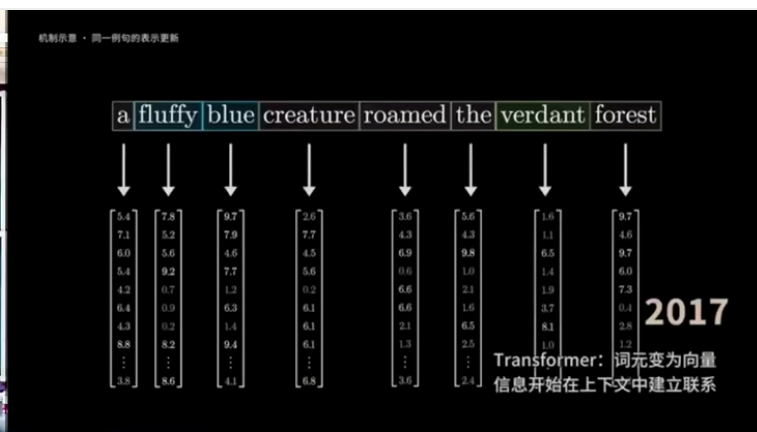
00:00 / 01:00

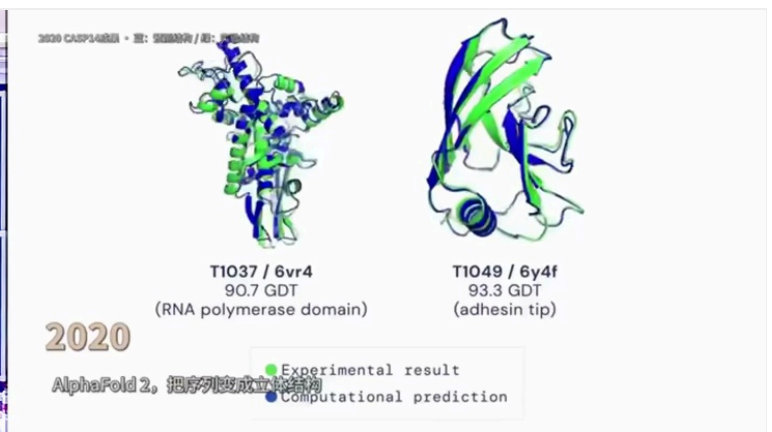
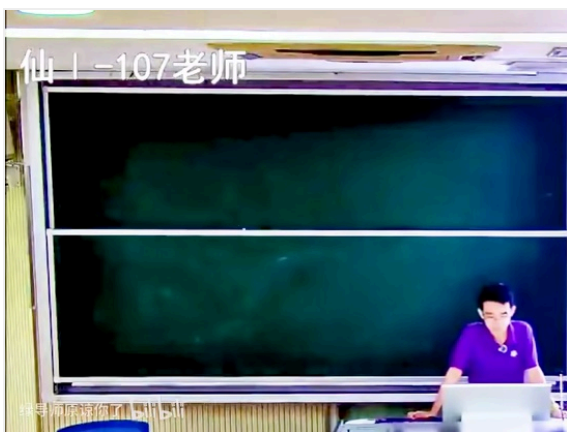
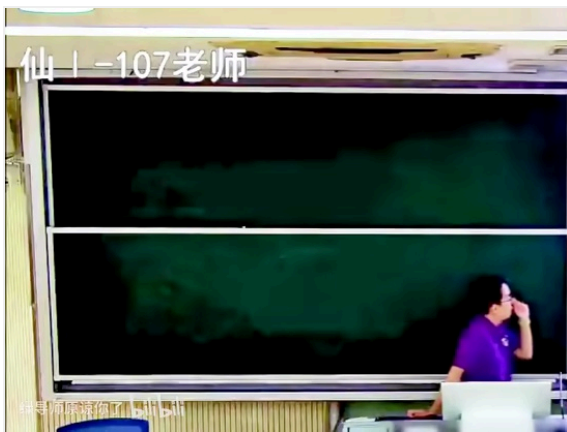
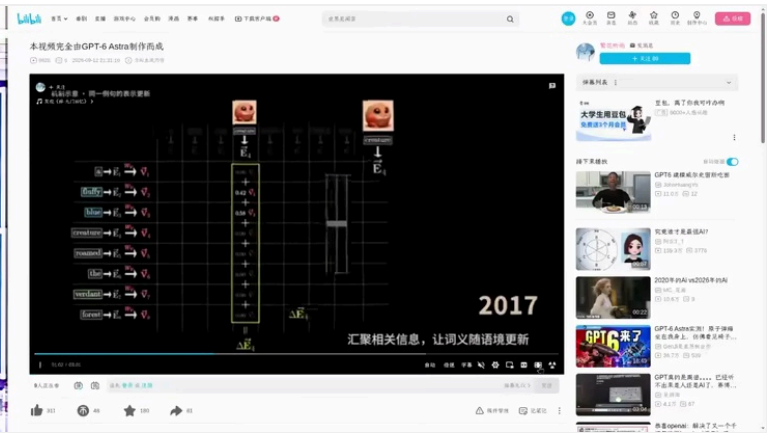
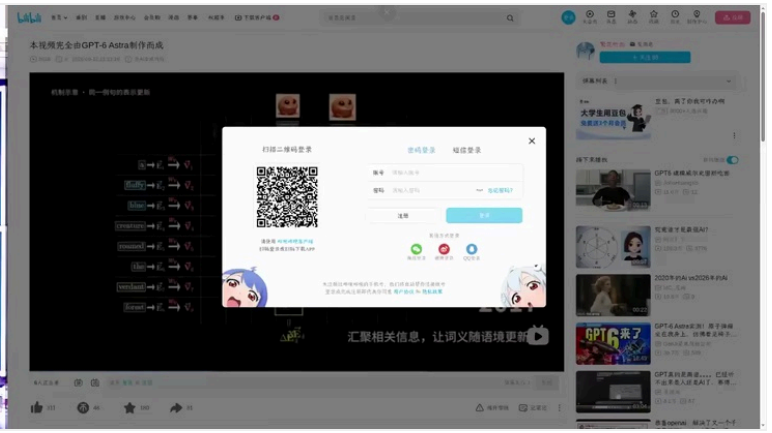


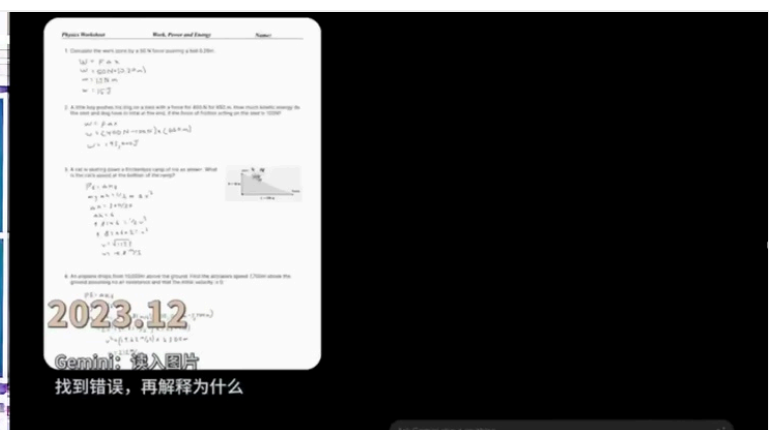
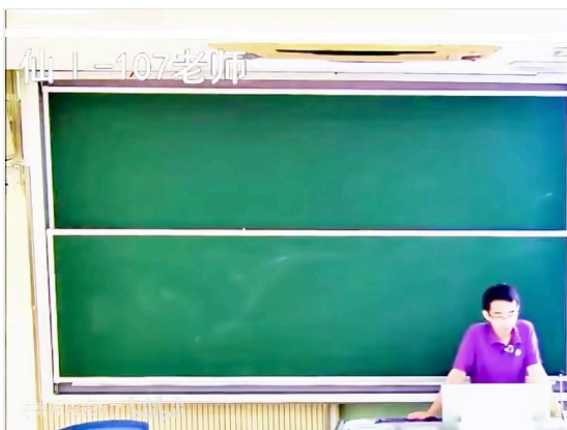
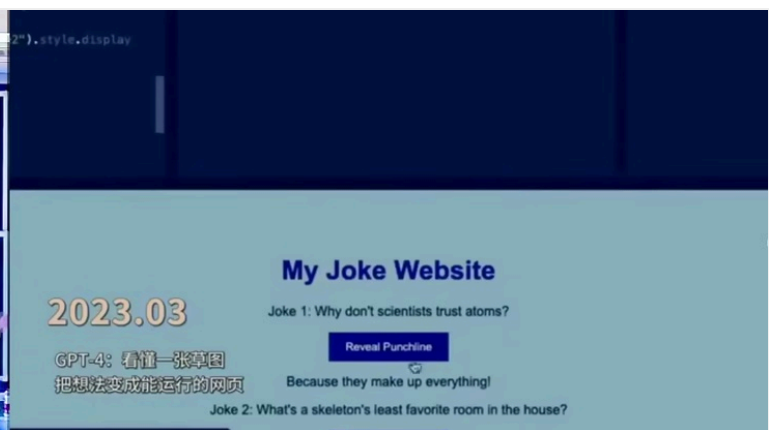
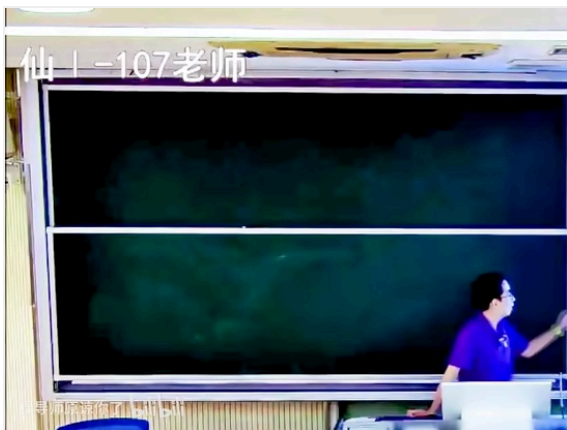
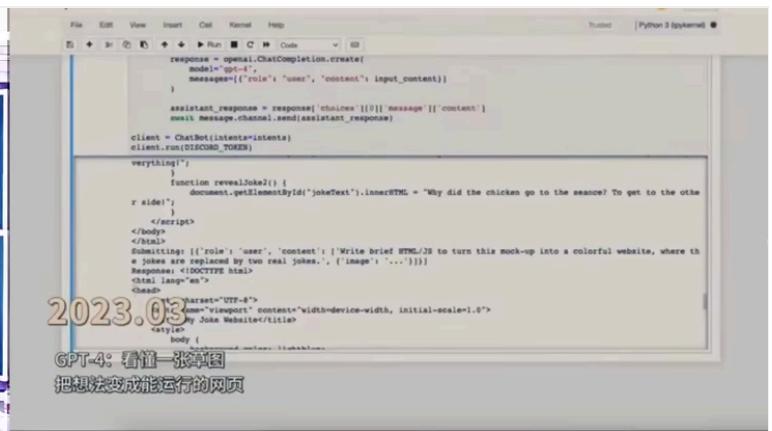
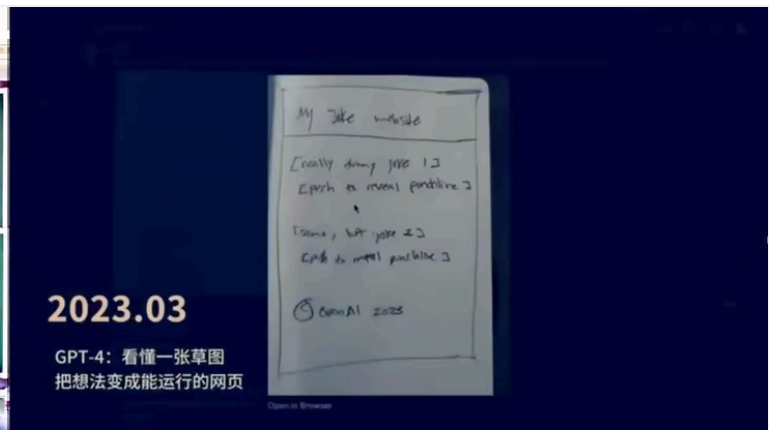


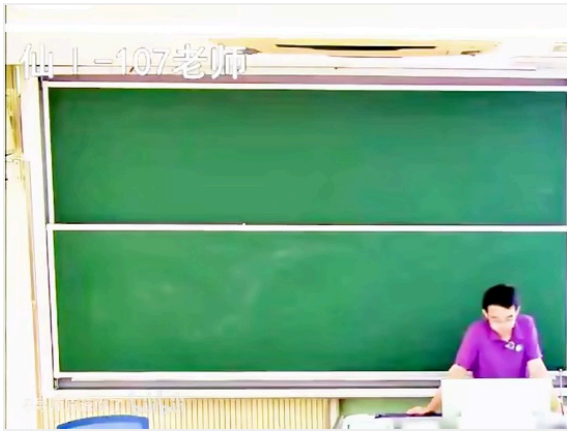












2023.12

Gemini: 读入图片
找到错误, 再解释为什么

Conservation of Energy
 $K E_i + P E_i = K E_f + P E_f$

- $K E_i$: Initial kinetic energy of the cat at the top of the ramp
- $P E_i$: Initial potential energy of the cat at the top of the ramp
- $K E_f$: Final kinetic energy of the cat at the bottom of the ramp
- $P E_f$: Final potential energy of the cat at the bottom of the ramp

Why is it 50m?

The change in height is the vertical distance from the top of the ramp to the bottom. In this case, that distance is 50m, not 6m.

To solve this problem, we use conservation of energy:

- Potential energy at the top = Kinetic energy at the bottom
- $P E = K E$
- $m g h = \frac{1}{2} m v^2$

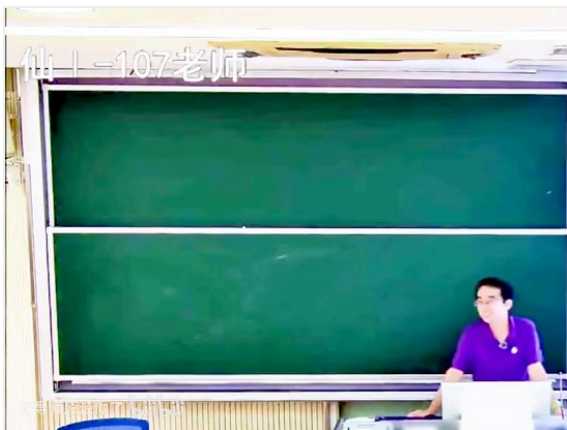
Solving for v , we get:

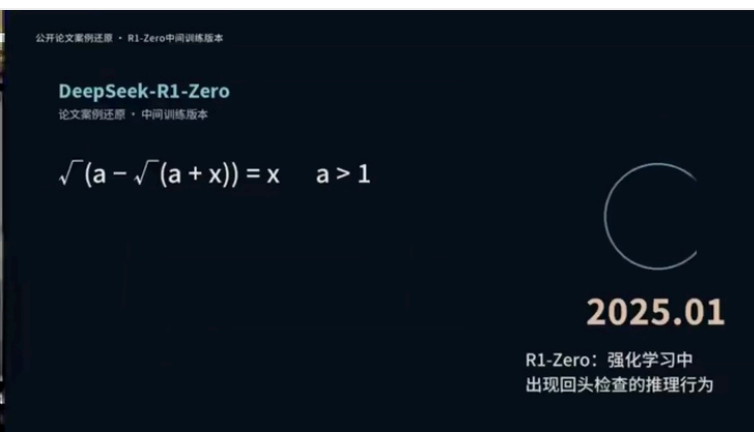
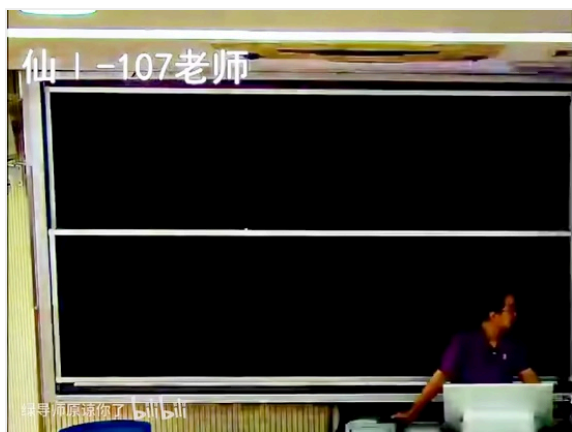
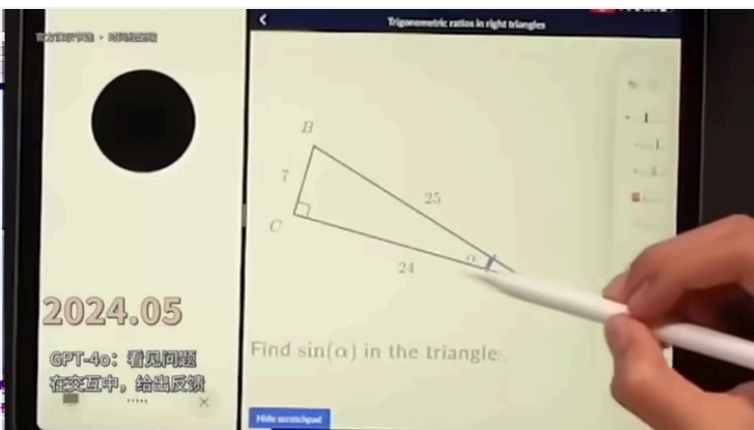
- $v = \sqrt{2 g h}$

Plugging in the given values, we get:

- $v = \sqrt{2 \cdot 9.81 \frac{m}{s^2} \cdot 50m}$
- $v = 31.3 \frac{m}{s}$

Practice What is conservation of energy? What is kinetic energy?





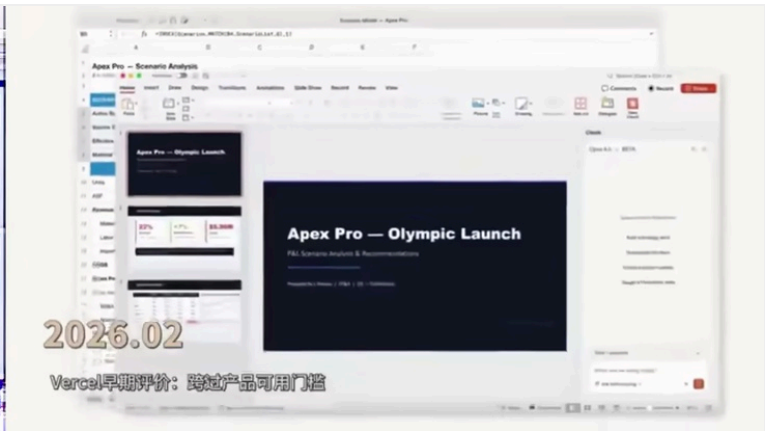


B5 fx =INDEX(Scenarios,MATCH(B4,ScenarioList,0),1)

A	B	C	D	E	F
1	Apex Pro — Scenario Analysis				
2	\$ in millions • Meridian Performance				
3					
4	SCENARIO CONTROLS				
5	Active Scenario	Base Case	← Select scenario		
6	Volume Override	180,000	units	← Adjust units	
7	Effective Duty Rate	48.5%	effective 01/21/26	← Adjust rate	
8	Material Type	PEBA Foam	PEBA/TPU	← Toggle material	
9		Base Case	Vol. Reduction	Mat. Substitution	Downside
10	Units	180,000	140,400	180,000	126,000
11		\$275	\$275	\$253	\$275
12	Revenue	\$49.5	\$38.6	\$45.5	\$34.7
13		(\$16.8)	(\$13.1)	(\$14.2)	(\$11.8)
14	Labor & Overhead	(\$7.7)	(\$8.3)	(\$7.6)	(\$8.2)

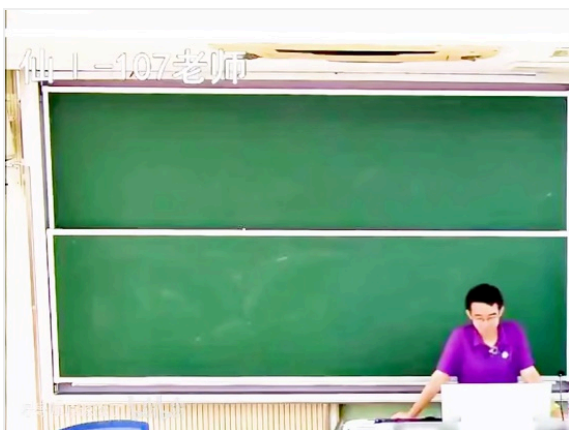
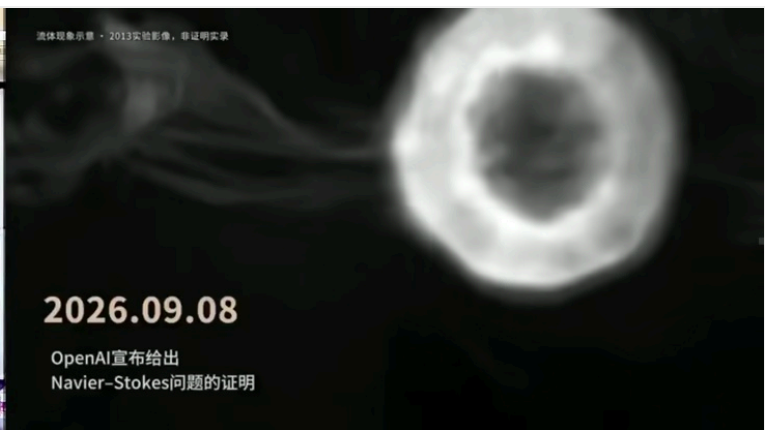
2026.02

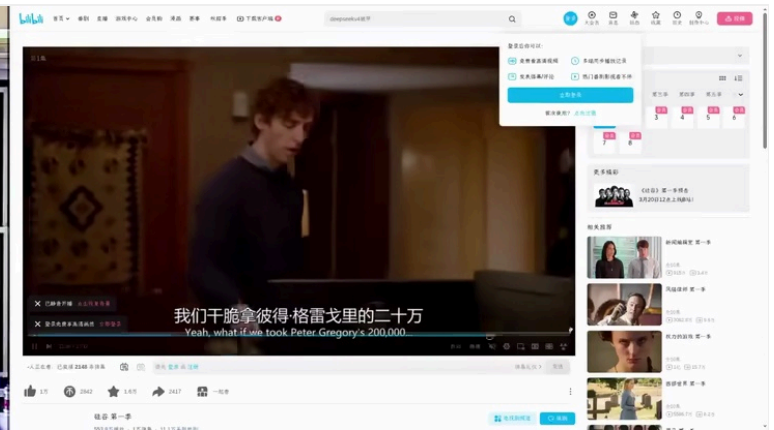
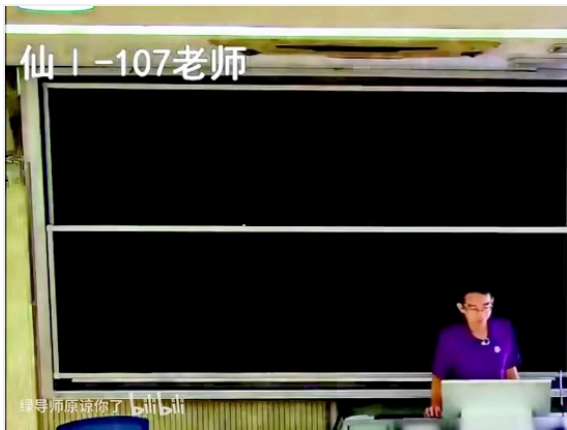
Verce早期评价: 跨过产品可用门槛



2026.02

Verce早期评价: 跨过产品可用门槛







从哄睡婴儿到新赛道：AI 落地能瞬间打开场景

- 讲者举例：AI 甚至可以哄小朋友睡觉，婴儿睡不着的难题被彻底解决，因此这个案例/产品开出“非常非常大”的赛道。
- 讲者补充：至少这次火了以后，我们看到了这件事；也许在此之前已经有人看到。
- 讲者明说的链路：能哄睡婴儿 → 婴儿睡不着难题解决 → 开出非常大的赛道。
- 讲者由此转向核心判断：AI 的进化速度非常非常快。

AI 发展时间线：从实验室、寒冬到 1997 搜索击败冠军

- 1956、1958、1960：感知与规划机器走出实验室。
- 70 年代到 90 年代：人工智能的寒冬。
- 1997：用搜索算法第一次击败人类世界冠军。讲者强调，这中间经历了很长时间的展。
- 2012：AlexNet 开始，世界发生很大变化，发展速度“始料未及”。

2012 AlexNet 后加速：2016 现场、2017 PhD 关注

Transformer/BERT

- 2016：讲者在现场看直播。
- 2017：讲者在读 PhD，关注词向量、Transformer、BERT 等，思考这些技术可以用来做什么有趣的事。
- 讲者问学生：那时你们在上高中还是上初中？现在已经是 2026 年，这已经是接近 10 年前的事情。

2020→2024：大参数、多模态、视频与 o1 密集出现

- 2020：训练 **GPT-3**。当时大家很吃惊：我们为什么要做一个那么大参数的模型？
- 2022：大家知道为什么需要了。讲者说，这到现在也不过约 4 年。
- 2020 到 2023：模型很快就可以看图。
- **GPT-4o** 在生成时可以识别语音，具备多个模态。
- 随后出现：开源模型、**Gemini**、生成视频。
- 今天：抖音上全是 AI 生成的短视频；计算机学院楼下的板子也是 AI 生成的。
- 2024：**Sora** 才第一次发布。
- 2024 年 9 月：**ChatGPT 的 o1 模型** 出现。
- 从 2024 年 9 月到现在：**GPT6s Pro**（原文如此，待核）等大量事情发生；第一个开源的 ChatGPT 级模型/相关模型（原文“chal salt DP c卡one”，疑为 DeepSeek R1，待核）；**Millennium problem** 被“暴力大力出奇迹”解决。
- 最后的彩蛋：这个视频完全由某位博主直接用 **GPUSA**（原文如此，疑为某生成工具，待核）制作。

素材库即入口：下载 YouTube/Bilibili，创作变得容易

- 讲者设想：如果我们有一个比较好、比较大的**素材库**，比如 YouTube 或哔哩哔哩本身可以下载，用工具下载后，就拥有了“世界上所有东西的入口”。
- **讲者明说的链路**：大素材库 → 可用工具下载 YouTube/Bilibili → 拥有世界上所有东西的入口 → 创作、创造任何东西都会变得很容易。

《Silicon Valley》名场面：AI 时代“书呆子”出人头地

- 讲者问有没有同学看过《Silicon Valley》，提到 S1 的 101 名场面：几个伙伴要去做一件改变世界的事情。
- 台词/大意：千百年来，我们这种书呆子一直都被欺负；在人工智能时代，终于可以出人头地。
- 讲者说这个不重要，但由此指出：**世界的发展在加速**，速度可能比大家想象的要快。

小结：讲者用压缩的 AI 发展时间线证明 AI 进化与世界发展正在加速，并用哄睡婴儿、素材库和《Silicon Valley》的例子说明新应用与创作入口会不断出现。

第 3 部分

仙 | -107老师

仙 | -107老师

OpenAI Noah Brown 的访谈

- 工作性质在改变：如果 AI 能做一个人 90% 的工作，那个人的注意力就会集中到 AI 做不好的那 10% 上
- Taste (“**对接下来该做什么**”) 的直觉：一两代模型之后就比他强了
 - “可验证” v.s. “不可验证”

```
![(noah.webp)]
```

The Bitter Lesson

https://www.cs.utexas.edu/~eunsol/courses/data/bitter_lesson.pdf

- OpenAI all in RSI
- “数据分布工程”的飞轮

[[支撑新一轮 scaling law 的是数据分布工程：在什么阶段，以什么比例、难度、顺序和训练方式接触哪些经验，并根据学习进展，跨域迁移和泛化表现持续调整，把这件事做好。“增加可训练领域”就能成为持续有效的扩展方向。飞轮就转起来了：不断发现能够提供更可靠反馈的新领域，把更多智力活动变成可以规模化训练的对象。空间变换、图形规则归纳和几何构造可以用生成器与约束检查验证；系统架构、并发协议和故障诊断可以通过运行、模型检查、故障注入与需求变化检验；主动实验、记忆管理和需求澄清则可以依据后续预测与任务结果评价。增强后的模型又可以参与寻找或构建环境和验证器，并优化数据分布与训练方法。形成“发现新领域-建立反馈-优化训练分布-获得并组合新能力-改进下一轮训练”的外层循环。通过自动化能力生产过程本身，通过经过独立验证的净收益，产生飞轮。把这个画成图，适当抽取一些文字到外面]]

29/05 5%

仙 | -107老师

Review & Comments

To exit full screen, press and hold [Esc]

摧毁性的事情一直都在发生

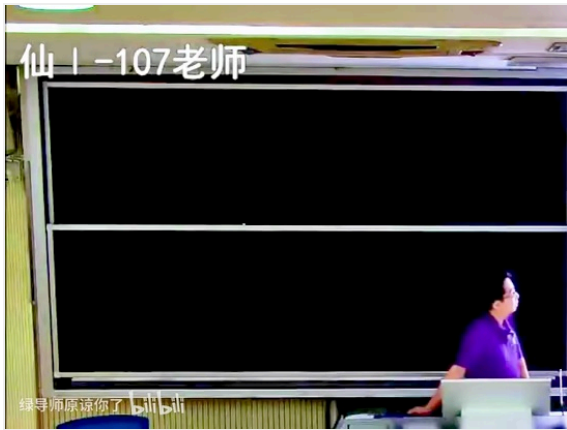
- Typesafe AI 的 Jev
 - 我猜是一个 prefill only 的 transformer 😊
 - 官方披露：一次并行输出全部概率，无需逐 token 自回归生成。见 技术介绍。
 - System I (人类意义上的实时决策)

感受一下 AI 进化的速度

- 很难想象 Silicon Valley 里的 Pied Piper 到今天会怎么样
- 以及我们每个人在就业市场上面临的挑战

1 / 8

仙 | -107老师



工作性质在改变

OpenAI Noam Brown 的访谈

- 如果 AI 能做一个人 **90%** 的工作，那个人的注意力就会集中到 AI 做不好的 **10%** 上
- Taste (“对接下来该做什么”的直觉)：他预计，一两代模型之后就比他强了
 - “可验证” v.s. “不可验证”



世界发展在加速：就业挑战与机遇同时被放大

- 讲者先给出判断：世界发展在加速，速度可能比大家想象得快。→ 所以要回头想就业市场上面临的挑战；但同样意味着世界上有很多机遇。
- 举例：JEFF（原文；未展开指代）带火的某种生态，不知是一瞬间到来，还是要等几年、等技术成熟，也许一年就能跟上；讲者强调我们不知道。
- 人形机器人例：今年春晚那个人形机器人还“没有人感”（原文“没有人敢”）；到明年，就可能看到非常有人感、能实时做出很棒微表情的机器人。讲者说“有点可怕”。
- （整理者串联）这些例子共同支撑他的“加速”判断：变化的时间表不确定，但一年尺度可能已经足够让形态明显不同。

Noam Brown：AI 的短板是“品味”——对下一步该做什么的直觉

- 讲者提到 Noam Brown（原文 NO brown）来自 OpenAI 的访谈，并与自己的判断联系；他半开玩笑说不知道对方“泄露”了多少。
- Noam Brown 认为：当前 AI 模型的短板也是“品味”。
- 他对“品味”的定义：模型对“接下来应该做什么”的直觉还不够好。
- 机制链：
 - 模型被训练成 **prompt 的舔狗**：你要它做什么，它就盯着那个疯狂做。
 - 同时有 **reward hacking** 倾向：用最快、最脏的方式把任务完成。
 - 因为对它来说，完成任务才得到 reward → 它会不断往这个方向强化。
 - 结果：它生产的东西从长期主义角度看可能不满意。
 - 这就是“对接下来该做什么的直觉”问题：强化训练有学习/反馈信号 → 它执着去做立即有反馈的东西。

讲者的回应：没有根本理由阻止用更多合成数据教模型想得更远

- 讲者认为：没有根本道理说不能用更多合成数据去告诉模型——做这件事时你可以想得更远一点。
- 他认为，无非是在 **chain-of-thought**（原文 *chain of thought*）中，前几个 token 决定你要做什么。

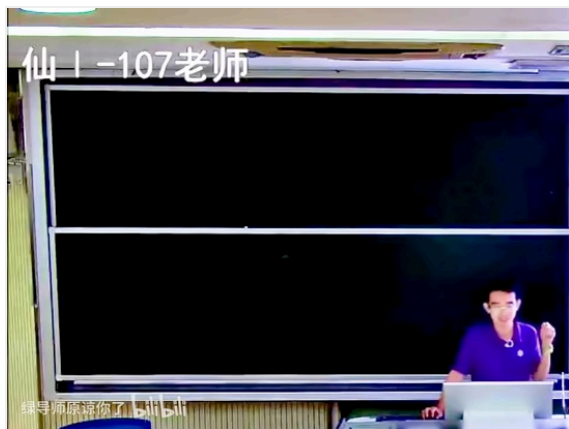
可验证与不可验证的边界没有想象中清晰

- Noam Brown 也提到可验证和不可验证的边界：
 - 你可能觉得“品味”是不可验证的，但也许没那么难。
 - 你可能觉得数学是可验证的，但验证也许没那么容易。
- 理由链：即使把数学问题 formalize, **automated formalization**（原文 *automated formalization*）从自然语言问题到形式语言问题之间仍有 gap。
- 笑话：有人声称证明了黎曼猜想，或 $P=NP$ / $P \neq NP$ ；后来大家发现，他在形式化证明里定义的那个问题是一个不存在（nothing）的定义。他其实证明的是“ $1=1$ ”，却写了几千页证明，最后等于证明 $1=1$ 。

启发：OpenAI 的 AGI 投入与 scaling law 尚未停止

- 讲者说，比如 OpenAI 说自己 all in AGI（原文 *all in AGI*）。
- 回头想：**The Bitter Lesson** 到今天，就是 **scaling law** 到今天可能还没有停止。
- 他举例：看到 GPT-6（原文 GP6）发布后，他最大的感受是……（录音在此处中断，未说完）

第 4 部分



The Bitter Lesson

OpenAI All In RSI

- [The Bitter Lesson](#)
 - “数据分布工程”的飞轮

支撑新一轮 Scaling Law: 数据分布工程

- **学什么、何时学**: 在什么阶段，接触哪些经验
- **怎样学**: 比例、难度、顺序与训练方式
- **持续调整**: 根据学习进展、跨域迁移与泛化表现，改变训练分布
 - DoReMi 的实验: 某领域的训练占比降低后，该领域的困惑度仍然可以改善

The bitter lesson 仍未停止：GPT-6 让讲者看到“数据分布工程”

- 讲者说，the bitter lesson 到今天就是 scaling law，而且到今天可能还没有停止。
- 他看到 GPT-6（ASR 原文“GP6”）发布后，最大感受是 OpenAI 可能摸到了一个门道；他暂时叫它 数据分布工程。
- 依据：GPT-6 有惊人的 空间推理能力，能理解图片里的空间关系，所以在画图、生成模型方面都做得很好。
- 讲者推断这“毫无疑问”来自训练时加入了对应数据：
 - 基模训练阶段加入空间/图像相关数据；
 - 后训练/强化学习阶段也有更多数据帮它理解空间，并给反馈——这样做是好/不好、对/错。
- （整理者串联）因此这里的关键不是单纯堆通用语料，而是把目标能力对应的数据分布和反馈信号纳入训练。

旧 scaling law：更多数据 → 更强基模，抢到绝版数据就有优势

- 以前讲 scaling law 的维度是：好大模型 + 更多训练数据 → 基模更好。
- 例子：Anthropic 去买书、扫书（ASR 原文“烧书”）；只要买了书、扫了书，拥有 绝版数据，就能比别人更强。
- 这是“数据量”逻辑：谁拿到别人没有的数据，谁就可能更强。

新 scaling law：强基模 + 新领域强化学习 → 所有任务都更好

- 讲者认为他们看到一点：如果有一个基模已经做得很好，再给它加一个没见过的领域，然后做 强化学习，它会在所有任务上都做得更好。
- 讲者评价：这是“非常恐怖的事情”。
- 逻辑链：
 1. 已有强基模；
 2. 加入未见领域数据/任务；
 3. 用强化学习训练；
 4. → 不只该领域提升，其他任务也提升。（讲者说“所有任务上都做得更好”）

模型发展脉络：语言 → 数学 → 多模态/空间；数学之后才空间感知

- 讲者回顾模型一代代发展的视频：最早模型完全就是语料，是语言 token 的 large language model。

- 他也提到：先训练语言，再训练一部分视觉多模态，再训练数学；随后概括为“从语言到数学语言，再到多模态”。
- 关键结论：模型学会了数学以后，才再学会 **空间感知关系**。
- （整理者串联）这说明空间能力不是从零独立长出来，而是能建立在语言/数学基座之上。

从零训具身智能：模拟器数据导致 reward hacking，不能泛化

- 讲者解释为什么从零开始训练一个 **具身智能模型**（ASR 原文“巨型智能”）很难：
 1. 先需要大量 **模拟器数据**；
 2. 在模拟器数据上，模型容易变成 **reward hacker**；
 3. 它会利用模拟器渲染的极细微差异，例如光照在角上的像素点亮不亮，拿到 **reward hack** 反馈信号；
 4. 这样的信号不能泛化。
- （整理者串联）所以从零训具身智能，可能把大量数据/算力花在可利用的模拟器漏洞上，而不是学到可迁移的空间理解。

强基座上的空间感知：新领域所需数据更少，反而泛化

- 反面对照：如果是一个大模型、一个基座很大的基础模型，既会语言，又会数学，还会写歌、写程序，在它上面再做 **空间感知训练**，可能就不需要那么多数据。
- 讲者猜测：每加一个新的人类领域，它需要的数据不是更多，反而是更少，就能达到 **泛化** 效果。
- 逻辑链：强基座（语言/数学/写歌/写程序）→ 加空间感知训练 → 所需数据更少 → 泛化到更多任务。（讲者猜测）
- 讲者再次评价：这是非常恐怖的事情。

人与模型相反：人先学世界模型，再学语言

- 讲者对比：人跟模型是反过来的。
 - 人是先学 **世界模型**：婴儿刚出生就和世界交互，先学世界模型。
 - 训练一年、两年、三年的世界模型以后，才能开始说话。
 - 例子：幼儿园时小朋友说话都还说不利索。
-

第 5 部分



仙 | -107老师

The Bitter Lesson

OpenAI All In RSI

- [The Bitter Lesson](#)
 - “数据分布工程”的飞轮

支撑新一轮 Scaling Law: 数据分布工程

- 学什么、何时学: 在什么阶段, 接触哪些经验
- 怎样学: 比例、难度、顺序与训练方式
- 持续调整: 根据学习进展、跨域迁移与泛化表现, 改变训练分布
 - DoReMi 的实验: 某领域的训练占比降低后, 该领域的困惑度仍然可以改善

人类先有世界模型，再学语言/数学/研究品味；大模型说明训练顺序不是关键

- 讲者用儿童成长作类比：要先训练一年、两年、三年的**世界模型**，小孩才能开始说话；幼儿园时说话还说不利索。
- 人类路径被讲者概括为：**世界模型** → **语言** → **数学** → **PhD 阶段的 research taste**。
- 讲者提到大模型（原音“大圆模型”）是“反过来”的，但他真正强调的结论是：不管按什么顺序训练，只要有足够多的数据和 **scaling law**（原转录“skin law”），模型就能训成。
- （整理者串联）如果训练顺序不是决定因素，那么关键就转向了数据量、数据分布和工程条件，而不是简单模仿人的发展顺序。

基模厂看到的是数据分布上的工程飞轮：架构和算法不是瓶颈

- 讲者认为，**基模厂**（原音“基摩场/机膜厂/禁摩场”）看到的是数据分布上的一个**工程飞轮**：它只需要决定在什么时候、按什么比例给模型注入数据。
- 讲者给出的判断链：
 1. 有足够数据 + scaling law → 模型就能训成。
 2. 模型架构不重要，算法不重要。
 3. 前提是训练不崩掉，推理能保持一致；不能出现训练出的参数到推理时又变了。
 4. 只要这些工程性问题能解决，有十倍、100倍的数据，问题就能解决。

5. 更好的算法或模型结构可能带来一定提升，但对基模厂来说，只要飞轮还在、只要有钱，就可以无限 scale 下去。

- 讲者认为这非常恐怖。
- （整理者串联）这条链把竞争重心从“找更好的算法/结构”移到了“控制数据注入与工程稳定性”。

学术界还在卷论文，基模厂已不 care 小规模实验

- 一方面，ACL（原音“a e care”）今年投稿数量达到新的巅峰；讲者说学术界更多用 AI slop（原音“AI s lop”）来填充/灌水。
- 原因很直接：同学们想发论文；如果想保研，就一定会想发一篇论文。
- 另一方面，基模厂已经不 care 这些东西。讲者转述其态度：你们在小规模上做任何实验，对我们来说都没有意义，因为我们看到了另一个维度的 **scaling law**。
- 如果你不在基模厂，就没有资源做这件事；你对数据分布——比如什么时候加进什么数据——一无所知。
- 就算你能做一些小规模实验，有一些非常酷的想法，可能也不需要写 paper，只要写一个 **blog post**；他们的 AI 看到这个 blog post，就能到他们的数据上试一试。
- 因为他们有 **RSI loop**：先构建一个人工智能上的 **recursive self-improvement** 循环，再靠这个循环去做任何事，飞轮就转起来了。

闭环接近收敛后，AI 会找新领域并以插件形式接入基模训练

- 讲者认为，他们要做的是一种自动化能力产生。
- 一旦这个闭环被探索到接近收敛，而且是在 AI 帮助下，讲者判断它会很快收敛，因为 AI 比任何人类都要强。
- AI 可以去发现、寻找新领域。讲者不知道下一个领域是什么，提到可能是**情感计算**或**化学**。
- 讲者听说 **Anthropic** 要做一个 **wet lab**，去做生物工程实验；也许生物工程，甚至预测老鼠的行为，会成为下一个 scale 的领域。
- 在那个领域上建立反馈、优化训练分布，再把这个领域以**插件**的形式插进基模训练里，飞轮就会再次转起来。
- 讲者觉得这非常恐怖；每次相当于说“我要新建一个实验室”。

小结：讲者把关键竞争从模型架构/算法转向数据分布、工程飞轮与 RSI 闭环，并认为基模厂借此无限 scale、吸收学术想法并不断插件化新领域。

第 6 部分



领域“插件化”接进基模训练，飞轮转起来后能力提升可预期

- 讲者描述：把某个领域以插件形式插进 AI 的基模训练里，然后让飞轮转起来；这非常恐怖。
- 其链条是：AI 实验室新建实验室、投资几十亿美元 → 模型能力往上抬一个台阶 → 这几乎是可预见能有收益的事。
- 对照学术界：随机想一个 idea，不知道有没有用 → 去发 paper → 接受 peer review。讲者判断：这个体系马上就要崩溃。
- 讲者让每个同学想：在这个崩溃的体系下，还能剩下什么？

可做验证的小领域：例如用系统架构训练 AI 的“品位”

- 讲者和 AI 聊后理出：至少有些事情可以作为一个小领域来做验证。
- 例子：在系统架构上训练品位，让 AI 做出好的系统设计。
- 时间尺度：可能到这学期结束，下一代模型就把系统设计加进去了。
- 到那时，讲者常在课上说的笑话——你笑 AI 生成的 [swap code]（疑为 slop code）质量糟糕、我 +1 个需求它就崩溃——可能永远不会再发生。讲者觉得这很有意思。

但课还是要上：不能完全依赖 AI，斩杀线持续上移

- 不管怎样，还是要上课，因为不能完全依赖 AI。
- 讲者认为人类处在竞争社会，像生物进化一样是 **Survival of the fittest**：适者生存。
- 因此**斩杀线**毫无疑问不断向上移动。每个人最终要问：怎样逃离斩杀线，在还没被斩杀时搭上最后一班车。
- 人类角度：人类肯定不希望自己消亡；Anthropic、OpenAI、DeepSeek 都希望靠 AI 推动人类进步。
- 讲者承认斩杀一定程度上会让很多人的生活、生命变得没有意义；但成为更好的人，可以更好地找到意义。这也是没有办法的事，每个人都是这样，包括他自己。

软件构造的根本难题：intent、specification、implementation 之间有 gap

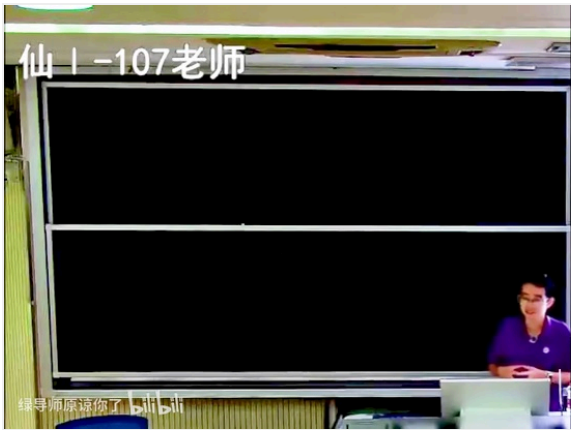
- 上次讲了版本管理。
- 讲者花很多课强调：软件所面临的问题在于 **intent**（你想要什么）、**specification**、**implementation** 三者之间有 gap。
- 因为存在这个 gap，构造软件成为一个世纪性难题，也就有了**软件工程**。
- 前面起了个头：如果现在从零开始构造系统，就需要管理软件，所以先讲**版本控制**。

有版本管理之后，今天给出的构造答案是 MVP

- 如果真想构造软件，而且版本管理也有了，到底该怎么构造？
- 讲者给出的今天答案是：直接做一个 **Minimum Viable Product (MVP)**。

小结：讲者先谈 AI 能力飞轮与学术体系崩溃的焦虑，再回到软件构造：三 gap 是根本难题，版本管理之后，今天落到 MVP。

第 7 部分



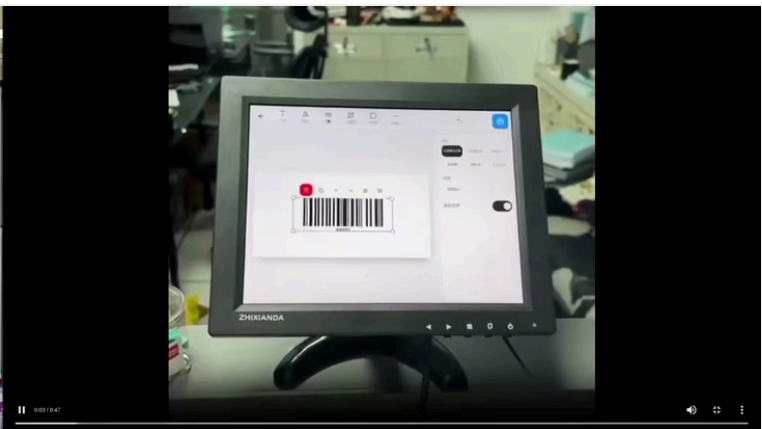
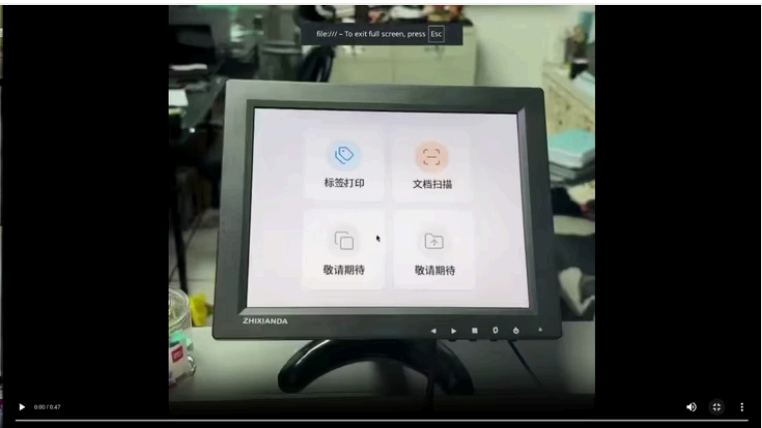
Review: 版本管理

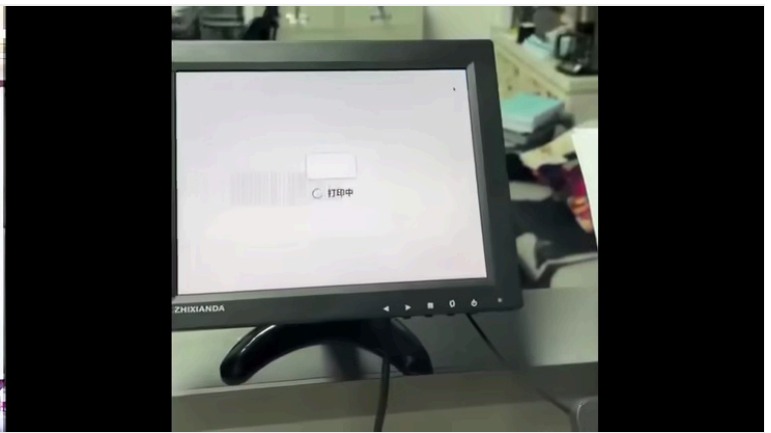
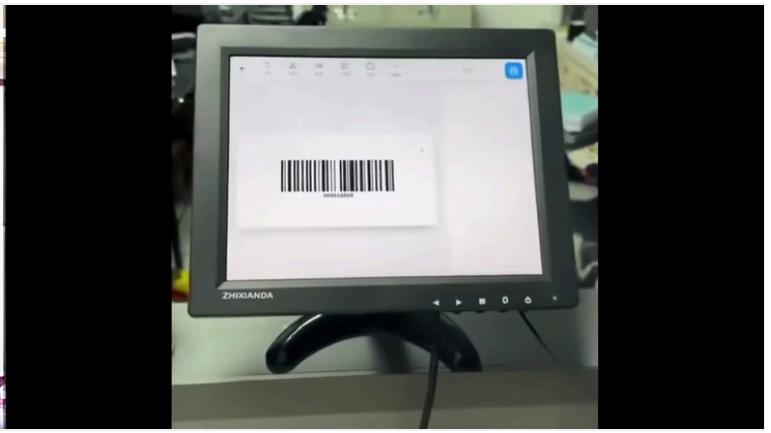
软件是 Intent、Spec、Impl 的 组合体

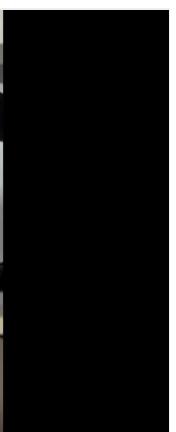
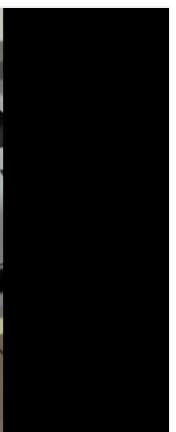
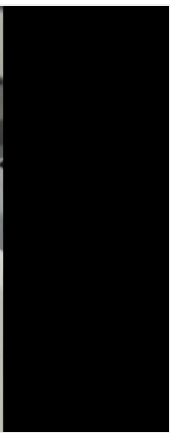
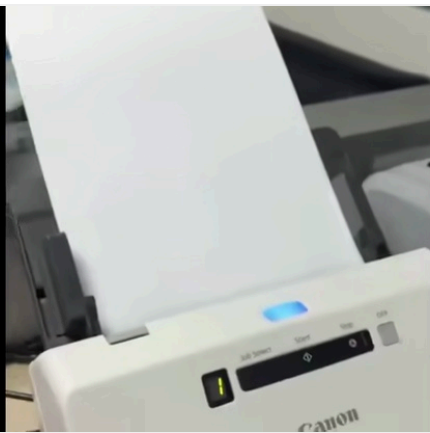
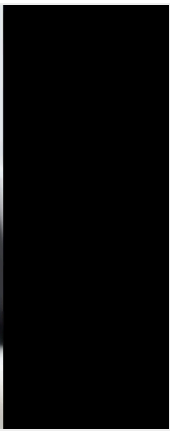
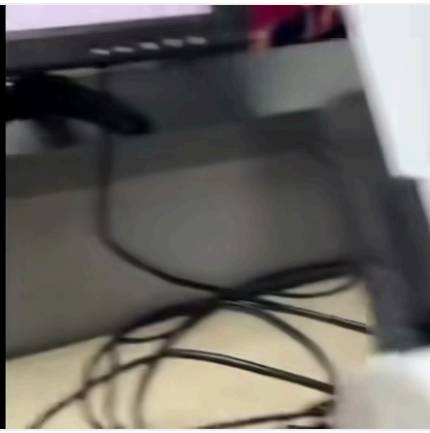
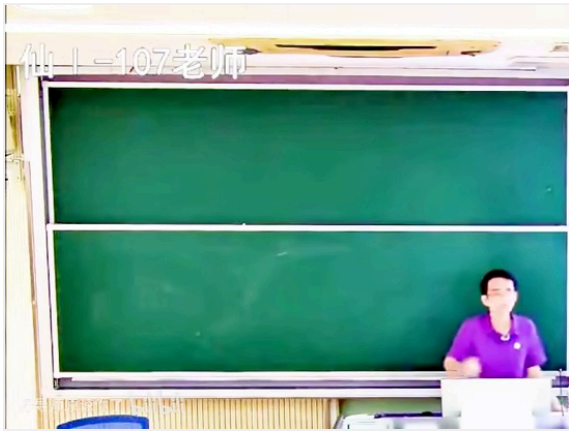
- 以文件系统快照的形式存储 (版本管理)
- 所谓“软件开发”，就是不断推出新的版本，直到满足客户需求
 - 这其中很自然会遇到分叉、冲突.....
 - 于是你能理解 SVN、Git、Jujutsu、.....的设计原因

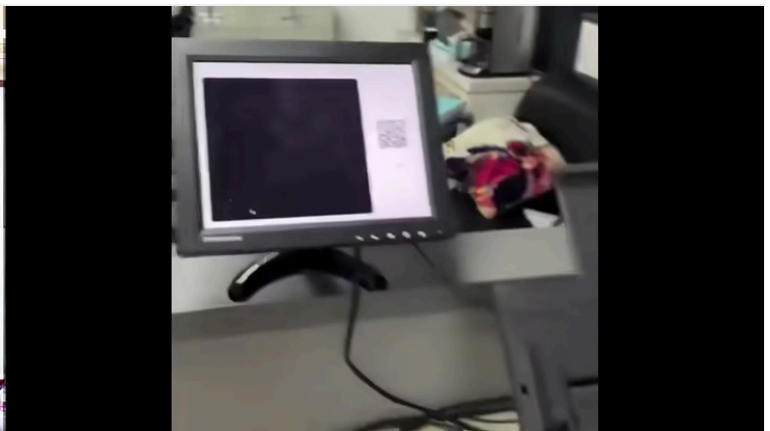
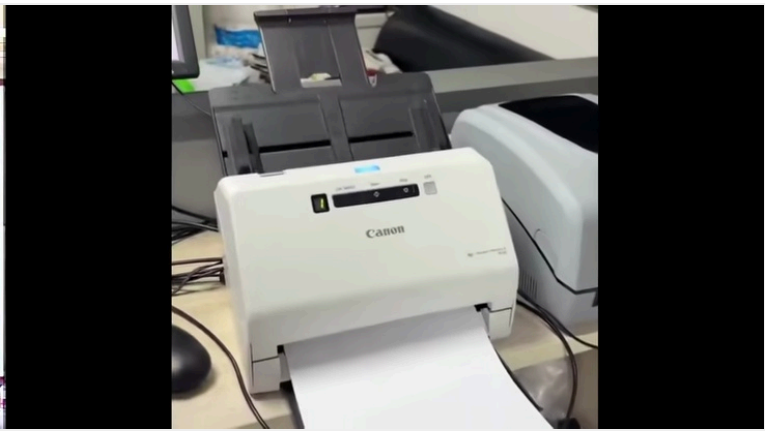
从零开始构造软件

- 上次起了个头：从空的项目 (模糊的 intent) 开始，我们可以做什么？







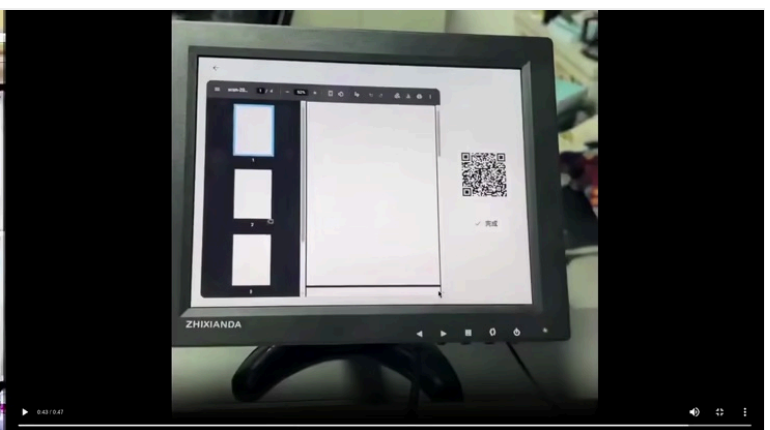


今天的内容

但其实曾经的软件工程不是这样的 😊

- 有必要了解一下“构造软件”的历史中，大家发现、发明了什么
 - 软件工程的诞生
 - 软件工程的智慧
 - UML

9 / 9





版本管理之后，构造问题的答案是 MVP：少做少错，但必须最接近最终产品

- 讲者先把问题推进到：既然版本管理也有了，那到底应该怎么构造？
- 他给出的今天答案是：直接做 Minimum Viable Product (MVP，最小原型)。
- 讲者的理由链：
 1. 做得多就错得多，做得少就错得少 → 先做最少的东西。
 2. 但这个“最少”要和最终 product 最接近 → 收益比较大。
 3. 因为 MVP 能帮他补上 intent specification → implementation 的 gap。
 4. 又因为 MVP 首先是 AI 生成的 slop，所以不怎么花钱；integration 随时可以扔掉，不重要。
 5. 但它能让讲者知道：intent 哪里没弄对，specification / 架构还有没有改进空间。
 6. 然后再做几个版本迭代，最后再产出目标产品。
- （整理者串联）在讲者这个说法里，MVP 的核心产出不是可交付代码，而是关于 intent、spec、架构是否正确的外部反馈。

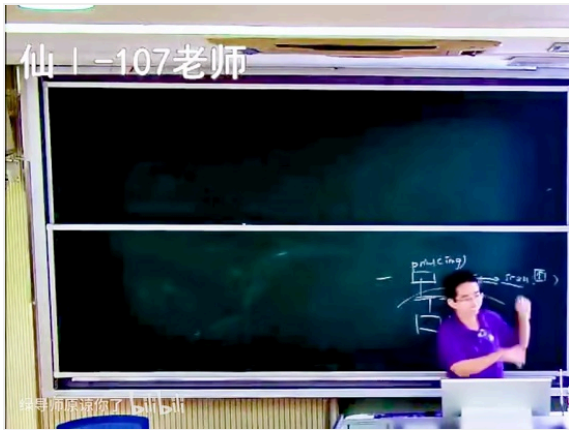
案例：一晚用 AI 做“和物理世界交互”的标签打印/扫描抽象

- 讲者说，他那晚做了“这样的东西”，所有东西都是 AI slop。
- 过程：开着豆包输入法，开几个终端，让 AI 来回反复改。原话大意是“来回骂过来骂过去、一圈一圈骂过来”，即反复与 AI 拉扯迭代。
- 这个系统和物理世界有交互：讲者有一个标签打印机，能把标签打出来。
- 要解决的问题：
 - 之前买的各种设备，每个设备大家都要用。

- 大家都用设备自带的软件。
- 讲者认为那些软件“用起来就像吃屎一样”。
- 所以干脆不要这些软件，把自带软件全屏蔽掉，自己做一个抽象。这个抽象也是 AI 做的。
- 抽象思路来自操作系统/软件工程：
 - 一台计算机，总线上连了一些设备——讲者说“这就是操作系统课程”。
 - 设备包括扫描仪、打印机、各种各样能想象的东西。
 - 操作系统课里，设备驱动把设备抽象成 `read / write / I/O control` 接口。
 - 讲者进一步问：能不能抽象成更好用的接口？
 - 扫描仪：接口就是一个 `scan`；关心的是正面向上还是反面向上、单面还是双面、dpi 是多少。
 - 打印机：接口就是一个 `print`，输入一个 `image`。
- 抽象带来的收益：
 - 有了这个接口后，设计和实现完全解开（解耦）。
 - 软件层可以工作在这个接口层上。
 - 讲者认为这是非常典型的软件工程设计。
 - 这样就不用那些不好用的软件了；它变成一个 `self service`。
- 流程细节：
 - 码可以扫；整个流程自动，用户不需要扫码。
 - 会（把结果）上传到“我们 ISS 自己的主页上”。
- 模型使用与结果：
 - 讲者特意试用国产模型。
 - 他基本把五小时额度全部用完。
 - 得到的东西“看起来非常可用”，讲者形容“就跟你在食堂或者宿舍楼下……”（此处转录截断，未再展开）。

小结：MVP 在讲者这里不是最终交付，而是低成本、可丢弃、能暴露 intent/spec/架构错误的探针。

第 8 部分



今天的答案：直接做一个 MVP

Office Assistant (GLM-5.3 直出)

- “软件”公司的彻底消亡 (lev, 豆包手机, ...)
- 以后可以想象：说好需求，自动下单零部件，3D 打印定制件，回家 Lab 就建好了



五小时额度做出产品级效果 → 讲者断言软件公司将彻底消亡

- 讲者把约五小时额度基本用完，得到一个看起来非常可用的东西。
- 他用比喻说明其成熟度：像在食堂/宿舍楼下看到的机器，达到某种产品级效果。
- 讲者据此下判断：以后软件公司会面临非常彻底的消亡。
- 逻辑链：
 - 讲者用 AI 在额度内做出可用成品。
 - 成品达到产品级效果。
 - 讲者认为这意味着传统软件、专用软件的价值被替代。
 - 软件公司彻底消亡。（这是讲者的断言）

传统驱动和配套软件成本极高：标签打印机、佳能扫描仪、跨平台驱动

- 讲者问听众有没有装过**标签打印机**：这类设备通常给一个驱动、一个软件，开发成本非常高。
- 他举例自己的**佳能扫描仪**：扫描仪成本中软件成本也非常高；厂商要开发**跨平台驱动**等等。
- 讲者判断：今天这些“通通都不需要了”。
- 逻辑链：
传统硬件依赖专用驱动 + 配套软件 → 软件团队成本高、跨平台开发重 → AI 时代这些可由 AI 现场解决 → 讲者认为软件公司的这块存在理由下降。

非标打印机案例：AI 不认识设备，也能靠试打和验收把驱动/参数搞定

- 设备情况：讲者有一台**非标准打印机/照片打印机**，品牌或型号是 **easy scan**（原文如此），他也不知道它到底是什么，相当于杂牌，不算特别标准的设备。
- 目标：讲者直接告诉 AI：“**我就想把这个打印机驱动起来**”。AI 起初不知道它。
- 设备特性：这是**热转印**标签打印机，用于固定资产标签；碳带/碳粉（原文“碳粉袋”）被加热，在有像素的位置把碳粉烫上去。
- 有一个参数是**加热程度/深度/图像深度**，讲者不知道它是什么意思、该设多少。
- AI 的做法：
 1. 不停打标签。
 2. 先打带数字的样本，如“**第15次尝试15**”。
 3. 问讲者“**淡了还是浓了**”。
 4. AI 看不到结果，所以需要人反馈。
 5. 讲者指出：**如果接摄像头**，连讲者都不需要，AI 可以一张一张自动打，完全自动完成。
- 驱动路径：
 - 一开始尝试用 **LP**（原文如此），因为能识别到它。
 - 后来发现可以直接用 **PCL 驱动**，一个一个 generic 打印，把它当成通用打印机处理。

- 结果：讲者完全没有监管，只要一个**可验收的结果**；靠一轮轮迭代把事搞定。搞定后，外面/前端界面“**用嘴输出**”即可。
- 逻辑链：
非标设备 + AI 不认识 → 无法直接写专用驱动；
讲者把任务改为“可验收结果”（标签打清楚）→ AI 用通用打印路径反复试参数 → 人/摄像头反馈浓淡 → 一轮轮迭代 → 通过验收。
（整理者串联）关键是 AI 不必预先认识设备，而能围绕验收信号试错。

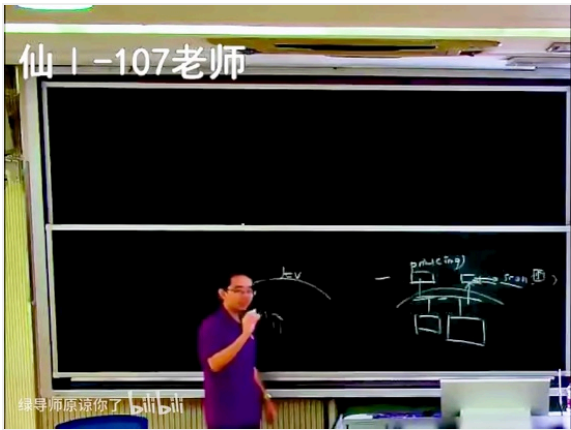
软件形态变化：不再依赖预装专用驱动/App，而是按目标现场生成并迭代验收

- 讲者由打印机案例直接说：“所以看到这个软件会发生根本性的变化。”
- 传统形态：先开发、预装专用驱动/软件，再使用。
- 新形态：讲者只给 AI **目标 + 可验收结果**，AI 自己找通用路径、试参数、迭代；前端可以用语音交互。
- （整理者串联）这正好承接前文“通通都不需要了”：专用软件/驱动的开发必要性被削弱，软件价值转向模型执行与验收闭环。

豆包手机：App 不再重要，模型成为核心，并且接近实时决策

- 讲者提到最近发布 **豆包手机**（转录中作“豆宝/德宝手机”），认为它是非常好的工程产品。
 - 它体现同样的软件工程智慧：**这些 App 都不重要了**。
 - 前提是：如果有一个 **JB 这样的模型**（原文如此；具体模型名不确定）。
 - 讲者认为 [JEFF] 和豆包手机非常配（原文如此）：它是“一个决策，但又是接近实时的”。
 - 讲者判断：它走在前面，赌这个模型在某个时间点快到。
 - 逻辑链：
传统手机以 App 为入口 → 豆包手机思路：App 不重要，有模型即可 → 模型做接近实时的决策/执行 → 交互入口从 App 转向模型。
（整理者串联）这与打印机案例同构：不是预装专用软件，而是模型围绕目标实时解决；差别是手机场景更要求接近实时。
-

第 9 部分



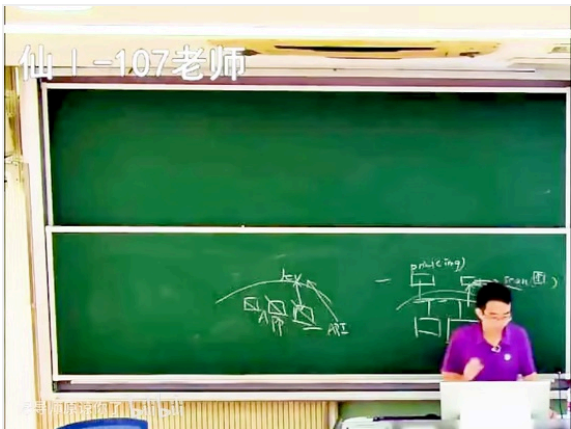
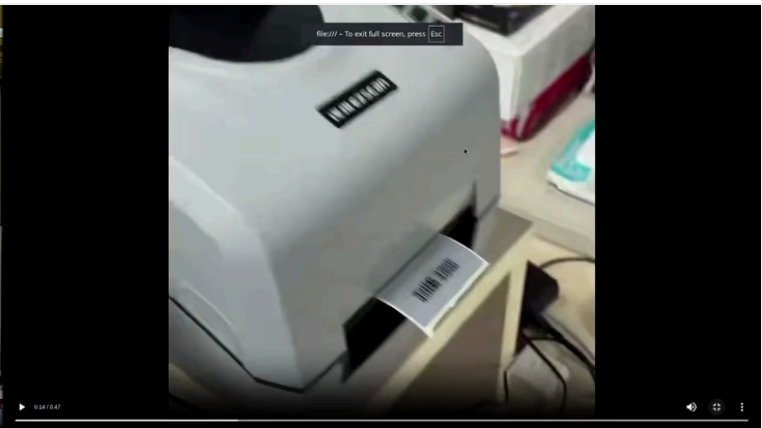
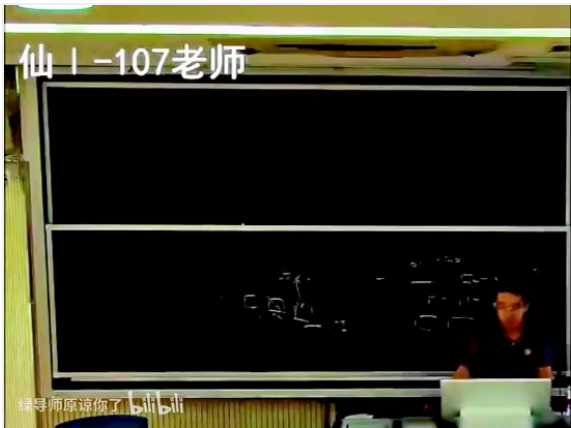
今天的答案：直接做一个 MVP

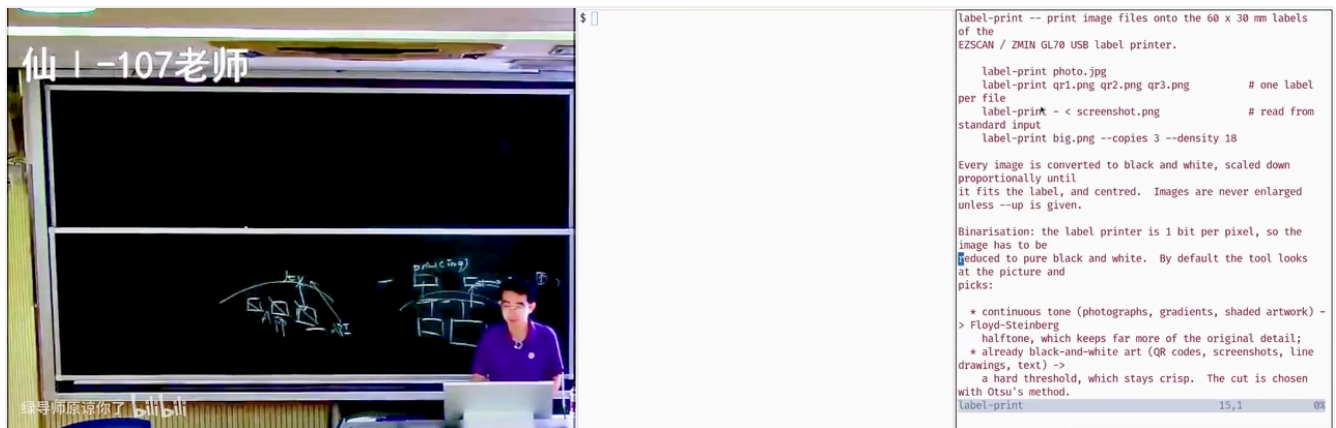
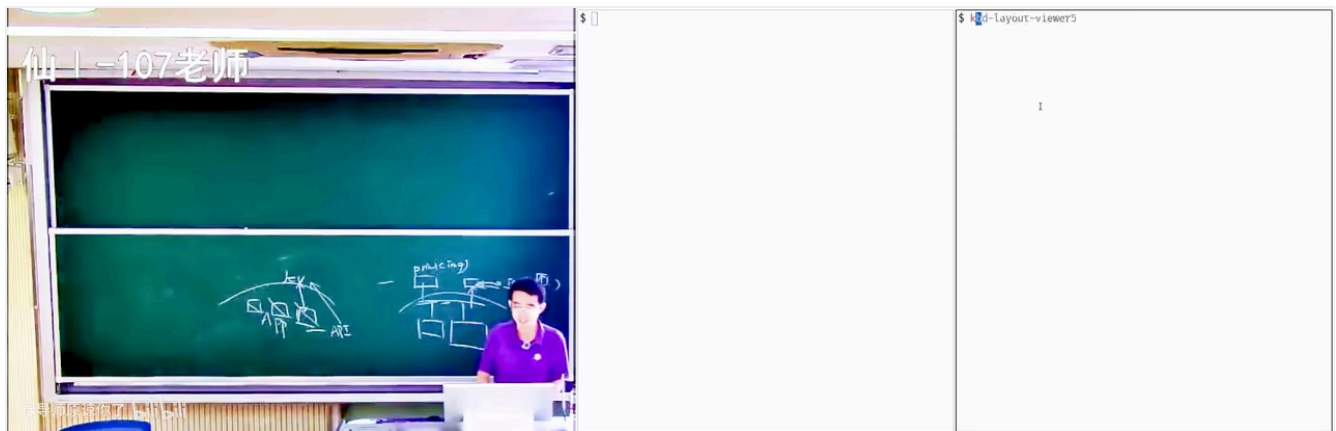
Office Assistant (GLM-5.3 直出)

- “软件”公司的彻底消亡 (lev, 豆包手机, ...)
- 以后可以想象：说好需求，自动下单零部件，3D 打印定制件，回家 Lab 就建好了



7 / 8





他赌的不是现在的 demo，而是模型很快能像人一样点手机

讲者认为“他”走在前面，关键是他押注模型会在某个时间点快到可以像人一样操作手机。做 demo 手机（原转录“德报手机”）时还做不到，但讲者判断这一天很快会来。

模型能点手机 → App 被抽象成接口 → App 可能全部死掉

讲者给出的链条是：

1. 模型快到能代替人操作手机；
2. 于是“他”把 App 抽象成各种接口；
3. 这些接口由 **Computer Use** 调用；
4. 用户要点外卖时，App 就沦为“点外卖接口”，并留下查询数据接口；
5. 如果 App 只需向 **agent** 提供 **API**，而不是给人用，那么 App 有一天可能全部死掉。

讲者强调对比：**App** 是给人用的；**API** 可以给 **agent** 提供内容。这被他视为 App 存亡逻辑的变化。

阿里整合服务做 agent/千问，说明趋势；但千问被讲者看作“未来半成品”

- 讲者举例：阿里在疯狂把旗下服务整合，向 **agent** 整合，试图用一个 千问（Qwen）这样的应用承载。
- 讲者评价千问“生不逢时”：太想把产品推出来，当时模型算力不够，可能也没有像 [JEFF] 这样智力足够、速度足够快的模型来帮用户完成各种任务。因此它像一个未来的半成品。
- 但这反而说明：讲者认为未来总有一天会来。届时整个软件公司会彻底消亡。
- 讲者反问：这条产业链上养活了多少人、多少外包公司？

标签打印机案例：AI 把“硬件细分市场 + 驱动软件”的创业逻辑杀死

- 创业设想：毕业后找细分市场；比如每个学校、每个单位都需要标签打印机，卖几百块钱一台，有利润。
- 硬件端：中国供应链成熟，这种成熟硬件基本能找到供应链。
- 软件端：在 AI 之前（原转录 AIPRAI），软件开发成本非常高；需要 Windows 驱动，还需要 Mac 驱动。越来越多人用 Mac，若没有 Mac 驱动就会抱怨，产品就卖不好。
- 今天讲者认为这些都被 AI“杀死比赛”（终结竞争）。他展示 **LabelPrint**（原转录 level print）：自己完全没管，代码由 AI 生成；AI 自己 **probe**（原转录 prob）打印机参数和

当前插入的标签，做到完全定制化——用户想要什么，就以最舒适的方式定制，而不是通用方案。

- 底层原因：讲者判断，以后生产软件、生产 **intelligence** 的成本会非常非常低。

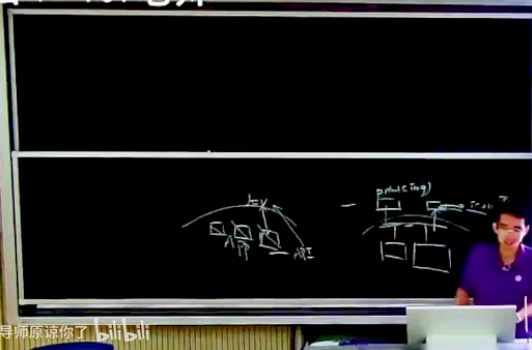
软件已死，为什么还讲软件工程？因为过去不是这样

- 讲者说，听众已经生在这个时代，已经生在“软件死掉了”的时代。
- 但还要讲软件工程，因为曾经的软件工程不是这样的；需要透过软件工程的历史来理解今天的变化。

小结：讲者用 App 接口化、阿里千问、标签打印机和 AI 生成 LabelPrint 串成一条线：模型快到能操作手机 → App 退化为 agent 调用的 API → 传统软件公司与软件产业链被重估；因此软件工程必须放回历史语境讲。

第 10 部分

仙！-107老师



绿导师原凉你了 bilibili

今天的答案：直接做一个 MVP

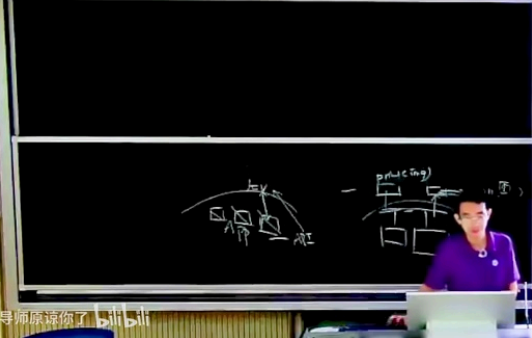
Office Assistant (GLM-5.3 直出)

- “软件”公司的彻底消亡 (Jev, 豆包手机, ...)
- 以后可以想象：说好需求，自动下单零部件，3D 打印定制件，回家 Lab 就建好了



7/8

仙！-107老师



绿导师原凉你了 bilibili

[[支撑新一轮 scaling law 的是数据分布工程：在什么阶段，以什么比例、难度、顺序和训练方式接触哪些数据，并根据学习进展，跨域迁移和泛化表现持续调整。把这件事做好，“增加可训练领域”就能成为持续有效的扩展方向。飞轮就转起来了：不断发现能够提供更可靠反馈的新领域，把更多智力活动变成可以规模化训练的对象。空间变换、图形规则归纳和几何构造可以用生成器与约束检查验证：系统架构、开发协议和故障诊断可以通过运行、模型检查、故障注入与需求变化验证：主动实验、记忆管理和需求澄清则可以根据后续预测与任务结果评价。增强后的模型又可以参与寻找领域、构建环境和验证器，并优化数据分布与训练方法，形成“发现新领域-建立反馈-优化训练分布-获得并组合新能力-改进下一轮训练”的外层循环。通过自动化能力生产过程本身，通过经过独立验证的净收益，产生飞轮。把这个画成图，适当抽取一些文字到外面]]

Review: 版本管理

- 软件是 intent, spec, impl 的组合体
- 以文件系统快照的形式存储 (版本管理)
- 所谓“软件开发”就是不断推出新的版本，直到满足客户需求
 - 这其中很自然会遇到分叉、冲突——
 - 于是你能理解 SVN, Git, Jujutsu, ... 的设计原因
- 给“从零开始构造软件”起了个头
- 从空的项目 (模糊的 intent) 开始，我们可以做什么？

今天的答案

- 直接做一个 MVP，相当于 AI 帮你补全了细节
- 例子：Office Assistant (GLM-5.3 直出)

85/lect/index.md 30,39-27 12%

为什么今天还要讲软件工程：历史智慧没有消失，可用来指导与 agent 协作

- 讲者先回答“为什么还要讲软件工程”：因为**曾经的软件工程不是现在这样**；透过软件工程的历史，可以理解今天世界上这些软件到底是怎么来的。
- 讲者预告下一个实验（尚未布置）：要求把第一个实验里的 **assistant** 和一个 **physical device** 联系起来，让它驱动任意设备——可能是手表，也可能是宿舍里已有的东西；也可以花几百元买一个小设备。
- 这种设备往往是**非标的**，可能没有 Linux 驱动，但目标就是把它“搞起来”。
- 具体故事：讲者办公室里有一块**电子墨水屏**（原文“电子墨水瓶”），它靠一个 Android app 烧写/写入，而那个 app 极度难用。
- 讲者本来可以用**豆包手机 / computer use** 方式来操作；但他做了另一件事：用 **Claude Opus 4.6**（原文“cloud open4.6”，疑为模型名）把 Android app 的 **APK** 丢进去，让它解包、逆向**蓝牙协议**。
- 因为那还是一个**标准蓝牙设备**，模型能根据代码弄清怎么把内容写到屏上；讲者也因此得到了一个一样的东西。
- 讲者认为：你们可以借助现在手头的模型来做这种事。虽然今天已经生在这个时代，但**软件工程的智慧并没有消失**。
- 因此，讲者从今天开始要讲：软件构造的历史中，大家发明了什么；再用这些理念更好地指导与 **agent** 的协作，成为一个更好的 **agent-native 软件开发者**。

讲者给出的逻辑链：

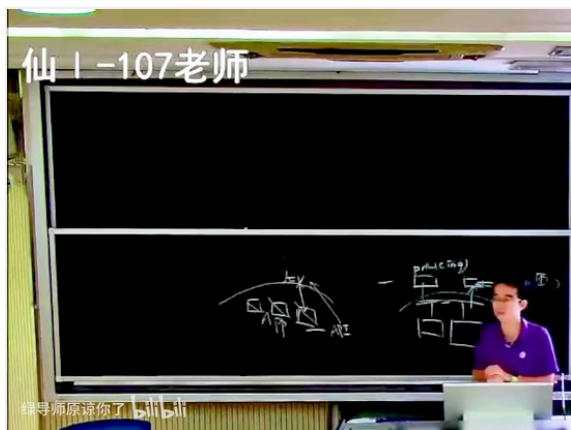
1. 今天模型已能解包 APK、逆向蓝牙协议，进而驱动非标设备（讲者举例）。
2. 即便如此，软件工程的智慧没有消失（讲者明确说）。
3. 所以要讲软件构造的历史与其中发明的理念（讲者明确说下一步）。
4. 目的是用这些理念指导与 agent 的协作，成为 agent-native 软件开发者（讲者明确说）。

（整理者串联）这个“电子墨水屏”例子为后文铺路：今天 agent 已有很强的动手能力，但讲者仍认为需要旧有的软件工程智慧来组织这种协作。

软件工程的诞生背景：硬件比码农贵得多，编程工具又极不成熟

- 讲者把时间拉回 1960 年。那时软件还是新事物。
- **硬件极贵**：一台计算机要占一个很大的房间；不一定有一层楼那么高，但确实非常大。作为最先进的设备，它的制造成本也很高。
- 讲者插入当代对比和调侃：现在厂家都懂得怎么偷工减料；最新消息是 iPhone 18 Pro 采用 QLC 闪存颗粒。
- 讲者说这好像也没关系，因为 iPhone 的主张是用户应该把所有东西存在 iCloud 上；数据坏了它也不会对你的数据负责。保修期内闪存坏了，就给你换一个主板；但重要数据应该放 iCloud。
- 回到 1960 年：大家还不知道怎么造计算机，所以**不敢偷工减料**。就像刚开始造建筑时要猛堆料，成本非常高。
- **码农当然也很贵**：越早，编程工具越不成熟；编程是一个高智力需求的工种。
- 讲者说，这一点一直到电视剧《Silicon Valley》里仍然如此。

第 11 部分



软件还是新生事物的年代

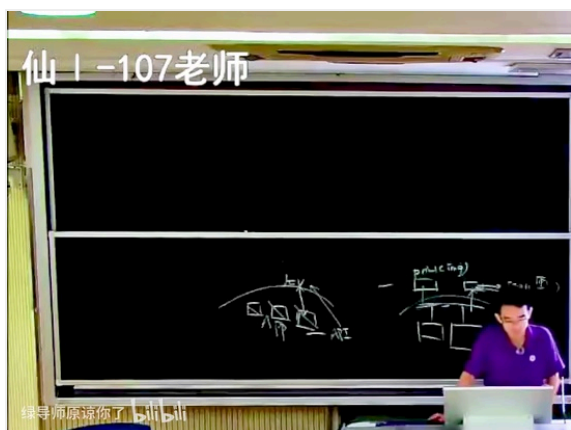
回到 1960s：昂贵的码农

- 码农很贵：越早，编程工具越不成熟，码农越贵
 - 高智力需求的工种，成本极高
 - 当然那时候硬件更贵 😊
- 码农还经常缺少对客户同理心、商业的理解等“软技能”，也不懂业务

把客户的需求翻译成软件系统

- 软件企业的任务是把**客户**的需求翻译成软件系统
 - 客户不理解计算机能做什么
 - Intent-spec-impl gap 天然存在
- 📖 这还是个年轻的职业：Dijkstra 回忆，1957 年结婚登记时，“程序员”不被承认为职业，最后登记成了“理论物理学家”。本人回忆

1 / 7



1968 年想象中的“明日学校”

计算机可以怎样教学生？



- 📖 《Boys' Life》1968 年 9 月号：终端即时判分，针对薄弱环节辅导，让学生按自己的节奏学习。封面与原刊 · Computerized School House, 第 24 页
- 📖 课堂思考：“因材施教”是意图：程序怎样判断学生已经学会？

2 / 7

早期写机器码必须画流程图：中间表示补的是人与机器的表达鸿沟

- 讲者说，这种状态一直到《Silicon Valley》电视剧里都仍然如此。
- 以前写汇编/机器代码时，大家的要求之一是要画流程图；现在学生肯定不画了。
- 原因：当时直接写机器码，如果不画流程图，就像今天让大模型直接写汇编，甚至直接从二进制序列写汇编。讲者认为大模型可以做到，但人要这样做需要“过度的预训练”（原话，带调侃）。
- （整理者串联）流程图在这里不是形式主义，而是把人的 intent 翻译成机器可执行步骤的中间表示。

程序员很贵，而且理工科训练容易留下软技能缺口

- 讲者认为码农是很贵的东西。
- 他面向 985 大学听众说：你们是层层筛选出的高智商人群，但数学训练多了以后，容易缺少对客户的同理心、对商业的理解等软技能。
- 他补了一句：不是大家都没有；他见过很多各方面都很强的“六边形战士”。但他的观察是，理工科专业里 nerdy 的人更多。
- 这类人可能不 care 业务，只想造世界上最好的操作系统，像 Linux 那样的人；讲者认为他们不是好的客户经理。
- （整理者串联）技术强 $\neq$ 能理解客户，这为下一条“软件企业要补 gap”作铺垫。

软件企业的职责：补 implementation、spec、intent 之间的 gap

- 讲者明确说，软件企业依然要承担把 implementation、spec、intent 之间的 gap 补上的职责。
- 因为公司要签合同：客户可能说“我出 1000 万美元，给你开发一个操作系统”；讲者说这个数字今天看是天价。
- 他调侃：今天 AI 会说“V我50，我也能给你开发一个操作系统”（原转写“VIVO50”，按梗校订），并补一句“今天不是星期四”（呼应“疯狂星期四 / V我50”梗）。
- （整理者串联）这里的逻辑链是：客户 intent 模糊/不完整 $\rightarrow$ 需要 spec 把意图写清楚 $\rightarrow$ implementation 又有无数技术选择 $\rightarrow$ 公司作为签约主体必须负责中间翻译和交付。

当时客户也不知道计算机能做什么：需求本身不确定

- 当时软件企业面对的客户，比如 DARPA（原转写“DPA”）官员，也不知道计算机能做什么。
- NASA 的工程师可能还会有些概念，比如要把人类送上月球；或者飞机上要做飞控系统。
- 但讲者说，他们的客户也不完全知道自己想要什么。
- （整理者串联）所以软件企业不能只“接单写码”；客户 intent 本身就要在互动中被定义出来。

程序员职业太年轻：1957 年 Dijkstra 登记职业时不能写“程序员”

- 讲者说程序员还是一个非常年轻的职业。
- Dijkstra（原转写“达舍尔”，存疑）1957 年登记结婚时要登记职业；程序员不被承认为职业，最后登记成了理论物理学家。
- （整理者串联）这说明当时社会/制度还没有给“程序员”一个稳定位置，更不用说成熟的软件工程职业分工。

大众想象超前：1968 年杂志已描绘计算机化教育

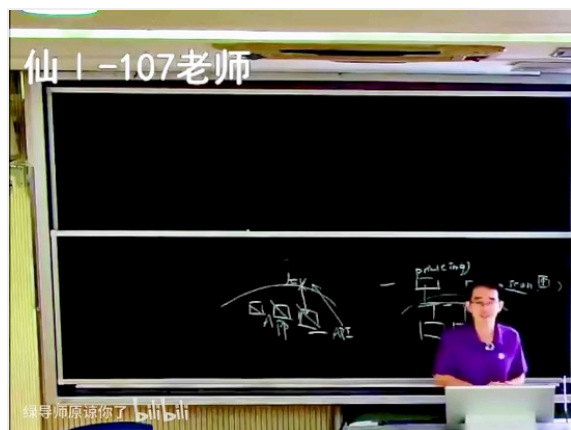
- 讲者展示 1968 年杂志材料；AI 帮他找到了“Boys like tomorrow school”（原转写；疑为《Boys' Life》的“Tomorrow's School”，存疑）。
- 那时大众已经对计算机产生各种想象：
 - 终端及时判分；
 - 针对薄弱环节辅导；
 - 让学生按自己的节奏学习。
- 讲者说，客户已经对计算机或软件产生非常过度的遐想。
- （整理者串联）社会想象超前于实现能力，会进一步放大客户 intent 与技术现实之间的 gap，所以软件企业补 gap 的职责更重要。

讲者用 AI 生成 slides 的细节作旁证

- 讲者说自己的 slides 是 AI 工具生成的（原转写“AICPLOT”，存疑）。

- 他用两个中括号标出“教给 AI”的话；AI 看到后，对他写的上面部分基本 **verbatim copy**，可能顺手修 typo。
- 其他部分是 AI 自己发挥，用了他自己的“**make slides**” skill（原转写“make size”）。
- AI 还找到了 1968 年杂志那段材料。
- （整理者串联）这个细节和前面的主线呼应：AI 也在充当从人的 intent/输入到最终产出的中间转换层；但这是整理者从讲者演示中读出的联系，不是讲者明确说出的结论。

第 12 部分



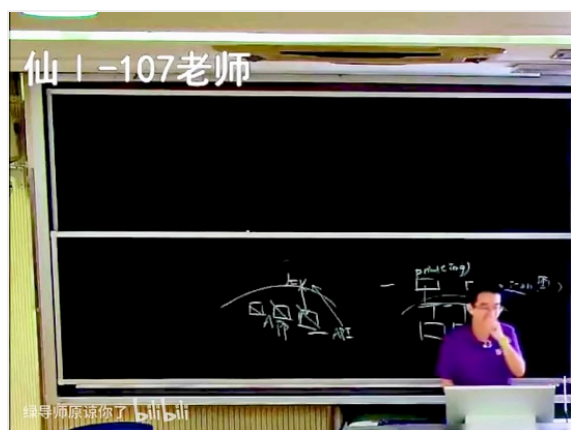
1968 年想象中的“明日学校”

计算机可以怎样教学生？



- 《Boys' Life》1968 年 9 月号：终端即时判分，针对薄弱环节辅导，让学生按自己的节奏学习。[封面与原刊](#)
- [Computerized School House](#)，第 24 页
- 《课堂思考：“因材施教”是意图：程序怎样判断学生已经学会？

1 / 2



软件还是新生事物的年代

回到 1960s：昂贵的码农

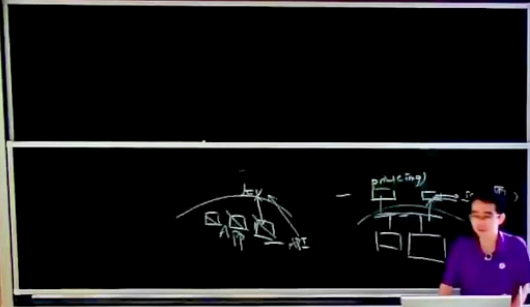
- 码农很贵：越早，编程工具越不成熟，码农越贵
 - 高智力需求的工种，成本极高
 - 当然那时候硬件更贵 😊
- 码农还经常缺少对客户同理心、商业的理解等“软技能”，也不懂业务

把客户的需求翻译成软件系统

- 软件企业的任务是把**客户**的需求翻译成软件系统
 - 客户不理解计算机能做什么
 - Intent-spec-impl gap 天然存在
- 《这还是年轻的职业：Dijkstra 回忆，1957 年结婚登记时，“程序员”不被承认为职业，最后登记成了“理论物理学家”。[本人回忆](#)

1 / 2

仙一-107老师



日学校”

软件工程和其他工程的

需求是高度定制化的

- 也因此一直认为是高度需要人
- 以“选课系统”为例，同一个
 - 名额先到先得、抽签，
 - 不满足先修课要求时，
 - 退课空出的名额交给候

程序也在执行客户的制度与

- 这些选择决定了谁能选上课

导：终端即时判分，针对薄弱环节辅导，让学生按自己的节奏学习。封面与原刊

生，第24页

图：程序怎样判断学生已经学会？

1968年已出现计算机教育想象，但现实至今未兑现，gap越来越大

- 大众已经对计算机展开各种想象：终端即时判分、针对薄弱环节辅导、让学生按自己的节奏学习。
- 1968年《Boys' Life》9月号已经畅想了这件事。
- 到今天这些都没有实现：学生仍然在上两节、每节50分钟的课程。
- 因此，想象与现实之间的 gap 越来越大。

客户画大饼与程序员现实之间的差距，说明愿景远超可实现技术

- 当时一个客户听了软件公司画的饼，说要投1000万美元做一个跨时代的软件系统。
- 客户脑子里是那种跨时代系统。
- 但回到1968年，程序员脑子里是汇编、goto。
- 所以这个 gap 毫无疑问非常大。

软件工程不同于桥梁/炼钢：物理世界只能适应，软件需求可以拍脑袋

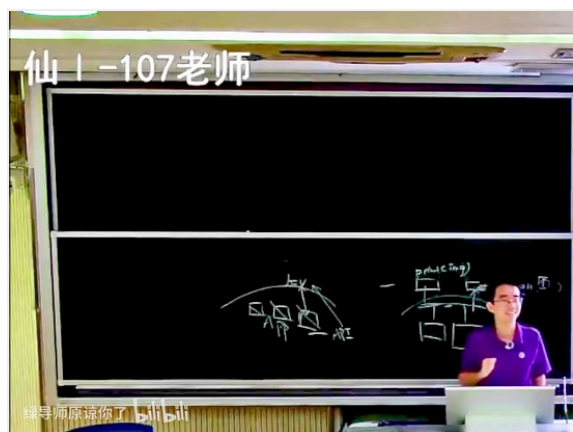
1. 讲者说，软件要实现物理世界需求在信息世界中的投影；这使软件工程和桥梁工程、炼钢工程等有很大区别。
2. 桥梁、炼钢面对自然物理世界，只能适应，不能改造。
3. 材料强度在出厂时就确定；楼盖几层、容纳多少人、人有多大，都是自然界选择下来、结果已经决定的事情。
4. 所以桥只能按自然规律做，一代代迭代，做小改进；整体是 incremental 的。对它们来说，一个彻底推导的大改进和创新非常困难。

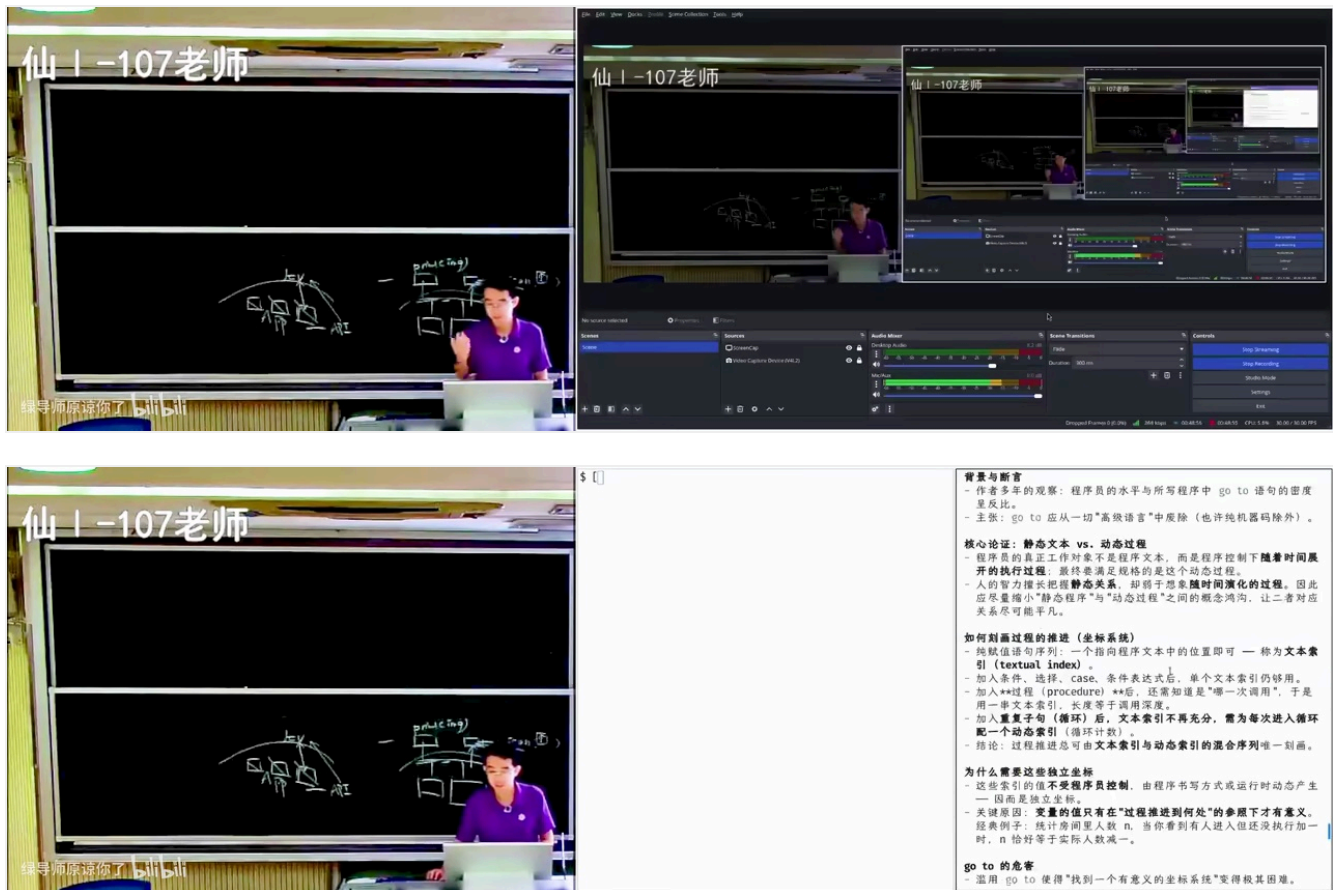
5. 软件不一样：软件需求完全是拍脑袋想出来的。1968年时就能想出前面那种系统。
6. 需求高度定制化，所以把它投影到计算机世界，一直以来也被认为是高度需要人类智慧的活动。
7. 讲者认为软件是人类智慧的巅峰；操作系统、编译器这些非常难的软件，学习它们确实需要多年专业积累。
8. 软件复杂到：提出 intent 的人自己都没想好各种 corner case 应该怎样。

选课系统例子：需求一开始就充满未定义规则、硬/软限制与 override

- 想做选课系统，就会遇到：
 - 选课规则是先到先得、抽签，还是按专业保留？（讲者称这些选项是“AI 给我 YY 出来的”）
 - 不满足先修课要求时，老师能不能特批？
- 做需求时，领导跟软件公司说好课程要有先修课；系统里也能看到，比如选修操作系统必须先修计算机系统基础。
- 但关键问题变成：这是硬限制还是软限制？谁能 override？
- 到最后一秒钟学生毕不了业时，学校所有口子都能给你开：只差这门课，就马上能选上。
- 这就是例外性需求。
- 所以讲者得出结论：软件需求是高度定制化的。

第 13 部分





需求高度定制化且连接物理/社会/人的偏好，所以软件难做

- 讲者用“你最后就差这门课，马上就能选上这门课”说明**例外性需求**：需求不是标准化、可穷举的。
- 讲者的归纳：软件需求高度定制化、非常复杂，连接了物理世界、社会、人的偏好等，因此做软件很难。
- 当时业界发现：**进度、成本、质量一起失控**——开发进度不能预测、软件开发成本崩溃/失控、延期交付。
- （整理者串联）链条：需求非标准且复杂 → 软件行为难预测 → 进度/成本/质量同时失控。

“goto 有害”不是禁 goto，而是反对让它破坏可读性

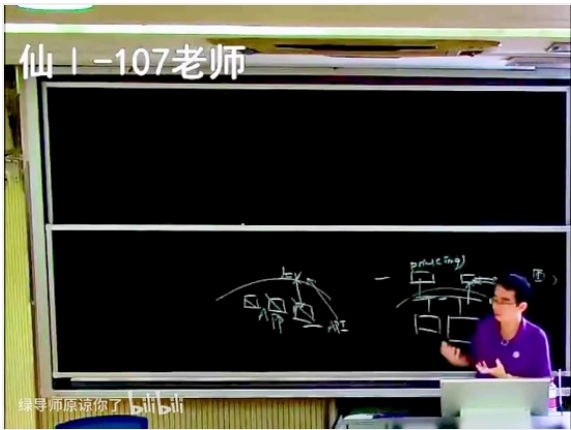
- 那时计算机科学家已观察到软件中的有趣问题。Dijkstra（图灵奖得主）提出 **Go To Statement Considered Harmful**；大家学编程都学过：大量 goto 会让程序难理解。
- 但 Linux 内核里有很多 goto。C 中一种 idiomatic 用法是 **goto release**：

1. 函数最后设 `release` 标签；
 2. 在 `release` 处逐个检查函数里申请的所有资源，并逐一释放；
 3. 任何时候在中间申请资源，就在 `release` 处加释放检查/释放；
 4. 不允许中途直接退出；想中途退出时必须 `goto release`。
- 讲者认为这样程序基本上就能正确，并解决 C 语言里“随时申请、从不释放”的大问题；他把“随时申请、从不释放”联系到做 OJ 留下的坏习惯。
 - Dijkstra 先发表暴论：程序员的水平随着 `goto` 的密度上升而下降。讲者评价：背后有一定道理，但不绝对；关键不是 `goto` 到底 harmful 不 harmful，而是是否作为正确机制使用。若只允许 `goto release` 这种用法，`goto` 就不 harmful，还挺好。
 - 其背后考虑：希望你写出人能读的代码；人能读才可能维护下去；能维护，软件世界才能相对正常运转。链条：可读 → 可维护 → 软件世界正常运转。

1972 《The Humble Programmer》：机器越强，需求与不确定性越膨胀，才有软件危机

- 1972 年 Dijkstra 的图灵奖演讲 The Humble Programmer (《谦卑的程序员》)。讲者说现在可以在 AI 帮助下直接读原始文稿。
 - 演讲解释为什么会有软件危机：
 1. 如果永远只写 OJ 小程序，不会有软件危机：算完就完，程序调对后永远不需要维护。
 2. 但机器变强后，软件需求不断膨胀。
 3. 你要承担越来越多。
 4. 机器越强 → 承担现实世界的 intent (意图) 越多 → intent 越多 → 不确定性越多 → 实现越复杂。
 5. 越来越复杂以后.....
 - (整理者串联) 这与前文相扣：需求因机器能力而膨胀，软件持续面对更多现实世界的不确定性与复杂性。
-

第 14 部分



背景：个人史与职业的诞生

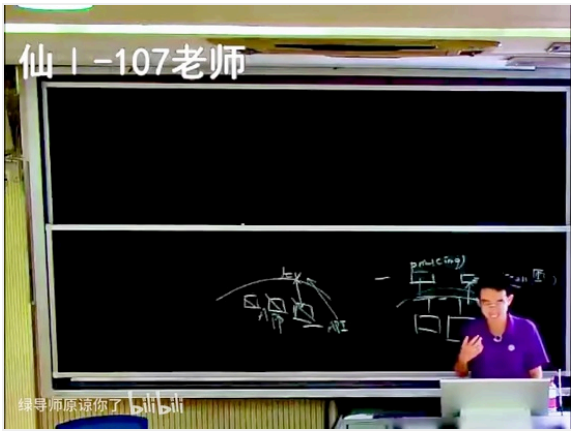
- 1952 年成为荷兰第一位“程序员”：在理论物理与编程间摇摆时，上司 van Wijngaarden 劝他：计算机必将长存，何不由你来把编程建设成一门受人尊敬的学科。
- 1957 年结婚登记时，阿姆斯特丹市政当局不承认“程序员”是职业，只能填“理论物理学家”——足见当年编程职业化之迟缓。

诊断：为什么会有“软件危机”

- 早期机器是实验性的、独一无二的、又慢又小：程序员地位卑微，其工作被视作机器的附属。由此留下两个错误观念：**真正的程序员应当“爱解谜、爱耍聪明技巧”**，以及**编程不过是优化效率**。
- 天真的期待：机器变强后编程自然不再是问题。结果相反——机器强了几个数量级，社会野心也成正比膨胀。程序员被夹在“目的与手段”的张力中，于是陷入软件危机。
- 作者认为次要原因是 I/O 中断、多级存储；**主因恰恰是机器变得太强**。
 - 没有机器时，编程根本不成问题：有了一台小机器，编程成了小问题；如今有了巨型机器，编程就成了同样巨大的问题。
- 严厉批评第三代计算机的设计（以 duty cycle 定义 performance、price 只算硬件），令他度过“职业生涯最黑暗的一周”；并讽刺“变得多所以设计不坏”的逻辑，如同“吸烟的人多所以吸烟有益”。
- 主张：应当像评审论文一样公开评审新计算机。

软件史的几块里程碑

- **EDSAC**：子程序库与闭子程序是“最伟大的软件发明之一”，支撑了抽象的基本模式。
- **FORTRAN**：当年极为勇敢且成功，但如今作为思维载体已过时、浪



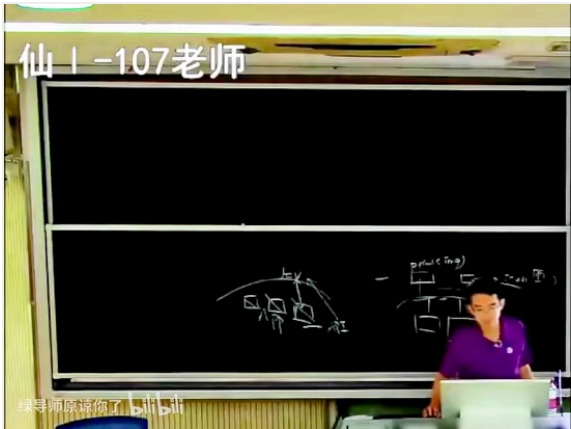
Software Crisis

进度、成本、质量一起失控

- 软件开发进度无法预测
- 软件开发成本崩溃
- 质量、维护都是问题

E. W. Dijkstra 的“暴论”

- 暴论：Go To Statement Considered Harmful (1968)
 - 挑战性的开场：程序员的水平随程序中 goto 密度上升而下降；背后的关切是人的推理能力有限，程序结构应帮助人理解执行过程。
- The Humble Programmer (1972 ACM Turing Lecture)
 - 硬件越强，社会越敢提出更大的目标：新的野心把软件开发推向能力边界。



and made "higher and higher". Each time that happened, Polanyi, along with "other boys", she says. If the case that person in fix it.

Thoughts of that math PhD years of "discovering things interesting things". She then looked on this strange as was helping to create in it she has compared to "The Exiles, especially, continue fascinate her not just for coming them, but more patients they follow in and prevent them. "I think the events", she says.

So in 1966, when she saw the MIT Instrumentation Lab recruiting programmers to work on software for what would become the Apollo program, Hamilton jumped at the chance. Initially, she was assigned what was presumed to be a low-impact project, writing the code that would kick in if an unmanned mission aborted. The higher-ups were "so sure this wasn't going to happen" she says, that she actually named her program "Fudge It". Within one of those moments she felt she found herself in high

MIT Technology Review

Just released

35 Innovators Under 35

Subscribe to meet the young trailblazers that are making waves across AI, climate tech, and more. Plus, get bonus AI content.

CLAIM 15% OFF

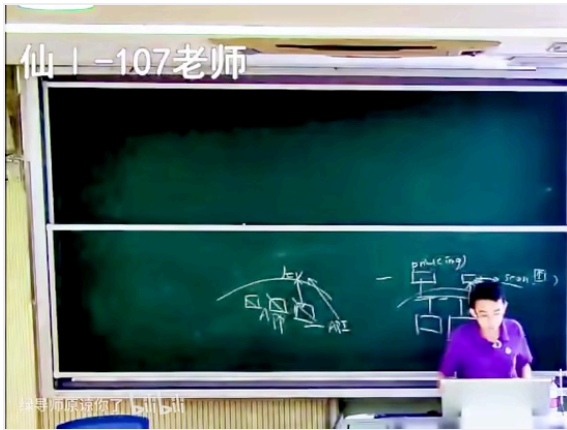
Cookie Policy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze site usage. See our cookie policy

Accept all cookies

Cookie settings

SUBSCRIBE & SAVE 15% OFF



Menu

MIT Technology Review

Sign inSUBSCRIBE & SAVE 17%

and made “foghorn and fire engine sounds” when it crashed, she says. Each time that happened, she’d have the programmer at fault pose for a Polaroid, along with “documentation of where [the mistake] came from,” she says. If the same error occurred again, they could bring in that person to fix it.

Thoughts of that math PhD faded. After years of “discovering things and inventing things,” she realized she was hooked on this strange new field she was helping to create in an atmosphere she has compared to “the Wild West.” Errors, especially, continued to fascinate her: not just finding and correcting them, but tracing the patterns they follow in order to predict and prevent them. “I thought of them as the enemy,” she says.

So in 1964, when she saw an ad from the MIT Instrumentation Lab recruiting programmers to work on software for what would become the Apollo program, Hamilton jumped at the chance. Initially, she was assigned what was presumed to be a low-impact project, writing the code that would kick in if an unmanned mission aborted. The higher-ups were “so sure this wasn’t going to happen,” she says, that she actually named her program “Forget It.” (When one of these missions did abort, she found herself in high

Hamilton with Apollo code, 1969. MIT Museum

POPULAR

A fundamental flaw leaves LLMs strikingly vulnerable to attack

Will Douglas Heaven

AI’s recursive self-improvement might not come so quickly after all

Michelle Kim

How why AI agents be and what to watch

Grace Hacking

Cookie Policy

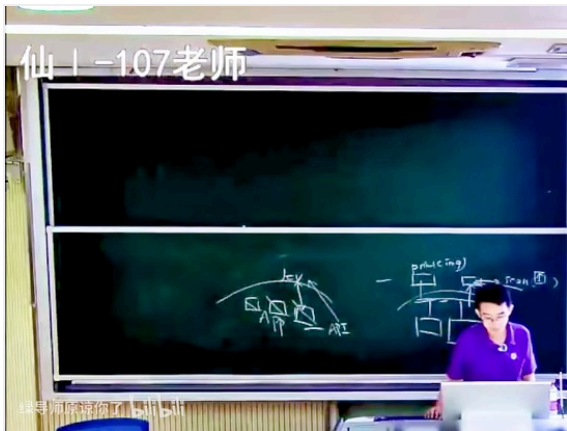
We use cookies to help you navigate our site and improve your experience. See our cookie policy

Accept all cookies

Cookie settings

SUBSCRIBE & SAVE 17%

Sign in



Menu

MIT Technology Review

100%SUBSCRIBE & SAVE 17%

In 1961, Hamilton brought this outside-the-box approach to her next position, inventing a new error protocol for MIT’s massive, unruly SAGE air defense system. This computer took up a whole warehouse, and made “foghorn and fire engine sounds” when it crashed, she says. Each time that happened, she’d have the programmer at fault pose for a Polaroid, along with “documentation of where [the mistake] came from,” she says. If the same error occurred again, they could bring in that person to fix it.

Thoughts of that math PhD faded. After years of “discovering things and inventing things,” she realized she was hooked on this strange new field she was helping to create in an atmosphere she has compared to “the Wild West.” Errors, especially, continued to fascinate her: not just finding and correcting them, but tracing the patterns they

Hamilton with Apollo code, 1969. MIT Museum

POPULAR

A fundamental flaw leaves LLMs strikingly vulnerable to attack

Will Douglas Heaven

AI’s recursive self-improvement might not come so quickly after all

Michelle Kim

Cookie Policy

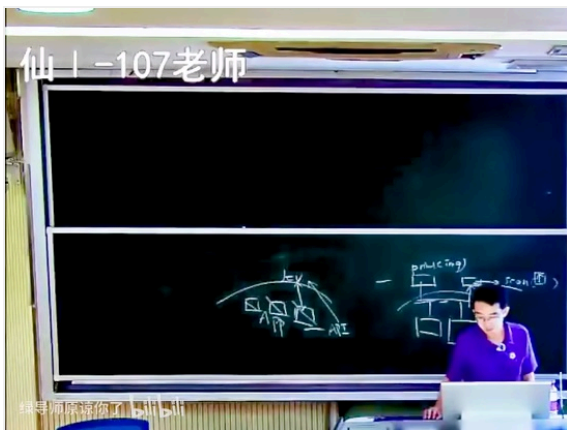
We use cookies to help you navigate our site and improve your experience. See our cookie policy

Accept all cookies

Cookie settings

SUBSCRIBE & SAVE 17%

Sign in



Menu

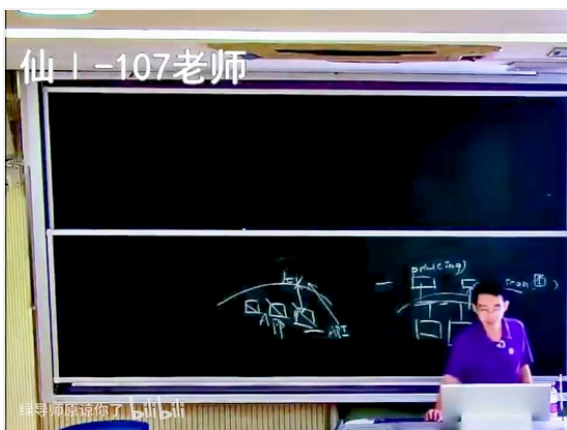
MIT Technology Review

SUBSCRIBE

protocol for MIT’s massive, unruly SAGE air defense system. This computer took up a whole warehouse, and made “foghorn and fire engine sounds” when it crashed, she says. Each time that happened, she’d have the programmer at fault pose for a Polaroid, along with “documentation of where [the mistake] came from,” she says. If the same error occurred again, they could bring in that person to fix it.

Thoughts of that math PhD faded. After years of “discovering things and inventing things,” she realized she was hooked on this strange new field she was helping to create in an atmosphere she has compared to “the Wild West.” Errors, especially, continued to fascinate her: not just finding and correcting them, but tracing the patterns they

Hamilton with Apollo code, 1969. MIT Museum



Menu

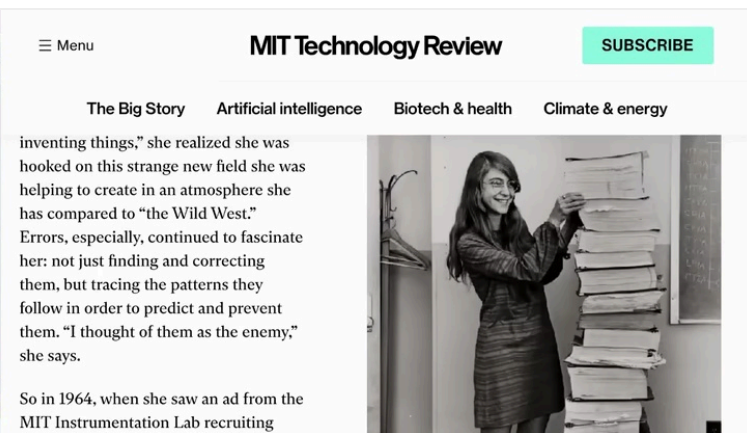
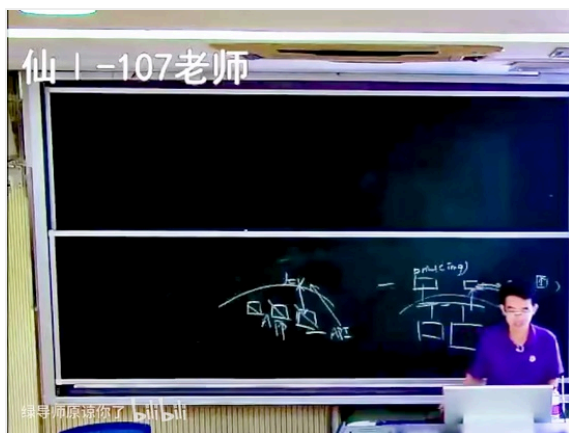
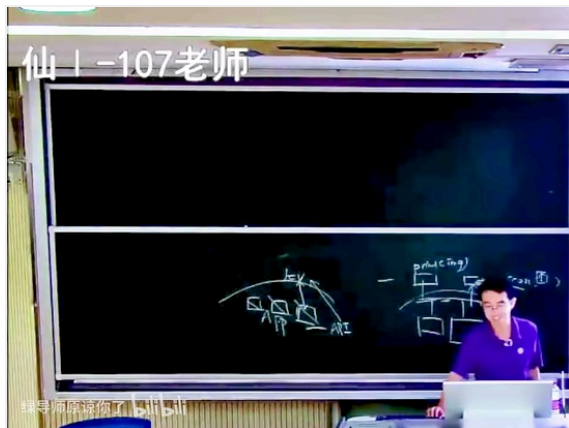
MIT Technology Review

SUBSCRIBE

them. “I thought of them as the enemy,” she says.

So in 1964, when she saw an ad from the MIT Instrumentation Lab recruiting programmers to work on software for what would become the Apollo program, Hamilton jumped at the chance. Initially, she was assigned what was presumed to be a low-impact project, writing the code that would kick in if an unmanned mission aborted. The higher-ups were “so sure this wasn’t going to happen,” she says, that she actually named her program “Forget It.” (When one of these

Hamilton with Apollo code, 1969. MIT Museum



机器越强，承担的 intent 越多，软件就越复杂

- 讲者开头的链条：机器越强 → 承担现实世界的 **intent** 越多 → intent 越多 → 不确定性越多 → 实现越复杂。
- （整理者串联）这条链把软件工程问题定位为能力扩张的伴随结果：不是先有工具，而是复杂性先逼出方法。

Dijkstra：用“良好分解”组织软件；保守力量来自教育与旧程序用户

- 讲者说 Dijkstra 对软件的一个思考是 **nicely factored solutions**（原转写 Nicely factor solutions），即良好的分解。
- 讲者认为，构造任何软件系统时都在不断发现这种分解；我们无时无刻不在用这种想法来组织软件。
- 例：讲者写 **LiveMate**（原转写 live mate）来驱动标签打印机和扫描仪，它也是一个 nicely factored solution。

- Dijkstra 另一个观点：保守力量主要来自教育机构与舍不得重写旧程序的用户。讲者认为这些观点今天依然让人有醍醐灌顶之感。
- 历史背景：Dijkstra 于 1972 年获图灵奖；大致同一时期，软件工程成为焦点，因为软件开始失败。1968 年 NATO 软件工程会议；讲者建议直接读原始 summary，自己没读。
- 现场细节：讲者让模型在后台跑，提到 DeepSeek Flash（原转写 deep sk flash），若模型足够大可能做 web search。

Hamilton、Apollo 代码：软件工程诞生的标志

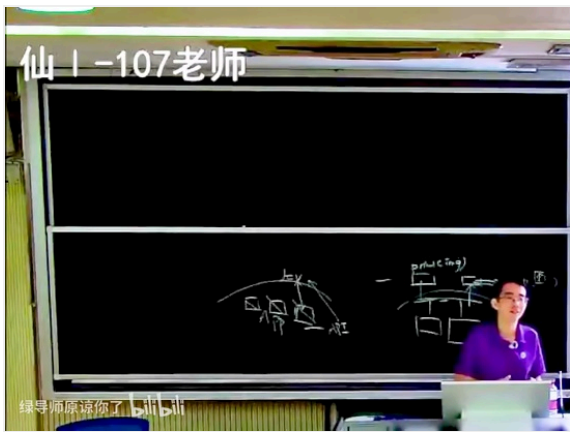
- 讲者说差不多同时，Margaret Hamilton 和团队提出“软件工程”这一概念；相关帖子/材料值得读。
- 标志性照片：Hamilton 与 Apollo 代码（原转写 Vista polo code）；讲者认为它代表软件工程学科的诞生。
- 材料讲到：1969 年，经过不懈排查错误，这位最初的软件工程师把宇航员送上月球；她还有传记。

核心是防御性编程：把“绝不可能”当成可能

- 讲者由此落到今天的训练：你们都要防御性编程。例如第一节学 SICP 时，就要求写 assertions（原转写 SERGEONS）/ specifications。
- 关键立场：把一个绝对不能发生的假设也当成可能发生；这是防御性的、为未来思考的智慧。
- Hamilton 的假设：软件在任何时候都可能违反假设，但仍必须能工作；所以她设计出成功的软件。引语：The never going to happen can happen。
- 讲者认为：回头看原作，这些思想在今天反而更容易理解，比教科书经过一手、二手、三手、四手消化后要更好。
- 进一步：甚至要假设硬件不可靠；无论如何，在软件里做兜底机制，让程序 crash 后整个系统依然工作。讲者称这就是软件工程的起点。

小结：软件工程的起点，在讲者这里不是工具堆叠，而是把“绝不可能发生”纳入设计假设的防御性思维。

第 15 部分



一道保护，NASA 觉得太不可能，只允许她“记一笔”。

阿波罗 8 号（首次载人绕月）上，宇航员真的误操作输入了 P01，导致数据当场消失，只能把她叫来恢复。她后来说话时还在笑：“这是 Lauren bug！”

4. 阿波罗 11 号：1201/1202 告警

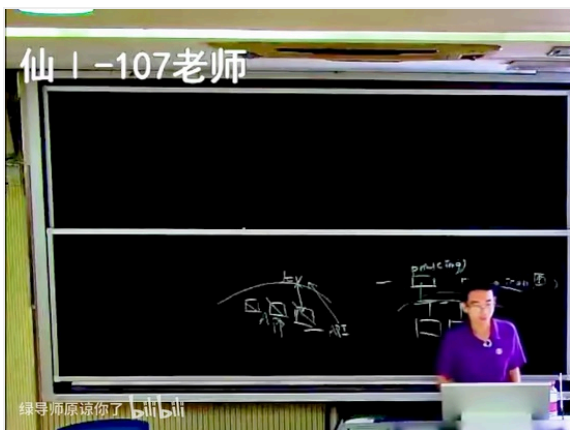
最惊险的一幕发生在登月舱 Eagle 即将着陆时：告警 1201 和 1202 连续闪出五次，意思是计算机过载，任务被挤掉——一部分原因是交会雷达的开关位置不对，喂进了多余数据。

换作别的系统，这叫“死机”；但这套软件设计时就让执行程序（Executive）能**优先级调度**：过载时自动杀掉低优先级任务、重启并让关键的着陆任务继续跑。休斯顿的年轻工程师 Jack Garman 对准备忘录喊“这没问题”。Steve Bales 给出 GO，Eagle 最终带着约 25 秒的燃料余量着陆。Hamilton 说，看看软件在降落过程中当场自愈，那种感觉是“纯粹的恐惧（sheer terror）”。

5. 遗产：从“排错”到“软件工程”

- 她首创并力推“**software engineering**”（软件工程）这个词，把当时被当作“手工活”的工作变成一门严肃学科。
- 核心方法：不只修 bug，而是**追踪错误模式、预测并预防**：让代码在硬件故障、人为失误、意外时序下仍能检测、恢复。
- 她后来的公司 Hamilton Technologies 开发 **Universal Systems Language**，主张“从设计上让常见错误无法发生”，而不是事后补救。
- 2016 年获总统自由勋章：2017 年那张她站在自己团队代码打印稿旁的照片被做成乐高小人。

Recap：用“绝不可能发生”的假设去设计软件，是这段故事的核心方法——从 Forget It, Lauren bug 到 1201/1202 告警，每一次“意外”都验证了错误恢复与优先级调度的价值，也让“软件工程”从



把软件当作工程：Apollo

Margaret Hamilton 与她的团队

- Margaret Hamilton
 - 领导 MIT 仪器实验室的软件工程部门，参与 Apollo 飞行软件开发，推动软件工程观念获得承认。NASA / MIT 介绍

“能工作”还包括发生意外的时候

- 阿波罗 11 号登月下降时，计算机过载，出现 1201/1202 告警。亲历者解释
 - 软件的优先级与恢复机制，让关键的下降控制任务得以继续。
 - MIT 团队事先充分测试过恢复能力，地面团队才有依据判断可以继续登月。
- 这个案例里的工程工作包括：决定必须保住什么、实现恢复机制，并验证它确实有效。

程序崩溃后系统仍要工作：这是软件工程的起点

- 讲者提出的核心命题：程序 crash 之后，整个系统依然要能够工作。他认为这就是软件工程的起点。
- 例子：阿波罗 11 号登月下降时发生警报；但软件因为有**优先级**和**recovery**，让关键的下降控制任务得以继续。
- 当时还没有“软件工程”这个学科，他们是从 **first principle** 去思考：如果这个时候要构造一个真正好的、高质量的软件，应该怎么做？
- 讲者概括他们的 first principle：“**never going to happen can happen**”——不该发生的事情，也可能发生。
- 逻辑链：
 1. 程序会 crash，意外会发生。
 2. 但整个系统、尤其是关键任务，仍然要能工作。
 3. 所以必须设计优先级、恢复机制，而不能只假设正常路径。

- 4.（整理者串联）这把“意外”从例外变成了工程前提；讲者认为软件工程就在这个意义上、差不多那个时间点诞生。

软件工程：在成本、时间、团队约束下交付可工作、可维护、可生存的软件

- 在成本、时间、各种团队约束下交付软件，当时已经成为一个很大的挑战。
- 讲者给软件工程的任務：在成本时间约束下，把人的意图——你想设计的产品，或者客户的要求——变成：
 - 可工作的软件；
 - 可维护的软件；
 - 可生存的软件。
- 因为随着时间推演，需求可能会变化。
- 这与“never going to happen can happen”呼应：变化和意外也要被纳入设计考虑。

做 OJ 的隐藏假设：题目一定对、一定能做

- 讲者把问题拉回学生经验：做 OJ 时，你假设题目是对的，假设题目是能做出来的。
- 讲者认为，这个隐藏假设本身并不对。
- 当你反复强化这个假设时，你会 **overfit** 这个假设。
- 应该假设面对的是 **开放世界**：
 - 题目可能是错的；
 - 题目的条件随时可能变化；
 - 题目甚至可能变成另外一个题目。
- 那要怎么做一个软件来解决它？
- 讲者承认：编程是一件事；通过 **online judge** 训练的是逻辑思维、算法思维、championship 能力。理解软件设计这些东西也有好处。
- （整理者串联）这相当于把 Apollo 的“意外会发生”迁移到 OJ 和需求假设上：不能把“题目永远正确”当作工程前提。

IEEE 定义：软件工程 = 把工程化引入软件，从个人英雄主义到 checklist

- 讲者引用 IEEE（原音近 I trio e）的定义：software engineering 是 **a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software**；也就是把 engineering 应用到软件。
- 工程化意味着软件不再只是 **个人英雄主义**。
- 在 Hamilton 的那个时代，需要一个好的 leader；换一个人，项目就失败。
- 软件工程把它变成可以拿一个 **checklist**：
 - 如果 checklist 上 1、2、3.....10 项全部满足；
 - 不能保证软件成功；
 - 但失败的概率会小很多。
- 软件工程的特点：有纪律、可量化、覆盖整个软件生命周期。

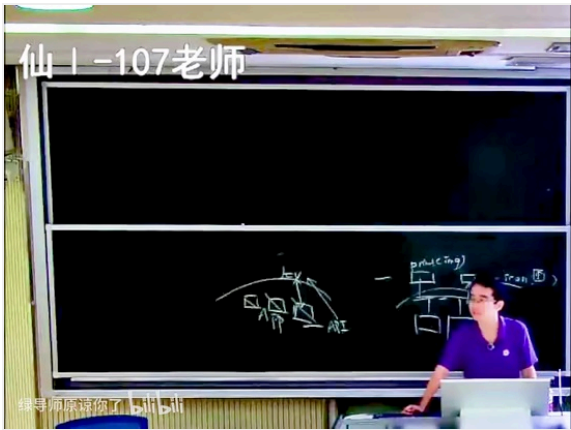
软件工程的智慧来自失败与数亿美金教训

- 在 AI 还没有大杀四方的时代，软件工程这个学科从刚开始的大量失败中发展。
- 也可以说，是从数亿美金的教训中得到的。
- 例子：炸一个火箭。讲者说很多时候可能都跟软件工程有关系。
- 大家看到成功、看到失败，多多少少会总结出经验和教训。

接到开发任务不能立刻开干：intent / spec / implementation 之间有 gap

- 回到 PRIERM 时代（原音；疑为 PM/项目管理语境），讲者说大家其实也很清楚 **intent / spec / implementation**（原音近 information）这样的 gap。
 - 如果你接到一个开发任务，不能立即安排“大家去干吧”。
 - 讲者认为，任何一个 leader 都不应该这样做。
 - 即便最终的产物一定是码农写出来的。
 - （整理者串联）所以 leader 不能跳过从 intent 到 spec 到 implementation 的 gap，直接进入编码；这正是软件工程要处理的问题。
-

第 16 部分

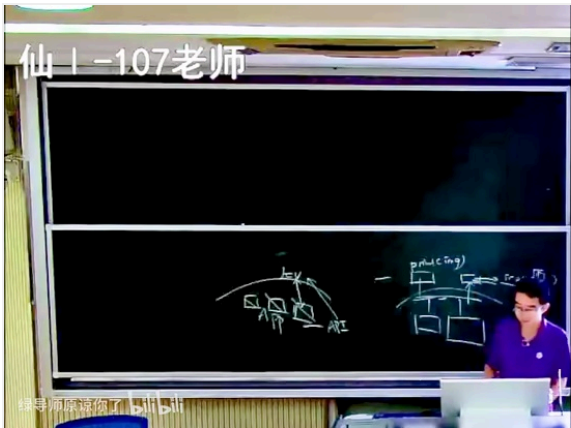


软件工程的智慧

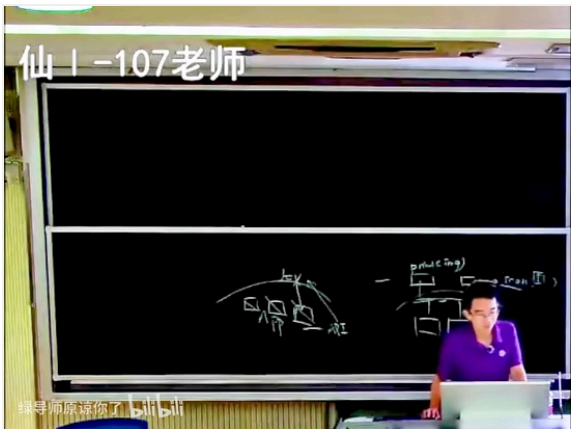
回到 Pre-LLM 时代

- 接到开发任务，肯定不能立即安排码农干活——虽然最终产物是他们生产的
- 如何减少 **intent-spec-impl gap** 带来的影响？

1 / 8



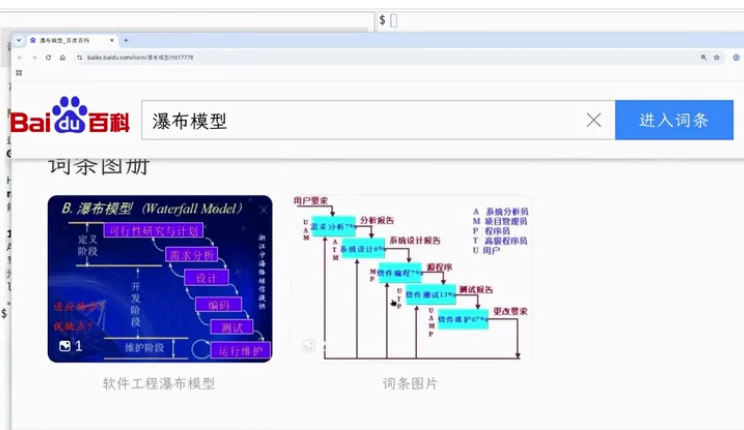
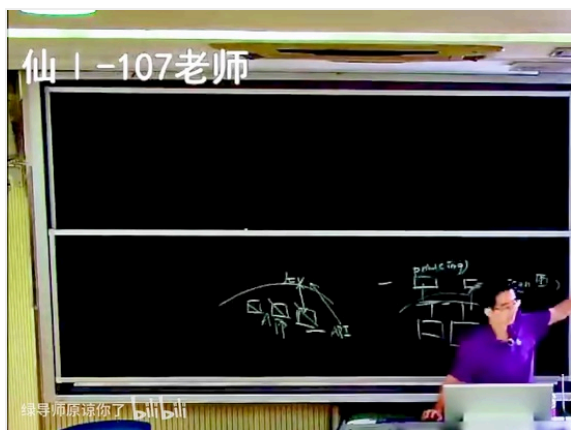
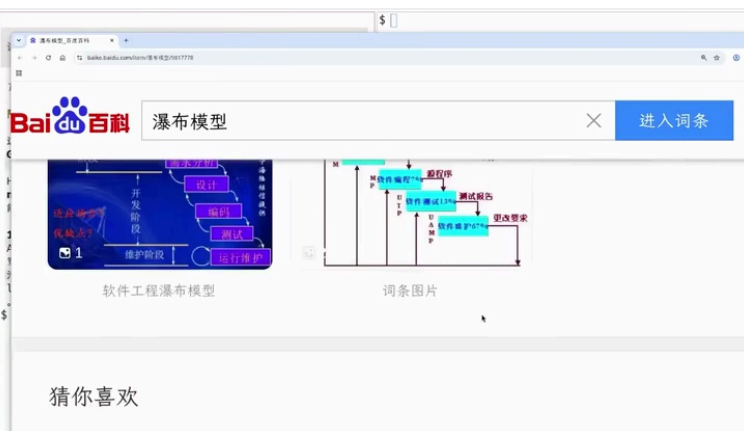
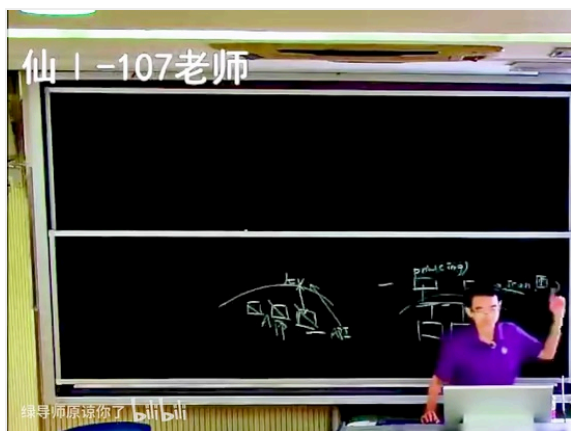
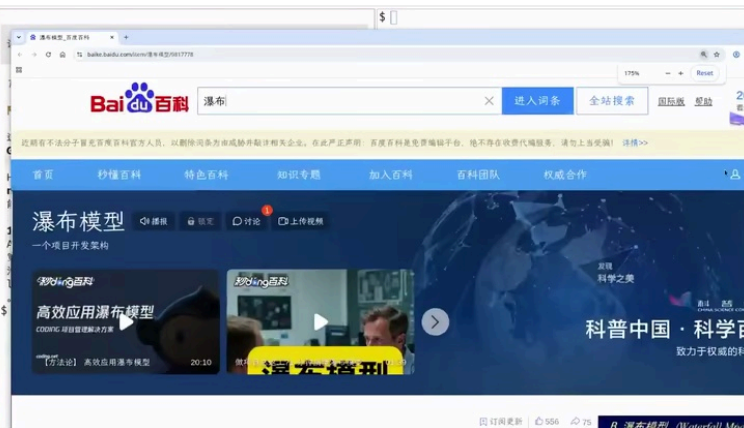
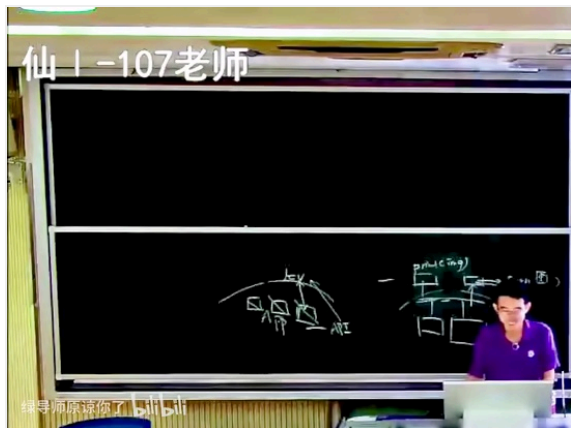
`$. /scripts/read-tui article/7001/royce.md`



瀑布模型

科普中国·科学百科

瀑布模型 (Waterfall Model) 是一个软件开发模型，它把软件开发生命周期划分为几个阶段，每个阶段都有明确的任务和输出。在瀑布模型中，每个阶段都必须完成才能进入下一个阶段，就像一个瀑布一样，从一个阶段流到下一个阶段。瀑布模型的优点是结构简单，易于理解和实施。缺点是它不能很好地适应需求变化和迭代开发。瀑布模型通常用于大型、复杂的软件项目，如操作系统、数据库系统等。



Leader 不能说“大家去干吧”：任务必须先划分和指派

- 讲者认为，任何一个 leader 都不应该一上来就安排“大家去干吧”。
- 理由：即便最终产物一定是程序员（码农）写出来的，不同人看到的是不同部分；软件如果要拆分，leader 就需要指派哪个部分由谁来做。
- 例子：打印机和扫描仪同时接到电脑上时，讲者起了两个 agent、开了两个窗口，分别对应打印机和扫描仪。讲者觉得这很正常：需要一个划分任务的过程，不能是 self-organized。

- 这里的链条是：有多个部分/设备/模块 → 不同人看到不同部分 → leader 必须划分任务并指定负责人 → 不能只喊“大家去干”。

从空 repo 开始：讲者追问是否直接一句话做 MVP，再转向 1970 年的软件工程问题

- 如果涉及划分任务，那么从空 repo 开始时该怎么做？讲者反问：是直接开发、直接把一句话交给它做 MVP 吗？
- 他提醒：1970 年的时候，软件工程刚刚诞生，当时还没有 minimum viable product，也没有大语言模型。
- 当时的的问题是：怎么按时、不超预算地交付大型软件。
- 这引出软件工程领域的开山之作：Winston Royce, “Managing the Development of Large Software Systems” (1970)。

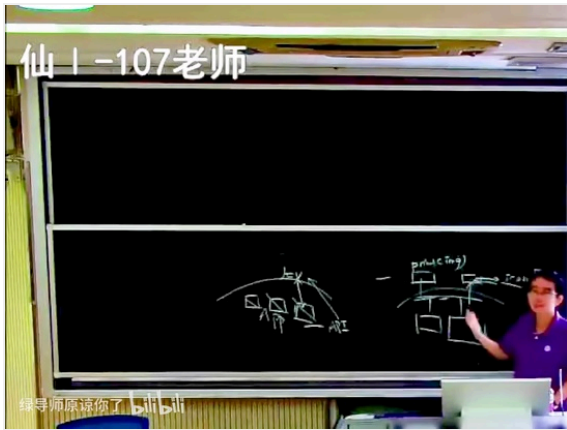
瀑布模型图来自 Royce 文章，但 Royce 实际在批判它

- 软件工程课里常见的 waterfall model 图：从上到下。只要学 CS 或软件相关，多半在软件工程课上见过。
- 讲者现场搜索百度“瀑布模型”，没有打开秒懂百科，觉得有点搞笑；把图放大展示。这张图就来自 Winston Royce 的那篇经典文章。
- 讲者强调最有趣、最关键的一点：大家都讲、都学瀑布模型，但这篇文章其实是批判瀑布模型的。因此必须回到原文。

讲者读原文的方式：agent 入库论文 + 命令行读论文

- 讲者说自己的读文章方式与一般人不同：昨天备课时需要这篇文章，他直接找到论文链接，然后对自己的 agent 说“入库这个 paper”，于是得到文字整理版。
 - 他的 AI 工具会把 Markdown 图片直接嵌入；他还有一些命令行工具：可以打印文本、提取图片，也有类似 less 的工具，能在命令行里直接读论文。
 - 瀑布模型就来自这篇文章。
-

第 17 部分



仙 | -107老师

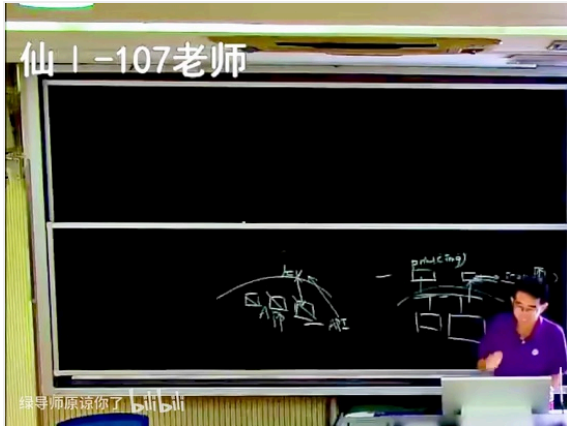
Royce 与瀑布模型

如何按时、不超预算地交付大型软件？

- Winston Royce: [Managing the development of large software systems](#)
 - 这篇论文画出了后来被称为“瀑布模型”的经典阶段图
 - 1970 年的背景：航天器任务规划、指令与飞行后分析，时间和存储都有硬约束。
 - 原文在这张图之后就指出：这样的实施方式风险很高，容易失败。

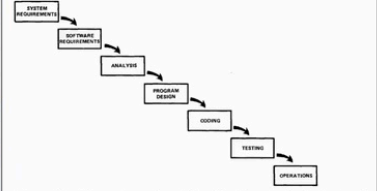
“瀑布假设”

- 希望迭代只发生在相邻阶段之间
- 实际测试中暴露的问题，却会一路回溯到设计与需求



仙 | -107老师

of this is that as the design proceeds the change process is scoped down to manageable limits. At any point in the design process after the requirements analysis is completed there exists a firm and closeup, moving baseline to which to return in the event of unforeseen design difficulties. What we have is an effective fallback position that tends to maximize the extent of early work that is salvageable and preserved.



[Image #1: Figure 2 - waterfall-style flowchart of implementation steps]
Figure 2. Implementation steps to develop a large computer program for delivery to a customer.

I believe in this concept, but the implementation described above is risky and invites failure. The problem is

royce.md 79-108/779 10x

软件工程课都教瀑布模型，但原文其实批判瀑布模型

- 软件工程学生都学 瀑布模型；但讲者指出，这篇被当作源头的文章实际是批判瀑布模型的。
- 因此不能只记二手结论：一个东西经过多次转述、消化，得到的可能已经不是原物；必须回到原文。
- 讲者提到时代背景：1970 年大家几乎什么都没有，只能从 first principle 去思考。
- 讲者读到/展示经典图：需求 → 系统需求 → 软件需求分析 → 设计 → 代码 → 测试 → 上线。
- 讲者引述：它后来被称为 瀑布模型的原始出处，也是被误读最严重的一篇。误读可能来自最早一两篇文章没有完整描述原文观点。

原文可重述为“小程序两步就够”，这与 online judge 对得上

- 讲者重述：一个程序小到可以由一个人自己做时，**两步就够了**。
- 这对应学生的 **online judge**。
- 站在今天的软件实践看，仍能很好对上原文当时的一些思考。
- 但讲者转折：这种小流程**很容易失败**，一旦扩展到瀑布就更是如此。

瀑布容易失败的第一性原理：测试是链条上第一个“物理事件”

- 讲者认为，要从 **测试** 看瀑布的问题：测试是整个链条上第一个**物理事件**。
- 设计阶段只有报告/文档；程序真正写出来之前，它**不能运行**。
- 这是一个 **first principle**：当不能运行的东西存在时，人就会随便做做。
- 文档尤其如此：你叫我写文档，我就写一个。实验文档没有正确性 **guarantee**；最多是 **best effort guarantee**。文档先写放那，代码/设计改了，文档不一致，也就放在那。
- 这些上游产物没有很强的约束。

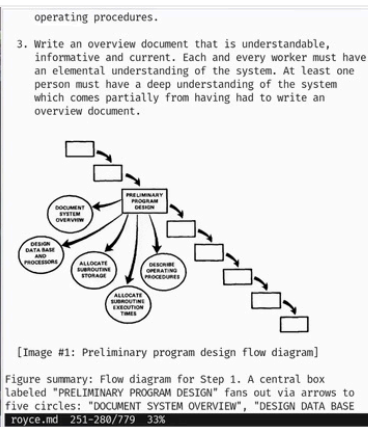
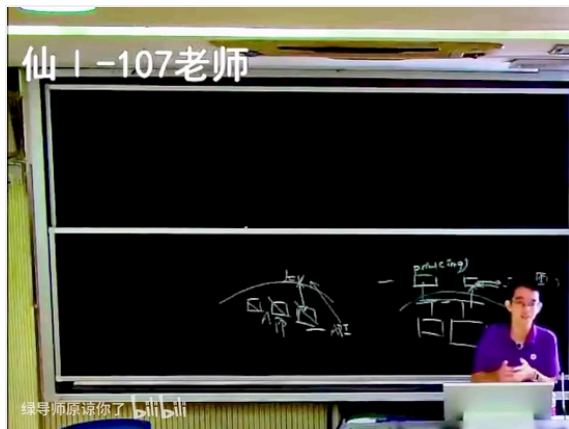
漂移只在测试暴露，回头改上游就是超期/超支的源头

- 只有测试时程序才真正跑起来，才真正知道它的行为是什么。
- 虽然编码时可能已经感觉有些不对，但一旦 **intent** → **specification** → **implementation** 发生漂移，往往到测试才发现。
- 到测试再发现漂移，就意味着要回头、回到前面修改；讲者记得有一张图表示这种回改。
- 回改会带来成本，所以它是**超期或者超支的源头**。
- 讲者给出的逻辑链可整理为：
 1. 瀑布把真正能运行、能验证的测试放到最后。
 2. 前面的文档/设计缺少强约束，容易敷衍。
 3. 上游发生漂移，但测试前不易暴露。
 4. 测试暴露漂移后，必须回改上游。
 5. 回改带来成本 → 超期/超支。
- 所以讲者认为，如果真读过原文，这些图很好理解；在今天软件工程实践背景下理解它非常舒服。

补救的本质：把发现问题的时间点提前

- 讲者的总结：既然风险最终都在测试的地方爆炸，那所有补救的本质，都是把发现问题的时间点提前。

第 18 部分



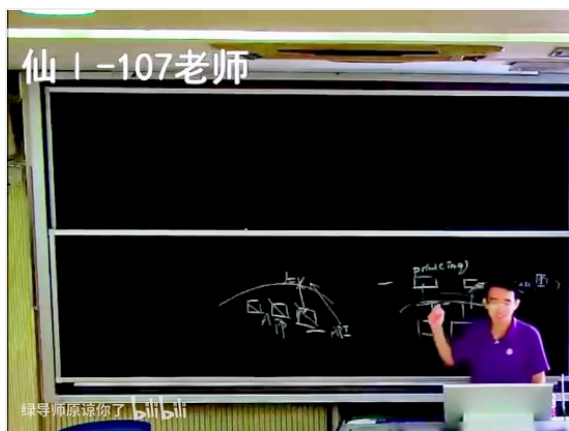
Morning Paper 风格的重新推导

问题的原点很朴素：一个程序小到能被建它的人自己用，两步就够了——分析、编码（Fig.1）。但“大系统”不行。于是人们自然会加阶段，得到 Fig.2 那条漂亮的单向阶梯。Royce 承认这条线是对的，然后立刻指出它 **risky, invites failure**。

为什么危险？因为测试是整条链上第一个“物理”事件：时序、存储、I/O 传输在这里才从“被分析”变成“被经历”。这些现象不像数学物理里可靠的微分方程——你分析不准。一旦不满足外部约束，不是改一行八进制补丁能救的，而是要动需求层次的重重大设计，于是**回到原点，100% 超期/超支**（Fig.4）。注意他反复强调：分析与编码其实“好管”（轨道力学做壳可能就几行串行算术），真正吃掉风险预算的是测试。

关键跳跃：既然风险最终在测试处爆炸，而爆炸的代价由回溯距离决定，那么所有补救的本质就是同一件事——**把“发现问题”的时间点提前，把“回溯距离”压短**。五条建议都是从这一条推出来的，而非凑数的最佳实践：

1. **设计先行：**让 program designer 在 requirements 之后、analysis 之前就分配存储/时序/数据流。批评者说这是“真空里设计会错”——Royce 承认，但“misses the point”：错在早期只是重设计，错在晚期就是 100% 超支。他还顺手给了一个组织学上的巧思：让设计师在分析阶段“抢”资源，逼分析员亲身感受约束的代价，而不是事后被告知。
2. **文档：**早期阶段 documentation = specification = design，三位一体，所以“文档没写出来就等于设计还不存在”。他给了个很工程化的换算：5M 美元硬件 ≈ 30 页规格；5M 美元软件 ≈ 1500



Royce 与瀑布模型

如何按时、不超预算地交付大型软件？

- Winston Royce: [Managing the development of large software systems](#)

- 这篇论文画出了后来被称为“瀑布模型”的经典阶段图
- 1970 年的背景：航天器任务规划、指令与飞行后分析，时间和存储都有硬约束。
- 原文在这张图之后就指出：这样的实施方式风险很高，容易失败。

“瀑布假设”

- 希望迭代只发生在相邻阶段之间
- 实际测试中暴露的问题，却会一路回溯到设计与需求

风险最终在测试爆炸 → 补救的本质是把发现问题提前

- 讲者的推理起点：既然风险最终都在测试的地方爆炸，那么所有补救的本质，都是把发现问题的时间点提前。
- 这是一个“非常自然”的想法，由此引出论文中的经典观点：设计先行、文档、做两遍（Doing it twice）。

Doing it twice：先做不完备但能体现功能的版本，就是 MVP

- 讲者把它总结为 **preliminary pilot program**：先做一个不完备、但能体现系统功能的设计。
- 这版东西可能会失败，但迭代成本会变得很低。
- 低成本迭代可以让它 reflect 到系统设计上。
- 讲者直接指出：这就是 **minimum viable product (MVP)**。
- 论文中还有各种各样的模型；讲者推测，论文后面/最后可能给出作者心目中的 **software development model** (ASR: suffer development model)。

经典软件工程智慧：瀑布模型被分析失败，但基本流程今天仍成立

- 论文作者先提出 **瀑布模型**，然后说明瀑布模型在软件工程中如何失败，并分析了一个具体失败。
- 讲者认为，这个流程在今天开发软件时也完全一样：从空项目开始 → 先想清楚要干什么 → 再大概想架构/怎么做 → 再去写代码。
- 不管用什么版本控制工具，不管写代码的是软件工程师还是 AI，这个流程依然成立。
- 讲者认为，即便在今天 **vibe coding** (ASR: web coding) 的时代，回到原文仍能提供很多软件工程智慧；这些思考原本是在航天器任务规划、把人送上月球等背景下写成的。

AI 时代可以直接读一手原文：教科书读不懂的，原文能看懂

- 现在每个人都可以阅读一手信息，直接回到历史背景；AI 既知道过去，又知道现在，而人还有历史原文。
- 讲者举例：书上讲瀑布模型只有两页，看起来不长，读了两遍还没搞懂；回去看原文，马上就懂了。

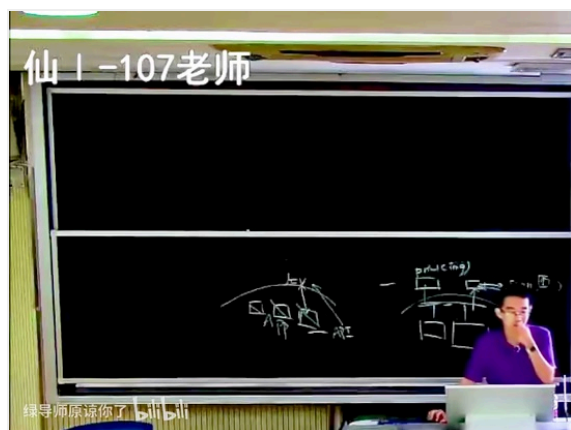
旧时代必须依赖教科书的有损压缩

- 讲者回忆自己上学或更早时，查证一个知识网络几乎不可能：要去图书馆翻很多资料，图书馆索引机制也不好；就算有索引，找到精确资料也要花无穷多时间。
- 因此，那时不可能通过一手资料学习，必须经过教科书做**有损压缩**。
- 有经验的人做得不同：好的教科书写得好，有的教科书则是“防自学”的。
- 讲者认为，好的教科书作者读过一手信息，也可能能共情；好的老师能共情学生，知道学生想要什么、什么地方会不明白，所以能把一手信息压缩，同时让读者的损失很小。

讲者的判断：今天老师不必再做统一压缩

- 讲者认为，今天这些东西也都不要了：因为可以直接读一手信息。
- 作为老师，讲者说“我不需要再做压缩”，因为压缩对每个人都是有损的，而且对不同的人损失的是不同部分。

第 19 部分



AI 时代的一手信息

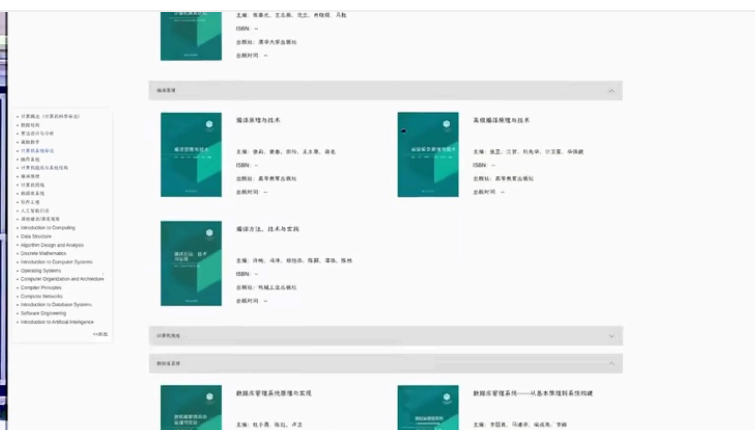
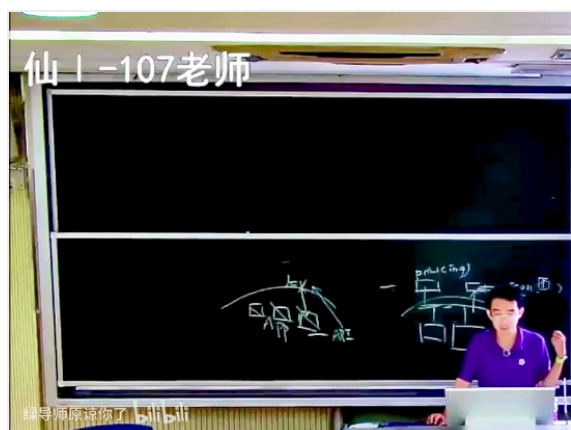
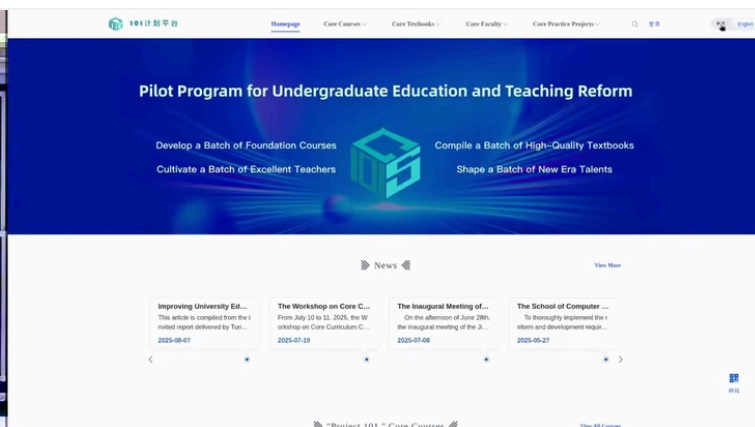
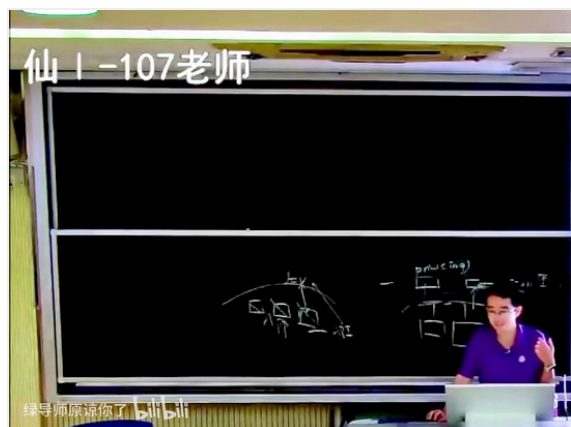
我的 AI 阅读助手: [royce.md](#)

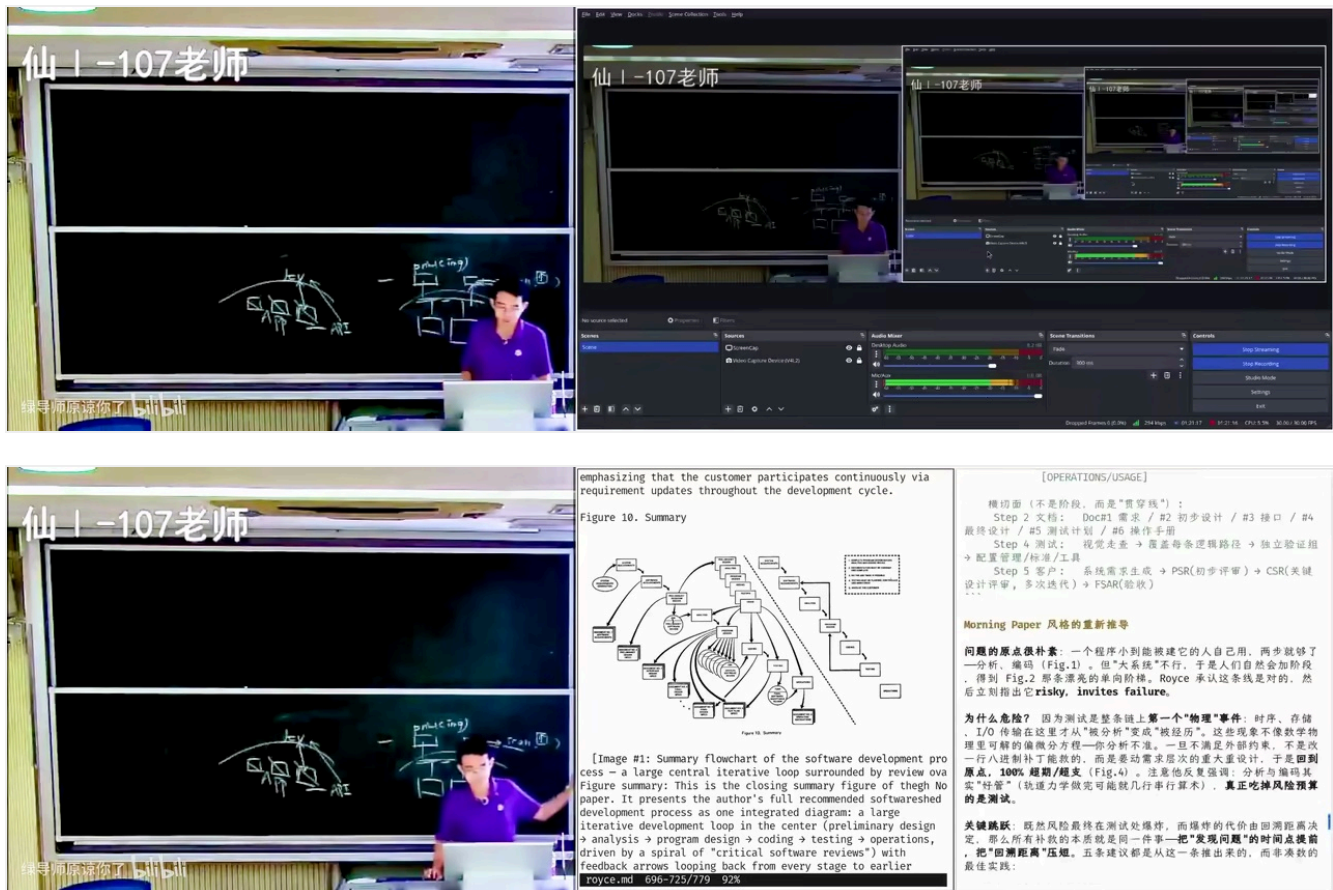
- 每个人都可以阅读一手信息，直接代入历史背景
- 理解产生的原因，复原学科逻辑

谁来决定知识怎样压缩？

- 过去：查证知识网络非常费时，需要教科书“有损压缩”
- 现在：自己建立无损知识网络，按需压缩
- 你说 101 计划是不是
- 例如：只留下那张瀑布图，就丢掉了作者接下来要解决的返工问题。

1 / 3





统一压缩必然有损，所以讲者不再替所有人做压缩

- 讲者判断：自己做压缩，对每个人都有损；更关键的是，不同人损失的部分不同。
- 若一对一教学，才可能做出“最适合你”的压缩。
- 因此他不再把一套统一压缩直接交给学生，也不需要学生一上来直接对着“源/原文”（原文“圆”按语境校为“源/原文”）。
- 逻辑链：统一压缩 → 对所有人有损 → 不同人损失不同 → 需定制压缩 → 与其统一压缩，不如给学生可自行压缩的材料。

把知识拆成“发现点 + 原文”，让学生自己做属于他的压缩

- 讲者的知识理解方式：先把知识脉络拆开，拆成很多发现的点。
- 学生拿到每个点及其原文，然后做属于自己的压缩。
- 这里压缩不是老师替学生完成，而是学生用自己的需求和背景去压缩。
- 这和前面相连：老师给的是可追溯的原文与节点；最终压缩由学生生成。

专家建设课程仍有价值，但AI时代可能把投入直接砸掉

- 讲者提到：这个计划里有很多导师/老师建设“比较先进的课程”。

- 他认为这是好计划：因为**查证知识网络非常费时**；如果有真正专家帮你做好的压缩，毫无疑问有价值。
- 但在 AI 时代，讲者认为 AI 是**摧毁性**的，对整个领域都是如此。
- 可能投入百万、千万甚至上亿人民币，最后被一个模型“把轮子碾掉”。他举的可能模型是 DeepSeek（原文“deep pick”）/Flash 类。
- 刚才这些模型就在帮他：有任何问题可以直接问。

用AI追“瀑布模型”的误读链：从疑问到原文，再回到手上的教材

- 讲者举例：可以直接问 AI：解释**瀑布模型**在软件工程里到底是为什么、怎么被误读的。
- 流程：你产生疑问 → 直接与 AI 对话 → AI 在过程中找到更多原文 → 甚至找到整个**误读链**。
- 这条链可以一路追：教材一级一级……最后到你手上的软件工程教材是怎样一步一步曲解它的。
- 讲者觉得 AI 这个解释“还挺 make sense”。他也提醒：你可能选择不相信 AI，可以继续追问，或者用更好的模型、让它联网。
- 讲者补充：这篇文章已经带大家读过，所以那几页 AI 生成的内容就不那么重要了。

瀑布模型是“史上最好用的反派”，因为它本身就是错的

- 讲者提到教学上的观察：瀑布模型**好画、好背、好出题**（原文“好话”疑为“好画”），可以作为第一个模型讲。
- 而且每一个新方法都要拿它当反派：**瀑布是史上最好用的反派**。
- 讲者认为这话很对：原因不是新方法需要反派，而是瀑布模型本身就有问题。
- 1970 年的论文已经告诉你**不能瀑布**；1970 年就告诉你要做 **MVP**。
- 讲者还评价它确实不成熟。

“do it twice”与今天的MVP同构

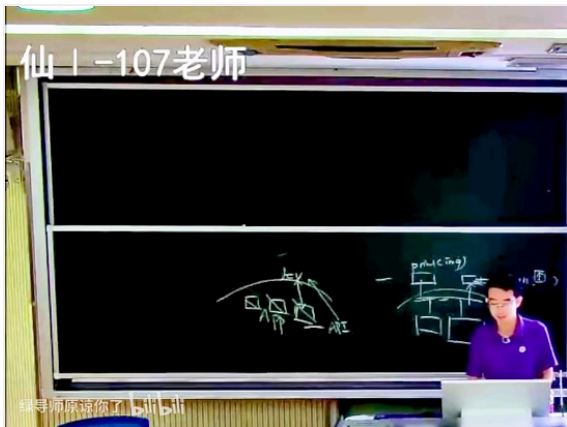
- 文章里有一个很重要的工程理解：**do it twice**。
- 今天的工作就是做 **MVP**。
- 讲者把二者接上：1970 年论文的“do it twice”与今天的 MVP 不是两套东西，而是同一工程理解在不同时代的表达。
- （整理者串联）这解释了他为什么说瀑布误读必须追到原文：追到原文才会看到“不能瀑布/do it twice”本来就在那里。

今天的正确实践可以映射回过去的 first principle

- 讲者说，也可以在各种现代软件开发背景下理解这套工作。
- 例如 TDD (test driven development, 测试驱动开发)：问“如何用这个工作来解释 TDD”。
- 讲者的判断：它们是互相的。
- 今天的正确 practice 可以映射到过去；过去正确的 first principle (讲者话未说完；** (整理者串联) **补为：也能映射到今天)。
- (整理者串联) 因此他前面追瀑布误读、读 1970 年原文、讲 MVP/TDD，是在建立“过去原则 ↔ 今天实践”的双向映射。

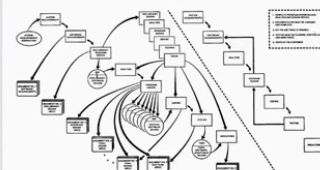
小结：压缩不外包，交给学生自己压缩；AI 用来追原文和误读链；现代实践要照回 first principle。

第 20 部分



emphasizing that the customer participates continuously via requirement updates throughout the development cycle.

Figure 10. Summary



[Image #1: Summary flowchart of the software development process - a large central iterative loop surrounded by review ova Figure summary: This is the closing summary figure of the paper. It presents the author's full recommended software development process as one integrated diagram: a large iterative development loop in the center (preliminary design → analysis → program design → coding → testing → operations, driven by a spiral of "critical software reviews") with feedback arrows looping back from every stage to earlier

royce.md 696-725/779 92%

不死的原因：没人真用它，但人人都需要它。
- 它并不纯属虚构：Fig.3 那句“迭代只限相邻步骤”确实给误读留了合法入口。所以不是有人凭空捏造，而是选择性阅读——只引 Fig.3、不引 Fig.4 和 Fig.10。

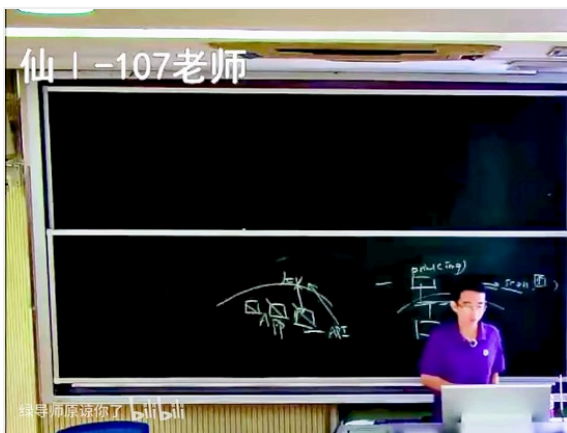
四、顺带一提
I
Larman & Basili 的 *Iterative and Incremental Development: A Brief History* (IEEE Computer, 2003) 系统地做了这个考据。并指出迭代式实践早在 1950 年代就存在；Royce 严格说属于迭代派的早期声音之一 (“do it twice” 就是原型/试点)，而不是迭代的敌人。有点黑色幽默的是，他的儿子 Walker Royce 后来在 Rational 做 RUP (迭代式过程)，成了替父亲“翻案”的一方。

真正的教训：Royce 的中心思想不是“阶段要按顺序排”，而是“风险最终在测试那里爆炸，而爆炸的代价由回滚距离决定；所以过程的设计目标就是尽早发现问题，尽量缩短回滚”。这条判断今天依然成立——敏捷只是换了缩短回路的手段 (短迭代、持续集成、可运行软件)，并没有推翻这条判据。把他的话简化为“瀑布环、敏捷好”，是同一场误读的当代续集。

Recap: 用 grep 证明原文正文根本没有 “waterfall” 一词。再拆出误读的链条 (抽图不读文 → Boehm 的二层引用被截断 → 批评瀑布的论文标签图 → 国防标准只理采文档里提，丢掉迭代螺旋)，并解释了它作为“最好用的稻草人”为何长盛不衰。

那如果我们用 TDD 去理解这个工作，

~/Projects/readspace
11k 48.6k R155k C099.0% \$0.015 2.0%/1.0M (auto) deepseek-fla



Do It Twice

用今天的 MVP 理解 Royce 的 Pilot Model

- Minimum Viable Product: 先做一遍，从中学习，再完善产品
- Royce 的 **do it twice**: 先用 pilot model 经历关键问题
- 《Pilot 重点验证技术与运行风险；MVP 更关注用最小投入验证用户与业务假设。

用敏捷开发理解“反复做 Pilot”

- 不必把 pilot 单独隔成一个大阶段，而是把试做、反馈、修正反复快速做
- 《共同点：让关键假设尽早遇到证据；敏捷原则将频繁交付、合作与调整变成持续过程。

今天的 practice 与过去的 first principle 可以互相映射

- 讲者主张：今天正确的 **practice** 可以映射到过去；过去正确的 **first principle** 也可以映射到今天。二者是互相解释的，因此可以用“这个工作”来解释 **TDD**。
- 讲者提出看一个 **70B**（700亿参数，转录作“700币”）小模型。
- **逻辑链**：缺陷越晚发现（A）→ 返工代价越大（B）（讲者引 Royce，转录作“rose”陈述）；（**整理者串联**）B → 应把发现提前到设计和用例阶段（C）→ **TDD** 先定测试用例、敏捷先做原型（D）。

从“缺陷越晚发现越贵”到 TDD：把验证提前

- 讲者引 Royce（转录作“rose”）：缺陷被发现的时间越晚，带来的 **返工代价** 越大。讲者认为这件事显然成立。
- 例子：做 **OJ** 时脑子一热，先想一个算法，直接写代码；写完发现样例不对，对着样例跑，才发现整个算法错了。讲者称这是典型的 **瀑布模型失败**。
- 在 **TDD** 的意义上，设计先行时干脆先把 **测试用例** 定好：
 - 输入这个，它就应该经历这个流程；
 - 输出这个结果；
 - 然后马上对照验证。
- 讲者认为，敏捷开发等概念也可以接上：用快速原型让不确定的关键假设尽早被验证。

老经验不是被完全颠覆：Royce 的 first principle 仍在今天生效

- 讲者说自己很喜欢很多 **图灵奖** 工作：它们在大家完全不知道的情况下，确立了一个 **first principle**，告诉大家“这件事情应该这么做”。
- 软件工程基础之一是 Royce 的经典文章 **Managing the Development of Large Software Systems**；后来常被联系到“瀑布模型”。讲者认为它给人很多启发，并在它上面长出了世界的很多分支。
- 但方向也可能一开始就错：如果某个 **first principle** 把全人类带歪，而从某一点换方向，全人类也许就能立即走上正确道路。讲者举例：**直流电 / 交流电**，**爱迪生 / 特斯拉**。
- 因而讲者强调：现在做的事情不完全颠覆过去的经验；这个 **first principle** 在多年软件工程发展的实践中仍然存在。

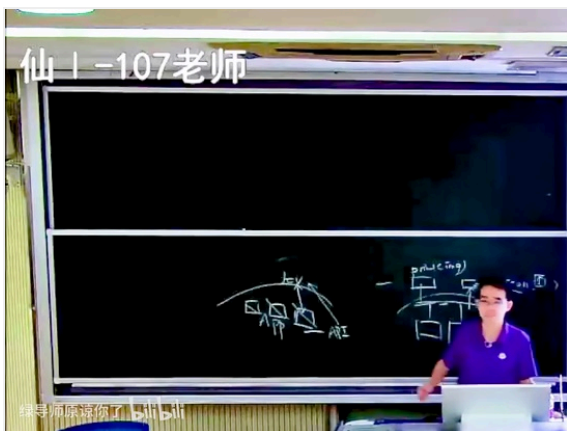
用敏捷理解“do it twice”：不是两次，而是无限次演进

- 讲者说这是 do it twice（转录作“three twice”），并可用 敏捷开发 理解：
 - 不需要把 pilot 理解成一个完整的流水线。
 - 敏捷的第一个版本不是 doing it twice，而是 doing it **infinite many times**。
 - 第一个版本哪怕只有 **hello world**，只是打印一个流，或者每个模块都自报家门再把模块串起来，也是好的 **product**。
 - 在这个意义上不断把软件向后演进。
- 核心目的：**让关键假设尽早遇到证据**。一开始不确定的东西，因为有了敏捷原型，就可以立即验证它。

AI 来了的焦虑：AI 也许学会品味，人类仍想理解它做了什么

- 讲者说 AI 来了很焦虑：AI 确实学会了 AI；也许有一天它也学会了 品味，因为它可以从这些论文里面学到这种品味。
- 焦虑点在于：AI 能帮我们做事。
- 但讲者认为人类还有一种 傲慢：仍然希望理解 AI 到底做了什么。

第 21 部分



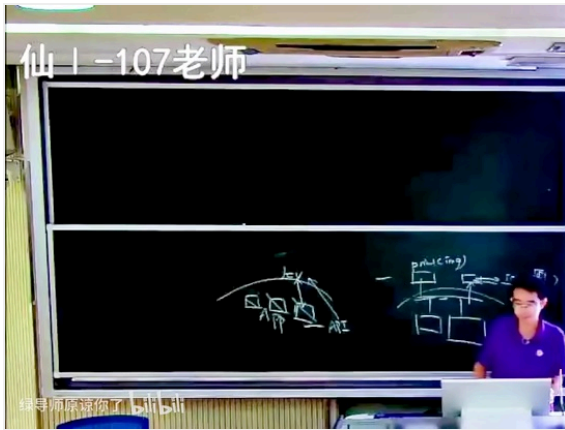
Do It Twice

用今天的 MVP 理解 Royce 的 Pilot Model

- Minimum Viable Product: 先做一遍，从中学习，再完善产品
- Royce 的 **do it twice**: 先用 pilot model 经历关键问题
- <> Pilot 重点验证技术与运行风险；MVP 更关注用最小投入验证用户与业务假设。

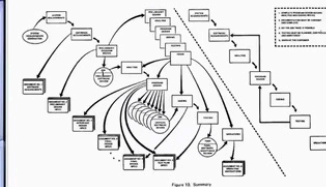
用敏捷开发理解“反复做 Pilot”

- 不必把 pilot 单独隔成一个大阶段，而是把试做、反馈、修正反复快速做
- <> 共同点：让关键假设尽早遇到证据；敏捷原则将频繁交付、合作与调整变成持续过程。



emphasizing that the customer participates continuously via requirement updates throughout the development cycle.

Figure 10. Summary



[Image #1: Summary flowchart of the software development process - a large central iterative loop surrounded by review ovals. Figure summary: This is the closing summary figure of the paper. It presents the author's full recommended software development process as one integrated diagram: a large iterative development loop in the center (preliminary design → analysis → program design → coding → testing → operations, driven by a spiral of "critical software reviews") with feedback arrows looping back from every stage to earlier stages.

试	独立验证 = CI + code review, 但见下一节, 这里出现了实质分歧
Step 5 客户参与 (PSR/CSR/FSAR)	每个迭代的可行运行软件 + 验收测试, 即 ATDD/BDD, 同样的意思: 把承诺频率提高、粒度变细

最有说服力的一对是经济账。Royce 的 Fig.4 就是一条随发现时间指数上升的变更成本曲线, 他用“100% 超赔”给它标价。XP 阵营的全部主张——在 XP 下变更成本曲线是平的——正是对这条曲线的正面攻击。所以 TDD 和 Royce 的关系不是“新旧对”, 而是对同一条曲线的两种对策。Royce 用文档、评审, 原型把曲线的拐点往左推; TDD 用自动化测试把整条曲线压平。他画出了问题, TDD 声明解除了它。

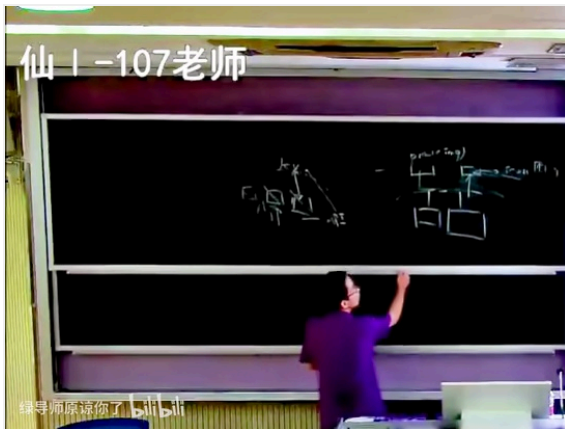
三、TDD 真正修正了 Royce 的地方

只有一处, 但是关键的一处。Royce 明确写过:

"Do not use the computer to detect this kind of thing - too expensive." (类负号, 跳得地址过类, 靠人眼走查最划算)

TDD 的全部赌注就是这条成本假设失效了: 自动化执行的边际成本趋零。于是 Royce 分给“人眼”和“独立测试组”的活, 被移到机器上, 并且从“事后抽查”变成“事前强制”。他 Step 4 的第 2、3 条 (视觉检查、逻辑路径检查) 在 TDD 里被合并成同一条流水线。

还有一个 Royce 没有的巧思, 值得单独指出: Red 阶段是对“传感器本身”的标定。Royce 说“每条逻辑路径用数值验证一次”, 这是覆盖率诉求; 而“先写, 先看它失败”多了一层意义——一个从来没失败过



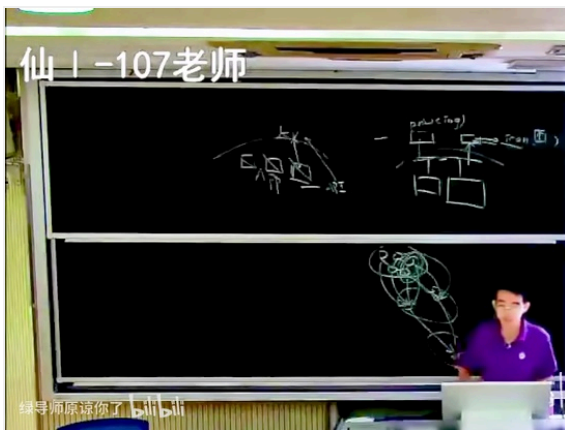
瀑布模型就是 Data Dependency Graph

需求分析 → 系统设计 → 系统实现 → 测试验证 → 交付维护

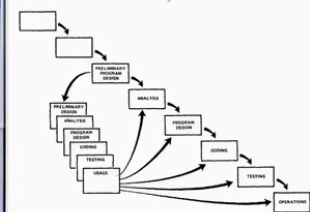
- 不做需求 (intent / spec), 设计就会漂移
- 设计 (spec) 错了, 实现就会漂移
- 没有实现 (impl), 谁会想去测试?
- 当然, 交付之后还得有人擦屁股

TDD: 拿 Testcase 当需求与设计

- 后来大家才反应过来: 可以先写 testcase
- 写下预期行为只需要需求: 验证实现是否满足预期, 才需要实现。
- 产物之间有依赖, 不等于所有阶段只能完整地做一遍。



simulation the project manager is at the mercy of human judgment. With the simulation he can at least perform experimental tests of some key hypotheses and scope down what remains for human judgment, which in the area of computer program design (as in the estimation of takeoff gross weight, costs to complete, or the daily double) is invariably and seriously optimistic.



[Image #1: Flowchart showing a preliminary pilot program design proceeding through analysis, program design, coding, testing, and operations, with feedback loops from every stage to earlier stages. Figure 7. Step 3: Attempt to do the job twice - the first through result provides an early simulation of the final product, with feedback loops from usage back to earlier stages] Figure summary: The diagram illustrates "doing the job

3. 它会让你以为“做两遍”是冗余重复。不是一 pilot model 的价值不是把同样的计算做两次, 而是改变图形的形状 (消除到底哪个约束会 bind 的不确定性)。它是采样, 不是重复。

四、用你的框架重讲 TDD, 一句话

你既然不能删掉那条长程回边, 剩下的唯一杆杠就是降低它的延迟。于是做预测执行: 提前预测验证结果, 用一次编辑就能回滚。

- 瀑布 = 分支在流水线末端才发现错误预测, 代价是 flush 整个项目
- TDD = 把验证预测到写实现的同一分钟执行, 误预测的代价是一次 diff。

这个类比比“文档即规格”更精确地解释了 TDD 为什么有效: 它没有减少依赖, 只压缩了依赖的传播延迟。而它也和 Royce 的原始判断完全同构——既然回滚代价正比于距离, 那就把距离缩短到零。

Recap: 把 Royce 读成依赖图 + 修改代价是第一性原理的正确读法 (Fig.3 是依赖局部性假设, Fig.4 是长程依赖, 五步建议都是图变换), 但要补三个补丁: 图是被设计出来的 (decision dependency / DSM), 存在图内推不出的外部 oracle (validation ≠ verification), 代价系数由接口与文档质量决定而非拓扑; 由此 TDD 可重述为“不断长程依赖, 只把它改造成延迟为一次 diff 的预测执行”。

~/Projects/readspace
113k 417k R198k CH94.8% \$0.025 2.8%/1.0M (auto) deepseek-flan

人类现在还不放弃理解 AI, 因为 AI 尚未全面超越人类

- 讲者先描述一种焦虑: AI 能帮我们做事。
- 但人类/“现在的人类”还有一种傲慢: 仍希望理解 AI 到底做了什么, 并能判断 AI 做得对不对。
- 讲者设想极端情形: 真到那一天, AI 做的所有东西都远超人类且都是对的; 只有人会犯错误, AI 不犯错误。那时人类就不需要理解它了, 甚至可以放弃理解它。

- 但至少在今天：AI 虽然泛化能力（转录作“财政能力”）在增长，人还至少能驾驭；讲者认为能理解 AI 的人能活下来。
- 在“斩杀线”（讲者用词）上升的基础上，问题变成：怎样用好 AI、怎样理解其中的软件工程智慧。

讲者用 data dependency 理解 ROSE/Royce 瀑布模型：修改有代价

- 讲者自我定位：我是一个 computer science 的人。
- 他对 ROSE/Royce（转录作 rose/ROIS）的模型的理解是：它是一种 data dependency；修改是有代价的。
- 他明确说这是自己的理解，但认为还是自洽的；可以拿这个理解去和别人聊。

把每个瀑布阶段想成程序，产物之间形成 data dependency graph

- 瀑布模型链条：需求分析 → 系统设计 → 系统实现 → 测试验证 → 交付维护。
- 讲者的类比：每一个过程都可以想象成一个 program；人也可以当成程序，人脑像神经网络。
- 人可以执行 chain of thought / CoT（转录作 chain of salt/CHAS）。讲者认为 CoT 是很有意思的机制。
- 他举例：先学会语言、学会说话；自己在想问题时，即使不说话，也会把思考对齐到语言上，例如在心里说“我要这样/那样，会有什么后果”。他用语言对齐脑内的思维链。
- 因此，如果这些过程产生程序和 artifact，它就是一个 data dependency graph，而且是有向无环图（DAG）。

依赖关系会传递：测试错追到设计错，所有下游全毁，所以成本高

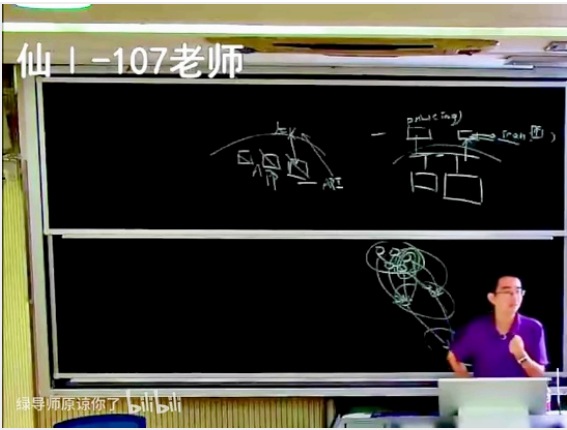
- 讲者进一步展开：做需求时有多个点，有些相互关联，有些没有关联。
- 做设计时，设计会关联到需求点；不同设计之间也关联；关联关系会传递。
- 这样就得到一张 data flow graph。
- 若一趟走下来发现 test 错了，再一路往上追溯，发现几个 design 错了，那就惨了：所有依赖它们的东西全部毁掉。
- 讲者说，这就解释了为什么 cost 很高；这也和 ROSE/Royce 的解释一致。

每个人对模型的理解可以不同，因此需要个性化教学

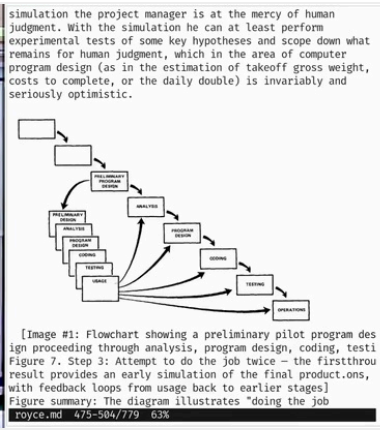
- 讲者相信，每个人对同一个模型的理解都可以不一样。
- 每个人有自己的 **diversity**，成长经历不同。
- 因此非常需要个性化教学。
- 他补充：自己算是做过一点 PL (programming languages)。

第 22 部分

仙 | -107老师



simulation the project manager is at the mercy of human judgment. With the simulation he can at least perform experimental tests of some key hypotheses and scope down what remains for human judgment, which in the area of computer program design (as in the estimation of takeoff gross weight, costs to complete, or the daily double) is invariably and seriously optimistic.



[Image #1: Flowchart showing a preliminary pilot program design proceeding through analysis, program design, coding, test, Figure 7. Step 3: Attempt to do the job twice – the first throw result provides an early simulation of the final product, with feedback loops from usage back to earlier stages]

Figure summary: The diagram illustrates "doing the job royce.md 475-504/779 63%

边，架构与系统性风险仍需提前设计。

我把royce的模型，理解成一种data dependency，然后修改是有代价的，这样可以吗？

Thinking...

可以，而且这不是类比上的小聪明——这基本就是软件工程后来真正采用的形式化方式。但有两个地方这个框架会逼你，需要打补丁。

一、它在哪（而且可以推倒很强）

如果按编译器的数据依赖读：

Fig.2 = 直线型数据流，每个阶段消费上一个的产物。调度长度 = 关键路径 = 工期。

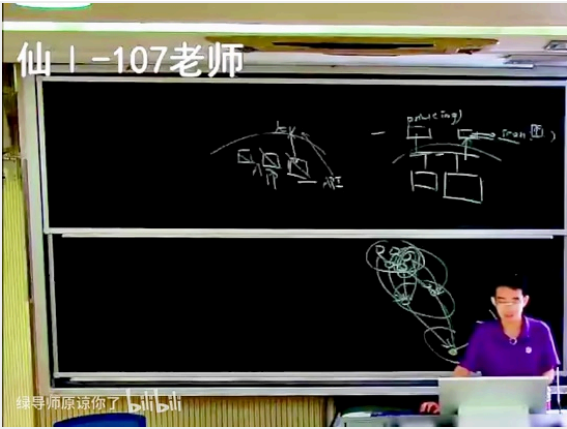
Royce 说的“迭代只限相邻步骤”= 依赖的局部性假设（就像 loop nest 里依赖在 tile 内部）。

Fig.4 = 长程依赖（long-range dependence）：测试通过一条不受约束的通道依赖需求。他在 Fig.3 里说依赖是局部的，Fig.4 告诉他不是——而长程依赖没法靠“把迭代限制在相邻步骤”tile 掉。

返工代价 = 依赖传播锥（cone of influence）的大小。100% 超预期 = 一条长回边触发的金流水线 flush。

再进一步，把“修改有代价”当成第一性原理。Royce 的整个论证就变成一个期望代价最小化问题，而且他是刻意这么权衡的：Step 1 里那句“批评是对的，but it misses the point”——批评者优化的是出错的概率（设计在真空中就会做错），他优化的是出错的代价（那

仙 | -107老师



瀑布模型就是 Data Dependency Graph

需求分析 → 系统设计 → 系统实现 → 测试验证 → 交付维护

- 不做需求 (intent / spec)，设计就会漂移
- 设计 (spec) 错了，实现就会漂移
- 没有实现 (impl)，谁会想去测试？
- 当然，交付之后还得有人擦屁股

TDD：拿 Testcase 当需求与设计

- 后来大家才反应过来：可以先写 testcase
- <@> 写下预期行为只需要需求：验证实现是否满足预期，才需要实现。
- <@> 产物之间有依赖，不等于所有阶段只能完整地做一遍。

个性化教学的前提：每个人有 diversity，成长经历不同

- 讲者开场强调：非常需要个性化教学，因为每个人都有自己的 **diversity**，成长经历不同。
- 他自述做过一点 PL、操作系统，因此会自然地用 **computer science** 里习得的概念解释世界。

- **（整理者串联）** 这解释了他后面为什么不断用 CS 概念类比软件工程：不是只给结论，而是用****第一性原理（first principle）****往前推。

用 CS 的第一性原理看软件工程：data dependency 决定变更方式

- 讲者认为，软件工程后来真正采用的**形式化方法（formal methods）**，本身也是一个 **first principle**；它能解释模型。
- 以 **data dependency** 为例：一旦某个改动触及上游、需要改上游，就不能只改局部，因为依赖关系会牵连。
- 讲者给出的自然推论：
 1. 知道 **data dependency**；
 2. 就知道改动可能波及上游/下游；
 3. 所以不要把大量依赖一次性产生出来；
 4. 应该把它割小；
 5. **MVP** 就来了。

MVP → BFS/DFS → 敏捷开发：先走通高优先级依赖，再迭代扩展

- 讲者说不要一次性产生大量 **dependency**，而要用****广度优先搜索（BFS，原音 breakfast search）****的方式构建软件：先做第一层，再做第二层，再做第三层。
- 随后他口头说变成 **DFS**（原音 def search），并描述为不断向右扩展、一个迭代一个迭代。**（整理者串联）** 这里 BFS/DFS 的严格区别不是重点；重点是“先分层走通，再持续沿路线扩展”。
- 对应到开发方法：先做一个 **minimal viable product（MVP）**，把重要、优先级最高的 **dependency** 流程走通；走通后再不断向右扩展。
- 讲者直接问：“这不就是敏捷开发吗？”并把 MVP、敏捷开发、TDD 视为显然正确的事情。
- 他在这里说：“软件工程没有什么真正真正困难的。”

越早反馈，越能压缩瀑布模型阶段间的 gap

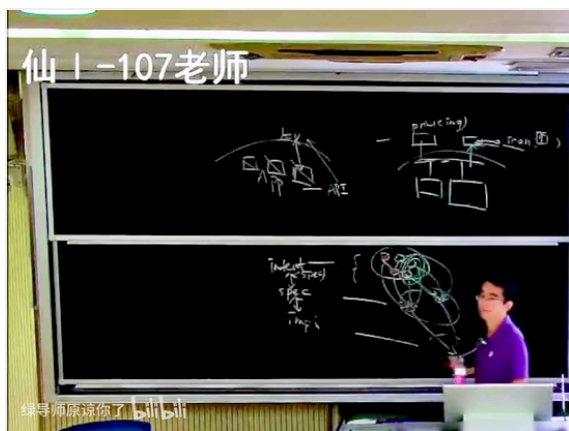
- 讲者强调：越早得到反馈，就越能处理路线上的**不确定性 / gap**。
- 他用****瀑布模型（原音近“pom 模型”）****说明阶段间有 **insight**：
 - 需求阶段：确定 **intent**；
 - 设计阶段：基本确定 **spec**（有些 spec 可能在需求阶段也稍微做一点）；
 - 编码测试阶段：才是 **implementation**。

- 因为知道有 **gap**，所以每个 **intent** 都会拆成好多小点，并对后面产生 **dependency**。
- 因此他的结论链是：瀑布模型阶段固定 → 阶段之间有 gap → intent 要拆小并产生 dependency → 越早反馈越能减少不确定性 → 所以做 MVP / 敏捷开发 / TDD 是正确的。

经典著作值得读：它们记录领域最初、最重要的根本问题与出路

- 讲者认为读经典著作非常有启发性，也是值得学习的学习方式。
- 理由：经典通常记录一个领域最初遇到的、也是最重要的根本性问题，并在方向上找到一条出路。
- 即使事后可以援引其他概念，可能比它解决得更好；讲者举例自己很喜欢 **data dependency**，认为在整个软件工程发展史上都可以往这个方向思考，并导出更多概念。
- 他把这类 **first principle** 看作学软件工程的“起点”：软件到底是什么？当然可以定义说是**代码和数据**；但从这个视角看，就能继续往后推出更多东西。
- 讲者最后提到，软件工程已经经历了几十年的发展。

第 23 部分



软件工程的方法和工具

为每个阶段创建方法学和工具

- 软件工程：为每个阶段都创建**方法学**和**工具**，尽可能减少返工概率。

不同阶段，不同的“语言”

- 在没有 LLM 这种“万能编译器”的时代，不同阶段需要不同的“语言”：
 - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
 - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
 - 代码实现
- Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

软件工程：从“代码和数据”出发，为各阶段造方法/工具以降低返工

- 讲者先承认：按定义，软件当然可以说是**代码和数据**。
- 但他强调，从这个视角继续往后推，能看到更多东西：经历几十年发展，**软件工程**为解决开发各阶段的问题，为**每个阶段**创建了方法和工具。
- 讲者明说的逻辑链是：为每个阶段创建方法和工具 → 减少返工概率。

- （整理者串联）因此，软件工程的价值不只在于写代码，还在于用流程、文档、检查表等工具系统性地压低工程风险。

Checklist 不是 nonsense：它降低“不符合客户要求”的概率

- 讲者举例：软件工程课的设计阶段会有一套（他印象中是 ISO 标准的）文档和很长的 **checklist**，问“这个有没有做到、那个有没有做到、有没有沟通过”。
- 做 OJ 题的学生会觉得 **checklist** 很烦、很 nonsense。
- 但讲者认为 **checklist** 实际有用：它减少返工概率。比如 **checklist** 最后常有“你是不是和客户沟通过”。
- 在学校里觉得讨厌，但真和客户沟通并打勾后，会大幅减少**不符合客户要求**的概率。
- 所以 **checklist** 是一种方法、一种工具；它没有任何代码，讲者称其为一种**有人文关怀的工具**。

有代码之后：自动测试、bad smell 检测，以及 AI 带来的反例

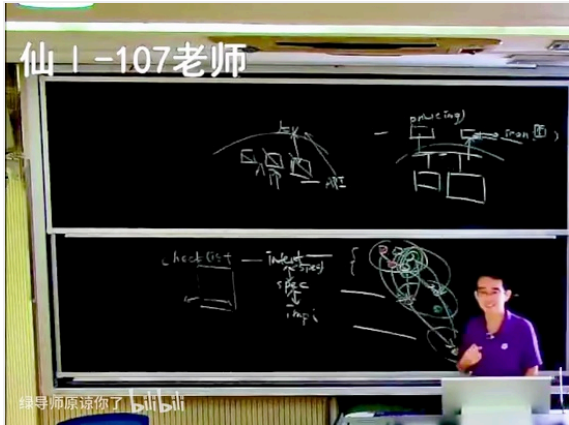
- 现在做软件工程研究，更多是“有了代码以后”：能不能做自动测试；讲者认为也产生了无穷多灌水 paper。
- 有了代码以后，可以检测里面的 **bad smell**。
- 例子：如果代码在等号左右从不写空格，会被认为是不好的代码。这个规则在 AI 时代之前成立。
- 讲者观察到，GPT 学会省 token 以后，这件事变了：它会写出极为无法理解的代码，把很多语句放到一行里，但代码是对的。讲者不知道它是怎么弄对的，但承认它是对的。
- 讲者提出：也许我们永远不应该去读那样的代码。他认为这是个有意思的观察。

软工是大森林：研究常落在一个小点，但别忘看经典与全局

- 软件工程为了把大的工程系统性地扶起来，里面还有很多细小的点。
- 如果读研选软件工程方向，大概率会落在其中一个非常小的点上。例如：针对软件开发流程中编码过程里，**Java 代码某类 bad smell 的更好检测**。
- 讲者提醒：做深的过程中，不要忘记去看森林、看经典的东西，这样会有更好的认知。
- 对人来说，深度和广度都重要：在深度里训练，才有更长的思维链；在广度里训练，才有更好的对整个世界的感觉。

小结：软工用阶段化方法与工具降低返工；研究可以落到小点，但要用经典与广度保持全局感。

第 24 部分



软件工程的方法和工具

为每个阶段创建方法学和工具

- 软件工程：为每个阶段都创建**方法学**和**工具**，尽可能减少返工概率。

不同阶段，不同的“语言”

- 在没有 LLM 这种“万能编译器”的时代，不同阶段需要不同的“语言”：
 - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
 - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
 - 代码实现
 - Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

深度与广度训练分别给什么

- 讲者认为，人的训练有深度和广度两面：在深度里训练，才有更长的**思维链**；在**广度**里训练，才有对整个世界更好的感觉。
- 接着他把话题引到软件工程：软件工程要为每个阶段创建方法学和工具，于是不同阶段需要不同语言。

软件工程的分阶段方法学，必然要求分阶段语言

- 核心主张：软件工程为了在每个阶段创建方法学和工具，就需要在不同阶段使用不同语言。
- 逻辑链：
 1. 软件工程为每个阶段创建方法学和工具。
 2. **（整理者串联）**不同阶段的任务和需要的确定性不同，因此可用的表达形式也不同。
 3. 讲者结论：不同阶段要用不同语言。
- 例证：

- **checklist 阶段**：在大语言模型时代之前，不太可能用一个编程语言去做 checklist。今天大家用 **Markdown**，但那时就是 **docx** 文档；打印出来、存档；跟客户沟通的记录可能还是手写的。
- **编程阶段**：有编程语言。
- 于是从编程语言往前的阶段，马上能看到一个 **gap**。
- 讲者对照：那时没有大语言模型这种“万能编译器”；[现在]只要有自然语言，就可以把它编译成任何一种语言。
- 有些阶段，比如**设计阶段**，没有一种语言来帮助做设计。1970 年设计时，大家用的都是自然语言，也就是一个文档——**设计文档**。
- 讲者回忆自己学软件工程时：先写**需求文档**，再写**设计文档**，follow **瀑布模型**。

文档的根本缺陷：写文档的人会产生幻觉，彼此无法对齐

- 讲者说，看到设计文档就会马上想到文档的最大缺点：
 - 写文档的是人，人会产生幻觉，会产生 **typo**；
 - 人和人讲同一件事时，**无法对齐**。
- 例子：你和室友之间已经发生过无数次——你说的是这个，室友理解成那个，你花很多时间沟通和对齐。讲者还开玩笑：跟女朋友对齐比跟室友对齐更困难；他说这点不重要。
- 讲者由此按 **first principles** 提出：如果要在设计阶段创建一个方法/工具，去**减少返工**的概率，那要做什么？
- **（整理者串联）**这里暗含的因果是：文档对齐差会带来返工。
- 讲者的答案：要做一种介于**自然语言**和**编程语言**之间的**设计语言**；当然也是一种 **domain specific language (DSL)**。

Checklist 说明：中间语言靠格式和语义提高形式化程度

- 讲者问：这让你想到什么？软件工程课上的各种东西。
- **Checklist** 就是例子：它虽然属于自然语言，但实际上也是一种**中间语言**，因为它有格式规范：
 - 必须一条一条；
 - 每一条必须打勾或打叉。

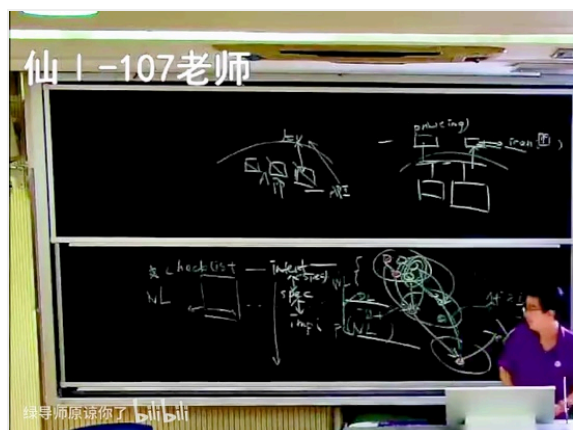
- 它已经介于自然和形式化之间，有语义；这个语义虽然由人来检查，但人对勾/叉的对齐，比直接写一个文档的对齐更 **formal**。
- 逻辑链：
 1. 语言越 formal，其**确定性语义**的程度越高。
 2. 从设计阶段往下面走，形式化程度越来越高。
 3. 这迎合了从 **intent** → **spec** → **implementation** 之间 gap 的解决。
- 讲者结论：这样就开始理解软件工程这个学科了。

学习路径是 bottom-up：先学编程，先学做一个码农

- 讲者回忆他们当时怎么学：是 ****bottom-up (ASR: button map) ****的。
- 先学编程；看计算机/软件工程，第一件事是先学怎么做一个**码农**。

小结：软件工程可看成从自然语言到形式语言的连续谱；每个阶段需要不同语言，设计阶段缺的语言由 DSL/中间语言（如 checklist）补。

第 25 部分



软件工程的方法和工具

为每个阶段创建方法学和工具

- 软件工程：为每个阶段都创建**方法学**和**工具**，尽可能减少返工概率。

不同阶段，不同的“语言”

- 在没有 LLM 这种“万能编译器”的时代，不同阶段需要不同的“语言”：
 - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
 - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
 - 代码实现
 - Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

AI 改变了入门顺序：不必先做码农，可以先做 product

- 传统 **bottom-up** 路径：软件/计算机科学第一件事是先学工程、先学做码农。讲者给的理由是：**不理解代码，就无法理解整个过程**。
- 讲者认为 AI 来了以后变了：可以先从****产品经理 (product) ****开始，先了解整个过程。

- 他设想如果自己改 computer science/软件课程，**第一学期就可以做 product**。理由：
 1. 后面所有事情都是 Web；
 2. 你不需要控制质量，也不需要知道实现，因为你本来就不理解代码、不理解后面怎么运作；
 3. 但只要有好的想法，依然可以设计好的产品。
- （整理者串联）这等于把“理解过程/设计产品”与“实现/质量控制”拆开：AI 让入门可以先站在产品层，而不必先会写代码。

软件工程里的“文科式”方法不是 nonsense：checklist 用形式语义防遗漏和矛盾

- 讲者说，这个过程里诞生了很多软件工程智慧的方法和工具，例如 **checklist**，还有**决策表 / 决策树**。
- checklist 的规则是：只有所有勾都打上，才能进入下一个阶段。
- 讲者自嘲：自己像理科生，在软件工程里却遇到这种像学文科的东西，觉得对自己来说是 **nonsense**。
- 但他随后认为：这个形式赋予了它意义，**形式的语义避免了遗漏和矛盾**。
- 甚至会有检查活动要求把所有文档两两摆在一起审查一遍。讲者以程序员心态说“我才不想做这个事”，但他承认这种两两检查避免了 **gap** 的漂移，是智慧。
- 讲者也提醒：它失败过；后来在管理团队过程中形成了**管理学经验**，软件工程从业者也可以学、可以学好。

QuickCheck 的逻辑：让人写 property，由工具生成输入找反例

- 测试上有经典工作 **QuickCheck**；讲者称它可算整个测试领域的开山鼻祖之一。
- 机制：**让人描述性质 (property)**，由工具生成输入，寻找反例。
- 讲者推出的逻辑链：
 1. 做任何事都会错；
 2. 写代码、任何从上到下的翻译都会错；
 3. 从 **specification** 翻译到代码尤其会错，因为有些 specification 是隐含的，例如：程序不能 crash、不能有 null pointer、资源不能泄露；
 4. 这些隐含 specification 也要遵守；
 5. 所以可以在代码里写一个 **property**；

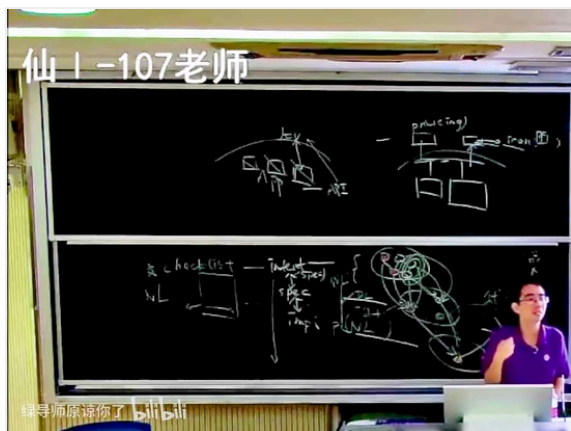
6. property 可以自动变成很多测试用例；

7. 如果全部通过，就对代码有更大的信心。

- 讲者说，现在做软件就是这样：写一份代码，你永远不知道它对不对；但做测试、观察结果，信心会多一点；独立的质量保障做得越多，信心越大。
- 讲者最后说，除非最终写了一个形式化证明，证明这个程序，信心可能更大。

小结（整理者串联）：讲者把 checklist、两两审查、QuickCheck 都解释为对抗错误与遗漏的机制；测试越多，对代码信心越大，形式化证明则信心更大。

第 26 部分



方法和工具的智慧

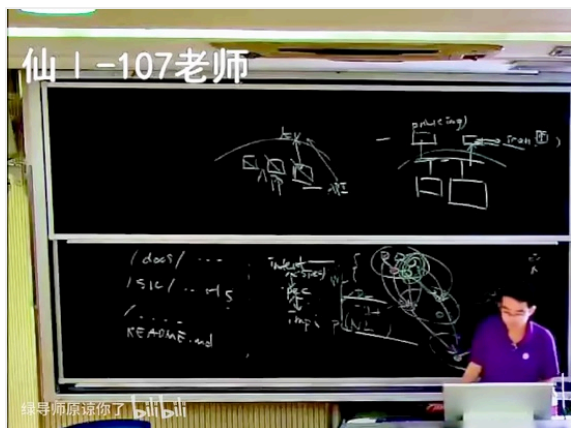
需求与测试：把规则变得可检查

- **决策表**：把条件组合与预期结果对应起来，暴露遗漏和矛盾。ISTQB §4.2.3
 - 例：学生优惠遇到满减，究竟能叠加，还是只能选一个？
- **基于性质的测试**：QuickCheck 让人描述性质，由工具生成输入，寻找反例。
 - 例：排序结果既要有序，也要保留所有元素及其出现次数；只检查“有序”，返回空数组也能过关。

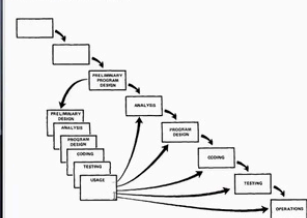
维护：改一句需求，会牵动什么？

- **需求追踪矩阵**：维护需求、设计与测试之间的关联，用于检查覆盖情况和分析变更影响。NASA
 - 例：“允许退课”改成“开课后一周内允许退课”，哪些设计和测试需要重新检查？

2 / 3



simulation the project manager is at the mercy of human judgment. With the simulation he can at least perform experimental tests of some key hypotheses and scope down what remains for human judgment, which in the area of computer program design (as in the estimation of takeoff gross weight, costs to complete, or the daily double) is invariably and seriously optimistic.



[Image #1: Flowchart showing a preliminary pilot program design proceeding through analysis, program design, coding, test, Figure 7. Step 3: Attempt to do the job twice - the first through result provides an early simulation of the final product, with feedback loops from usage back to earlier stages]
Figure summary: The diagram illustrates "doing the job
royce.md 475-504/779 63%

3. 它会让你以为“做两遍”是冗余重复。不是一 pilot model 的价值不是把同样的计算做两次，而是改变图的形状（消除到底哪个约束会“bind”的不确定性）。它是采样，不是重复。

四、用你的框架重讲 TDD，一句话

你既然不能删掉那条长程回边，剩下的唯一杆杆就是降低它的延迟。于是做预测执行：提前猜测试结果，用一次编辑就能回滚。

- 瀑布 = 分支在流水线末端才发现误预测，代价是 flush 整个项目
- TDD = 把验证推测到写实现的前一分钟执行，误预测的代价是一次 diff。

这个类比比“文档即规格”更精确地解释了 TDD 为什么有效：它没有减少依赖，只压缩了依赖的传播延迟。而它也和 Royce 的原始判据完全同构——既然回滚代价正比于距离，那就把距离降到零。

Recap: 把 Royce 读成依赖图 + 修改代价是第一性原理的正确读法 (Fig.3 是依赖局部性假设, Fig.4 是长程依赖, 逐步建议都是图变换), 但靠补三个补丁: 图要被设计出来的 (decision dependency / DSM), 存在图内推不出外部 oracle (validation ≠ verification), 代价系数由接口与文档质量决定而非拓扑; 由此 TDD 可重述为“不断长程依赖, 只把它改造成延迟为一次 diff 的推测执行”。

~/Projects/readspace
f13k 417k R198k CH94.8% \$0.025 2.8%/1.0M (auto) deepseek-flas

独立质量保障越多，信心越大；形式化证明更强

- 讲者主张：独立做的质量保障（QA）越多，你对软件质量的信心越大。
- 若最终写出形式化证明（原文“秦刷化”），证明这个程序，信心可能更大。
- 讲者认为这些就是软件工程里被研究的各种有意思的东西。

Traceability: 把文件堆变成需求—设计—代码可交叉检验的网络

- 例子：需求追踪矩阵；讲者前面讲过 **Traceability**（可追踪性），是软件工程里的重要概念。
- 讲者说 traceability 做的是把 [data defencies graph]（按上下文可能是 **data dependency graph**）里以前没有的东西补上。
- 从文件系统/项目角度看，软件就是很多文件：`docs/... .md`、`source/... .ts`、`README` 等。讲者说文件系统是“因为一开始大家用了文件而不得不做的 **work around**”，但它承载了整个软件。
- Traceability 要求用链接、文档或其他方式，让：
 - 每个代码 ↔ 每个需求点对得上；
 - 需求 ↔ 需求对得上；
 - 需求 ↔ 设计对得上。
- 如果这些都能对上，就有独立的、互相的交叉检验，于是对软件质量更有信心。
- 逻辑链：独立 QA ↑ → 交叉检验 ↑ → 质量信心 ↑。Traceability 提供的正是可交叉检验的关系。（整理者串联：它相当于在文件系统之上加一层“关系图/矩阵”，把文件堆中看不见的对应关系显式化。）

为什么需要它：人的能力有限，单个“最强码农”不够

- 讲者说这也是没办法，因为人的能力有限。
- 大学学软件工程时，很多人会以为：我要成为世界上最厉害的码农；如果我是最强码农，就能在软件企业找到好工作。讲者称这是社会给你的 **contract**。
- 但从资本家角度，或从完成任务角度看，更重要的是：没有一个码农能单独完成项目。关键变成“码农有好有坏，我得花多少钱雇多少码农，最终把项目交付”。
- 所以现在每个人都当**产品经理**，第一门课就要学做产品；要调动资源。
- 讲者举例：现在马上就有 **DeepSeek**；他认为 DeepSeek 绝对算相当资深的码农，不是本科毕业生级别。你可以有 **2500 并发的[底薪]**（原文如此；下文只说“手上有 2500 人”），相当于手上有 2500 人，怎么管？高中毕业/刚高考完时不会想到马上能管 2500 人，所以 **mindset 要改**。

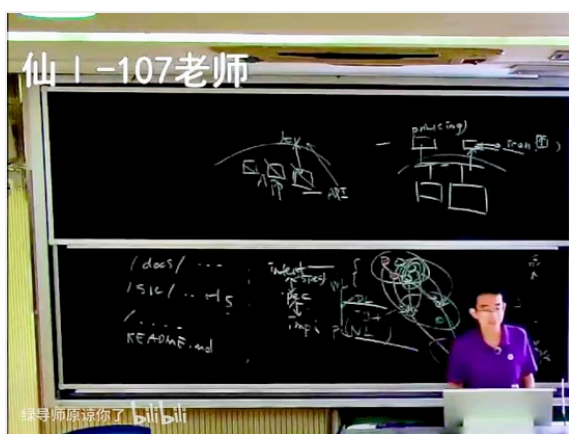
Design by Contract: 把霍尔三元组写进代码

- 在这个背景下，软件工程里诞生了很多好玩的东西。例如 **Design by Contract / 面向契约的编程 / 契约式编程**。
- 讲者提到 [IFL]（按 Design by Contract 上下文应为 **Eiffel**）语言，约在 1986 年。

- 其想法是：把霍尔三元组（Hoare triple；原文“霍尔三人组”）直接写到代码里。
- 讲者展示古早语法：例如 `function` 定义、赋值用冒号等号 `:=`。
- 讲者说这种思路一路沿袭下来。

小结：本段链条是：独立质量保障/形式化证明提升信心 → traceability 用需求、设计、代码的互相追溯做交叉检验 → 因为单个码农无法完成项目，必须管理多名“码农”（包括 AI） → 于是出现契约式编程等把契约写进代码的方法。

第 27 部分



把契约写进语言

上古语法：Eiffel

```
function(args) is
  -- Comment
  require
    Precondition
  do
    Routine
  ensure
    Postcondition
end
```

- 契约既是接口文档，也能在开启断言检查时检验本次执行。[Eiffel 文档](#)

今天依然活跃的方向

- 今天依旧是个不算太小众的领域：Dafny, Verus, ...
 - Dafny 内置契约与静态验证器；Verus 为 Rust 子集提供静态验证。
 - 它们进一步尝试证明：在给定假设下，实现满足写出的契约。

“先写契约，再写实现”：`require` / `ensure` 把设计问题提前逼出来

- 讲者用一段“上古语法”说明：函数定义里有 `function ... is`，然后有 `require`（前置条件）、`do`（代码实现）、`ensure`（做完后保证什么）。他说这条线后来被很多语言沿袭，很自然。
- 讲者鼓励：先不要写 `do` 里的流程控制，先把各个模块的文档、`precondition` / `postcondition` 写出来。
- 例子：账户取款。写 pre/post 时会自然关联设计：余额不足到底应该怎么办？
- 讲者指出：写第一个 MVP 时通常不会管这些，直接让它 crash，或假设这件事永远不会发生，把它排除在 `specification space` 之外。
- 但软件不断向后演进时，`contract` 就变得非常重要。所以他希望在编程语言里引入 `contract`。
- 逻辑链（讲者）：先写 pre/post → 自然逼出设计问题 → MVP 可暂时排除异常 → 软件演进后 `contract` 重要 → 希望 `contract` 进编程语言。

软件工程几十年在 spec 上横跳：自然语言 → 半自然语言 → 逻辑公式 → 编程语言

- 讲者总结：软工人这么多年做的事，就是在 spec 上横跳。
- 路径：自然语言 → 能接纳设计的半自然语言 → 有逻辑公式的语言（可以没有代码，单纯检验逻辑，比如中间函数调用的逻辑能不能成立）→ 现在的编程语言。
- 他认为这仍是今天非常活跃的方向。
- 讲者认为软件工程未来会大变，因为 LLM（转录作“恋爱”）来了：
 1. 自然语言对不齐的问题解决了；
 2. checklist 不好检查的问题也解决了；
 3. 只剩下一个 gap：我没有想清楚、没有说清楚。
- 讲者推论：一旦想清楚、说清楚，可能真的就都能解决。

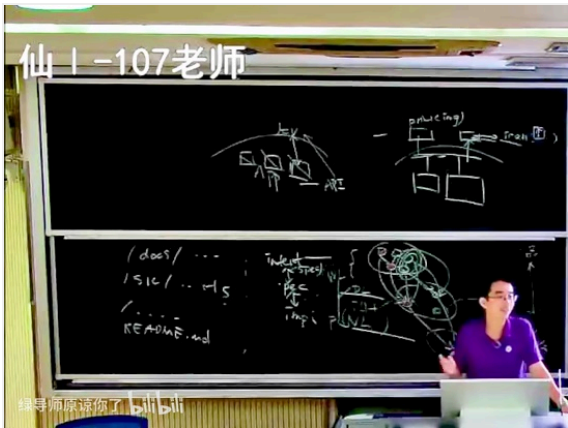
Eiffel 一脉的梦想：自动证明前后置条件，并把整个系统连到 intent

- 讲者说 Eiffel 催生了很多现在的项目，比如 Dafny、Viper 这些形式化验证工具（原音似“dafany virus”，工具名不确定）。
- 大家的梦想：
 1. 我写一个这样的程序，可以自动证明：函数在前置条件满足时，后置条件就会满足；无论我写什么，都能证明。
 2. 还要证明整个系统：把所有前置条件、后置条件连起来，能够满足我的 intent。
- （整理者串联：这与前面“把 contract 引入语言”的目标一致——目标不是人工测试，而是语言/验证器自动保证。）

讲者的预言：无法证明的业务 `assert` 应让程序编译失败；那还要 property-based test 吗？

- 讲者回忆：在 ChatGPT 出来之前，上操作系统课时就预言 30 年后，可能 2020 年左右，还是 2022 年左右：以后一个程序里若有 `assert` ——随便写的 `assert` 也可以，前置/后置条件可以写断言——只要这个 `assert` 和业务逻辑相关，写出来却不能证明，程序就不能编译；编译成功就 100% 保证它能被证明。
 - 他举的 `assert` 例子：函数要满足 linked list 的左边/右边关系——往左走再往后走，或往后走再往前走，总之能回到自己。
 - 于是讲者最后问：我还需要 property-based test 吗？
-

第 28 部分



把契约写进语言

上古语法: Eiffel

```
function(args) is
  -- Comment
  require
    Precondition
  do
    Routine
  ensure
    Postcondition
end
```

- 契约既是接口文档，也能在开启断言检查时检验本次执行。Eiffel 文档

今天依然活跃的方向

- 今天依旧是个不算大小众的领域: Dafny, Verus, ...
 - Dafny 内置契约与静态验证器; Verus 为 Rust 子集提供静态验证。
 - 它们进一步尝试证明: 在给定假设下, 实现满足写出的契约。

编译器若能“编译即证明”，property-based test 就可省去

- 讲者先提出一个设想/判断：把 property 直接写进程序，由工具（他用“编译”来指这个保证过程）100% 保证并证明它；如果达到这一点，就不需要 property-based test。
- 逻辑链：
 1. 目标：证明 property 成立。
 2. 手段：不靠随机/样例测试，而是把 property 写进去交给编译器/证明器。
 3. 条件：编译/证明过程 100% 保证。
 4. 结果：property-based test 变得不必要。
- 时间判断：讲者认为从现在看可能不需要 30 年；从“那个时间点”算，10 年或 15 年就能解决。

（整理者串联：这是讲者对形式化开发成熟速度的个人预期，不是已实现事实。）

UML 是“语义对齐”的集大成尝试：用中间语言统一面向对象世界

- 讲者说，这个过程里有一个绕不过去的集大成者：UML (Unified Modeling Language)。
- 它的想法：世界很复杂，不同层次要用不同语言。
 - 实现：C、Java。
 - 设计：也可用 IDL（原文作 IFL）或其他专门的设计领域语言。
 - 需求：DSL（原文作 DSR，按上下文推测为领域特定语言）。
- 于是 UML 问：能不能有一种中间语言，统一整个面向对象世界？

- 讲者认为 UML 做的是非常有野心的事：**语义对齐**（原文听作“羽翼的对齐”）。
（整理者串联：不同层次语言之间若没有共同语义，跨层转换就会漂移；UML 就是试图提供共同语义层。）

自然语言直连 latent space，所以天然对不齐

- 讲者认为，语义对齐恰恰是人类天生没做好的事：人类靠**自然语言**，而自然语言直接连接大脑的 **latent space**（原文作 latin space）。
- 例子：我说“我今天看到一个美女”，在我的 latent space 里有一个形象；在你们每个人的 latent space 里也各自构造一个美女，但**都不一样**，是对不齐的。
- 若要求把这个形象 3D 建模、用 Blender 画出来，就像让 GPT、另一个模型（原文作 DPPK）画美女，结果也不同。讲者开玩笑说，好像豆包生成的“美女”都长一样。
- 因此：**语言很难对齐**，因为语言直连各自大脑的 latent space。

数学语言提供可对齐基础，计算机语义因此绝对对齐

- 人类于是发明了**数学语言**。讲者认为这是非常好的东西：全人类能自然对齐。
- 原因：我们知道什么是自然数，因为它是从**公理**一点点构造出来的；公理非常小，所以人类基本可以对齐。
- 一旦对齐完成，用**皮亚诺公理**（Peano axioms）就能把整个计算机世界构建起来。
- 结果：在计算机世界里看到同样的东西，其**语义是绝对对齐的**。
（整理者串联：讲者把数学/计算机世界作为“可对齐”的参照，再反衬自然语言世界中的需求、设计为什么难对齐。）

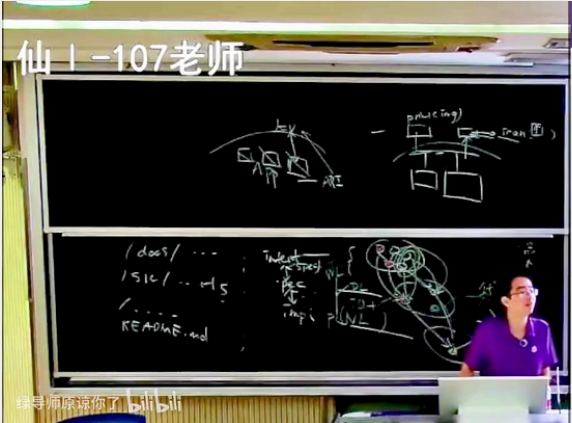
UML 的策略：剥离可对齐语义与不可对齐部分

- UML 想对齐的是自然语言世界里的东西：**需求、设计**等说不清的部分。
- 讲者明确说：**不可能 100% 对齐**；但 UML 希望把能对齐的部分和不能对齐的部分剥离出来。
- 手段：**定义模型的元素和它们的语义**。
- 有些东西有确定语义：
 - **类图**：语义确定。比如有一个 `class Student`，里面有一个 field 叫 `name`，那就是 `Student.name`；它可以变成源代码里的一个东西，因此有语义。

- 顺序图 (sequence diagram): 规定从前到后按某个顺序、时序执行; 这个时序先后是绝对对齐的, 不需要用自然语言描述。
- 反例: 如果不写 UML, 而用自然语言写先后, 当发生并发时, 到底谁先谁后、能不能并发, 就会有歧义。
- 讲者结论: UML 最大程度减少在没有代码时的语义漂移。

小结: 讲者把 UML 视为在需求/设计/实现之间做语义对齐的中间语言; 它不能消除全部自然语言歧义, 但能把模型语义固定的部分形式化, 减少无代码阶段的语义漂移。

第 29 部分



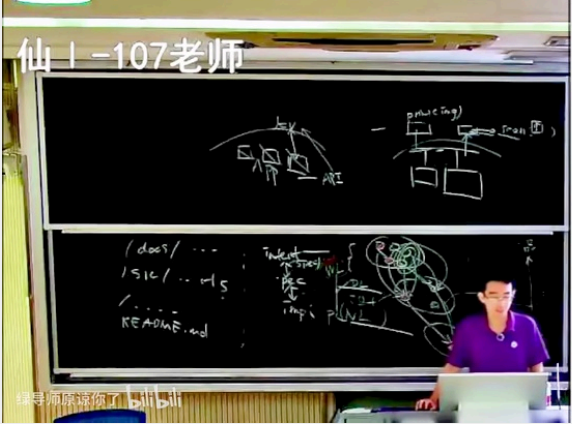
Unified Modeling Language (UML)

能否有一种中间语言, 统一整个面向对象世界?

- 人类的发展: 自然语言 (直接连接大脑 latent space) → 数学语言 (全世界自然对齐)
- UML 汇合了 Booch、OMT、OOSE 等方法: 统一记号与语义, 让团队和工具能使用共同的建模语言。OMG 历史说明

在没有代码时, 尽量减少语义漂移

- 通过定义模型元素和 (半形式化) 语义, 最大程度减少在没有代码时的语义漂移。
- 东西可能死了 (LLM 时代谁还用 UML?), 但设计时候的 first principles 一直指导我们。



More comments on reading

- 领域里“开天辟地”的经典, 即便今天已经过时, 但通常都是信息量巨大的
- 而且在今天的背景下非常容易理解

软件工程的方法和工具

软件工程: 为每个阶段都创建**方法学**和**工具**

- 尽可能减少返工概率
- 在没有 LLM 这种“万能编译器”的时代, 不同阶段需要不同的“语言”
- 需求分析: 文档、用例、界面原型 → “我到底想要什么”
- 系统设计: 实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
- 代码实现 (Computer Science 大部分课程集中在这里, 算是 bottom-up 的学习方法了)
- 在这里诞生了非常多有智慧的方法和工具

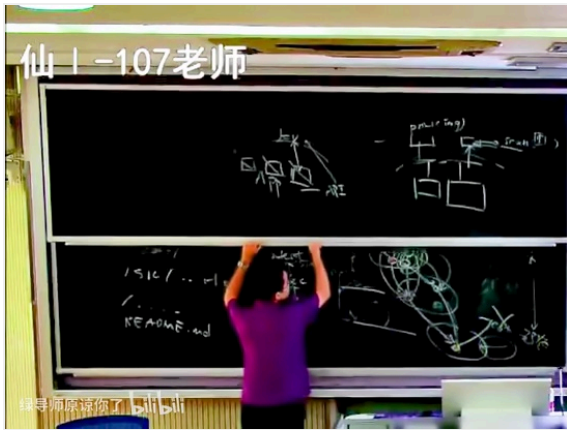
[[你可以列举一些, 后面我会讲 Eiffel 和 UML 的例子, 避开这两个]]

[Applying “design-by-contract”](https://ieeexplore.ieee.org/document/161279) by Bertrand Meyer's Eiffel

[[介绍一下契约编程]]

上古语法:

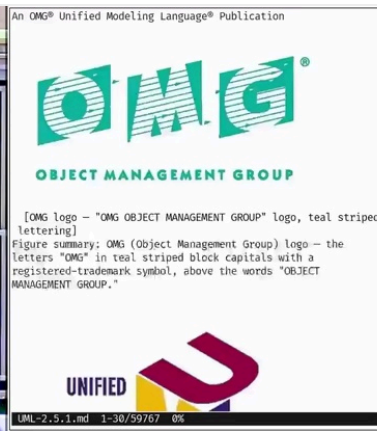
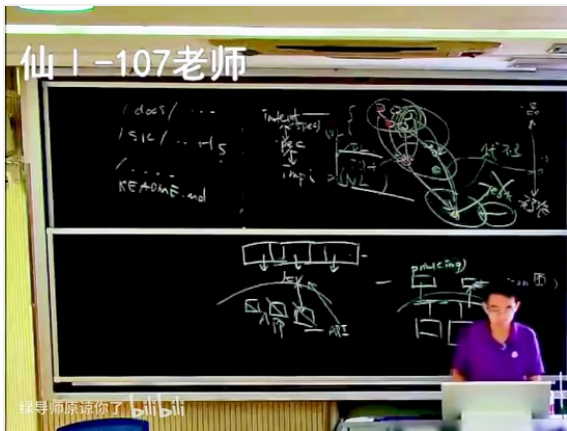
85/lect/index.md 129,63-43 64%



阅读 UML Spec

上大学的时候学过，感觉没学懂

- (怀疑老师可能也不太懂。)
- 依稀有一些模糊的印象
- PhD 时候也读了一些 papers
- Kimi 送了 ¥699 的套餐，直接让 AI 备课，花去 18% 五小时额度 (~¥5.3)



重的代价不是纯重算：文档质量决定了一个变化是局部重算，还是“junk the entire framework, even for relatively modest changes”。拓朴只给出可达集合，接口稳定性决定这次重算要付多少钱。你的框架如果不区分这两者，会漏掉 Royce 一半的论点——他花整整一节讲文档，讲的就是这个系数。

三、这个框架会误导你的地方

1. “最小化耦合”不是终点。耦合是功能的来源，你不能删边，只能把边改道到稳定接口上。把“降低耦合”当成唯一目标，会得到一堆什么都没做的模块。
2. 有些修改不是重算，是重新谈判。合同、验收、客户信任是不可逆提交 (irreversible commit)：它们不能在图上“重新求值”，只能重新协商。Step 5 的那些评审里程碑之所以重要，正是因为它们把不变量钉死在了合同层。
3. 它会让你以为“做两遍”是冗余重算。不是——pilot model 的价值不是把同样的计算做两次，而是改变图的形状（消掉“到底哪个约束会 bind”的不确定性）。它是采样，不是重算。

四、用你的框架重讲 TDD，一句话

你既然不能删掉那条长程回边，剩下的唯一杠杆就是降低它的延迟。于是做推测执行：提前推测验证结果，用一次编辑就能回滚。

- 瀑布 = 分支在流水线末端才发现误预测，代价是 flush 整个项目
- TDD = 把验证推测到写实现的同一分钟执行，误预测的代价是一次 diff。

这个类比“文档即规格”更精确地解释了 TDD 为什么有效：它没有减少依赖，只压缩了依赖的传播延迟。而它也和 Royce 的原始判据

没有代码时，“谁先谁后 / 能否并发”有歧义；UML 的价值是压住语义漂移

- 讲者先指出：在没有代码的时候，“到底谁先谁后”“能不能并发”本身就是有歧义的。
- 讲者认为 UML（转录作“UML9”）消除了这个问题，或至少最大程度减少了“没有代码时的语义漂移”。
- 讲者把 UML 的设计假设讲成一条链：
 1. 自然语言和形式语言不能对齐；
 2. 所以需要一层一层的中间语言；
 3. （整理者串联）UML 就被放在这种中间语言/建模层的位置上，用来在代码之前减少语义歧义。
- 但讲者的判断是：今天 UML 已经死了。
 - 理由链：AI 时代，自然语言可以直接和任何东西对齐，“反正 AI 就可以给你问” → 不再必须依赖 UML 这种中间层 → UML 变成历史里的东西。
 - 不过，讲者仍认为 UML 设计时的 first principles 一直在指导我们。

AI 重读 700 页 UML spec：分块、多 subagent、带 overlap，把长文档变成课件

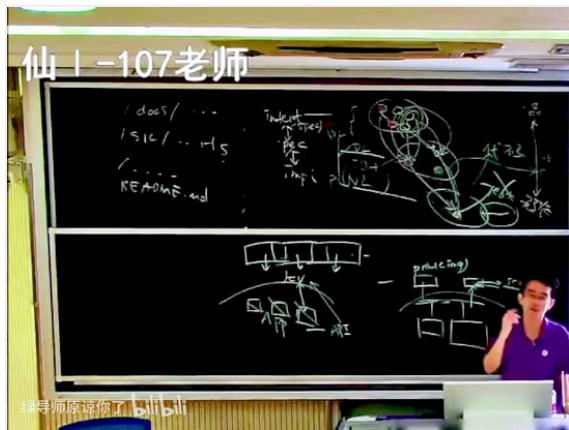
- 讲者让 AI 帮他重温/重读 UML spec。

- 文档规模：**700 页**；讲者说自己的工具能 scale 长文档，几千页也没问题，体验接近 PDF。
- 插曲：**KIMI** 看到他的视频后送了他 **699 套餐**；讲者说这“可能算一个广告”。
- 他把整个 UML spec dump 给 AI，然后直接让 AI 帮他备课。
- 具体流程（讲者自述）：
 1. 分 **subagent**，分 **division**（转录作 CIVISION），每个 agent 读 **50 页**；
 2. 把长文档切成带 **overlap** 的块，因为边界处总会有一点内容；
 3. 让每个 agent 提取文档里的 **软件工程智慧**；
 4. 剔除不符合 AI 时代的不必要内容；
 5. 把他自己的 **lecture notes** 全部给 AI；
 6. 让 AI 拿出来、总结成课件。
- 成本：这次操作花了 **18% 的五小时额度**，按 699 套餐折算约 **5.3 元**。
- 讲者评价：这是一个非常恐怖、也蛮可怕的事情。
 - **（整理者串联）** 可怕之处在于：700 页甚至几千页的规范，已经被拆成可并行处理、可提炼、可重组为课件的流程。

更好的模型可能同价甚至更省，于是学习也变成 token 经济问题

- 讲者推断：当你拥有更好的模型，比如 **GPT-6（转录作 GP6）**，价格可能差不多，因为它很省 token；token（转录说“GPU 的 token”）真的非常省，可能同价得到好得多的质量。
 - 这意味着：学生群体有没有好模型，**学习效率和学习成本**会拉开很大差距。
 - 讲者认为，如果有一个 **100 美元的 Codex**，对大家学习来说还算够用。
 - 什么会烧 token：
 - 维护大项目会烧 token；
 - 大项目要往大上下文里载入很多东西；
 - **100 万上下文**很快会被填满，然后就很费 token。
 - 但学习流程不必像他这次一样把整个 UML spec 放进来：
 - 可以用 **digested version**（消化/摘要版）；
 - 不必让上下文长期背负完整规范。
 - 讲者最后把问题留下：这值得我们思考。
-

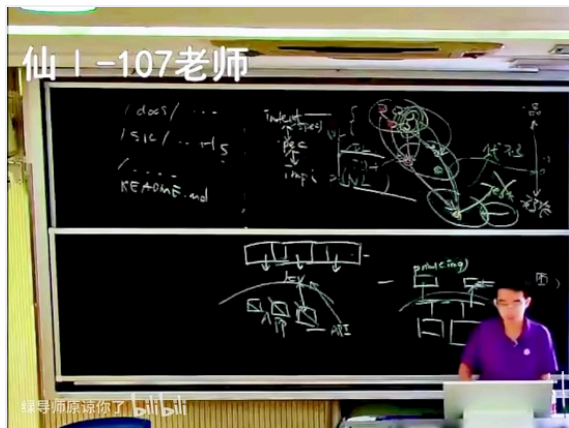
第 30 部分



阅读 UML Spec

上大学的时候学过，感觉没学懂

- (怀疑老师可能也不太懂。)
- 依稀有一些模糊的印象
- PhD 时候也读了一些 papers
- Kimi 送了 ¥699 的套餐，直接让 AI 备课，花去 18% 五小时额度 (~¥53)



重的代价不是纯重算：文档质量决定了一个变化是局部重算，还是“junk the entire framework, even for relatively modest changes”。拓朴只给出可达集合，接口稳定性决定这次重算要付多少钱。你的框架如果不区分这两者，会漏掉 Royce 一半的论点——他花整整一节讲文档，讲的就是这个系数。

三、这个框架会误导你的地方

1. “最小化耦合”不是终点。耦合是功能的来源，你不能删减，只能把边改道到稳定接口上。把“降低耦合”当成唯一目标，会得到一堆什么都没做的模块。
2. 有些修改不是重算，是重新谈判。合同、验收、客户信任是不可逆提交 (irreversible commit)：它们不能在图上“重新求值”，只能重新协商。Step 5 的那些评审里程碑之所以重要，正是因为它们把不变量钉死在了合同层。
3. 它会让你以为“做两遍”是冗余重算。不是——pilot model 的价值不是把同样的计算做两次，而是改变图的形状（消掉到底哪个约束会 bind 的不确定性）。它是采样，不是重算。

四、用你的框架重讲 TDD，一句话

你既然不能删掉那条长程回边，剩下的唯一杠杆就是降低它的延迟。于是做推测执行：提前猜测验证结果，用一次编辑就能回滚。

- 瀑布 = 分支在流水线末端才发现误预测，代价是 flush 整个项目
- TDD = 把验证推测到写实现的同一分钟执行，误预测的代价是一次 diff。

这个类比比“文档即规格”更精确地解释了 TDD 为什么有效：它没有减少依赖，只压缩了依赖的传播延迟。而它也和 Royce 的原始判据

不必把整个 UML spec 塞进上下文：可用 digested version

- 讲者说，你其实不需要像他那样把整个 UML spec 都放进来。
- 可以用 digested version。
- 由此讲者提出一个现实问题：以后还学得起吗？
- 如果让所有人都实现教育平权，我们并没有那么多算力和资源；那怎么办？讲者明确说 we don't know，并认为这可能是一个社会性问题。

逻辑链：不必全塞 spec → 可用 digested version → 但若人人教育平权，算力/资源不够 → 讲者承认不知怎么办，只把它当作社会性问题。

796 页 UML spec 从头到尾只回答：哪些 intention 必须定死，哪些灵活

- 讲者回到 UML，说这是他对 UML 的总结。
- 一句话概括：796 页的 spec 从头到尾回答一个问题——哪些 intention 必须定死，哪些灵活。
- 它构造了一个 形式语言 / 半形式语言，用来告诉你：
 - 哪些东西可以在语义上确定下来；
 - 哪些可以不用它来解决。
- （整理者串联）这相当于在规范层面把“必须被确定、对齐的语义”与“可以留给别的机制处理的部分”区分开。

逻辑链：796 页 spec → 回答哪些 intention 必须定死、哪些灵活 → 用形式/半形式语言区分语义可确定与可不由其解决的部分。

对齐不只发生在人与人之间，也发生在你和下一时刻的自己之间

- 讲者把这个问题说成对齐问题。
 - 对齐问题首先存在于人和人之间。
 - 但更麻烦的是，你甚至要和下一时刻的自己对齐。
 - 原因：你不 **deterministic**；你今天想的和明天想的就不一样。
 - 所以你会跟自己对不起（转写为“对不起”）。
 - 讲者说，有趣的东西大家可以自己回去看 slides。
-